UNIVERSITAS GUNADARMA

FAKULTAS TEKNOLOGI INDUSTRI

PENULISAN ILMIAH

ANALISIS KOMPARATIF PERFORMA LANDING PAGE ANTARA ASTRO JS DAN REACT JS MENGGUNAKAN GOOGLE LIGHTHOUSE, PAGESPEED INSIGHTS DAN JMETER

DISUSUN OLEH :

NAMA	**:**	**Farhan Harits Prakoso**
NPM	**:**	**50423457**
PROGRAM STUDI	**:**	**Informatika**
PEMBIMBING	**:**	**Dwi Widiastuti, S.Kom., MMSI.**

JAKARTA

2026

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya bertanda tangan di bawah ini,

Nama	:	Farhan Harits Prakoso
NPM	:	50423457
Judul Penulisan Ilmiah	:	Analisis Komparatif Performa Landing Page Antara Astro JS dan React JS Menggunakan Google Lighthouse, PageSpeed Insights, dan JMeter
Tanggal Sidang	:	1 September 2026
Tanggal Lulus	:	1 September 2026

Dengan ini saya menyatakan bahwa hasil tulisan ilmiah yang saya hasilkan adalah hasil karya saya sendiri dan benar-benar asli. Apabila dikemudian hari ternyata Karya Tulus Ilmiah ini merupakan hasil plagiarisme atau penjiplakan karya orang. Saya siap bertanggung jawab dan menerima sanksi sesuai peraturan dan menerima sanksi sesuai peraturan perundang-undangan Universitas Gunadarma. Demikian pernyataan ini saya buat dalam keadaan sadar dan tidak dipaksakan.

Jakarta, 16 April 2026

Farhan Harits Prakoso

LEMBAR PENGESAHAN

Judul Penulisan Ilmiah	:	Analisis Komparatif Performa Landing Page Antara Astro JS dan React JS Menggunakan Google Lighthouse, PageSpeed Insights, dan JMeter
Nama	:	Farhan Harits Prakoso
NPM	:	50423457
Program Studi	:	Informatika
Tanggal Sidang	:	1 September 2026
Tanggal Lulus	:	1 September 2026

Menyetujui

Pembimbing PI	KaSubbag Sidang
(Dwi Widiastuti, S.Kom., MMSI.)	(Dr. Achmad Fahrurozi, S.Si., M.Si.)

Ketua Program Studi

(Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc)

ABSTRAK

Farhan Harits Prakoso, 50423457

ANALISIS KOMPARATIF PERFORMA LANDING PAGE ANTARA ASTRO JS DAN REACT JS MENGGUNAKAN GOOGLE LIGHTHOUSE, PAGESPEED INSIGHTS, DAN JMETER

Penulisan Ilmiah. Informatika. Fakultas Teknologi Industri. Universitas Gunadarma. 2026

Kata kunci: *Astro JS*, *Google Lighthouse*, *JMeter*, *PageSpeed Insights*, *React JS*

(x + 65)

React JS telah menjadi pilihan utama pengembang frontend global dalam membangun antarmuka web modern melalui pendekatan Client-Side Rendering (CSR). Namun, pendekatan ini menimbulkan tantangan performa pada skenario landing page, khususnya dalam hal kecepatan pemuatan dan efisiensi eksekusi JavaScript di sisi browser. Framework Astro JS hadir dengan konsep Islands Architecture yang meminimalkan pengiriman JavaScript ke client, menawarkan paradigma yang lebih optimal untuk kebutuhan konten statis. Penelitian ini bertujuan untuk melakukan analisis komparatif performa antara React JS dan Astro JS melalui studi kasus landing page dengan aset media berat. Metode yang digunakan adalah eksperimen kuantitatif dengan membangun dua prototipe landing page identik, diuji menggunakan tiga instrumen: Google Lighthouse dan PageSpeed Insights untuk mengukur metrik Web Vitals (FCP, LCP, TBT, CLS, SI), serta Apache JMeter untuk mengukur Response Time, Throughput, dan Error Rate di bawah simulasi beban pengguna. Hasil penelitian menunjukkan bahwa Astro JS secara konsisten unggul pada metrik Web Vitals, dengan FCP dan Speed Index yang lebih cepat 22.6% pada mode mobile dan 28.6% pada mode desktop melalui Google Lighthouse, serta LCP yang lebih cepat 47.8% pada mode mobile dan 39.7% pada mode desktop melalui PageSpeed Insights. Sebaliknya, pada pengujian beban menggunakan Apache JMeter, React JS mencatat response time yang sedikit lebih rendah dibandingkan Astro JS dengan selisih 6.2–11.8 milidetik pada seluruh skenario, sementara kedua prototipe sama-sama menunjukkan stabilitas yang baik dengan error rate 0.00% hingga simulasi 300 pengguna virtual. Perbedaan arah hasil ini bukan merupakan kontradiksi, melainkan bukti bahwa kedua instrumen mengukur dimensi performa yang berbeda: JMeter mengevaluasi kecepatan respons di level server, sedangkan Lighthouse dan PageSpeed Insights mengevaluasi kecepatan rendering dari perspektif pengalaman pengguna di browser. Penelitian ini memberikan referensi teknis bagi developer dalam menentukan pilihan teknologi frontend yang sesuai dengan karakteristik kebutuhan performa dan fungsionalitas website yang dibangun.

Daftar Pustaka (2017 – 2025)

KATA PENGANTAR

Puji dan Syukur dipanjatkan ke hadirat Allah SWT, atas berkah rahmat dan karunia-Nya, serta doa dan motivasi dari berbagai pihak sehingga dapat terselesaikannya tugas Penulisan Ilmiah yang berjudul "**ANALISIS KOMPARATIF PERFORMA LANDING PAGE ANTARA ASTRO JS DAN REACT JS MENGUNAKAN GOOGLE LIGHTHOUSE, PAGESPEED INSIGHTS, DAN JMETER"** dengan sebaik-baiknya.

Dalam segala kerendahan hati, perkenankanlah ucapan terima kasih ini dihaturkan atas dorongan dan bantuan yang diterima, sehingga dapat menyelesaikan Penulisan Ilmiah ini, ucapan terima kasih dihaturkan kepada:

1. Prof. Dr. Hj. E. S. Margianti, SE, MM., selaku rektor Universitas Gunadarma.
2. Prof. Dr. Ing. Adang Suhendra S.Si., SKom., MSc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, SKom., MSc., selaku Ketua Program Studi Informatika, Universitas Gunadarma.
4. Dr. Achmad Fahrurozi, S.Si, M.Si., selaku Kepala Subbagian Sidang Penulisan Ilmiah Fakultas Teknologi Industri Universitas Gunadarma.
5. Dwi Widiastuti, S.Kom., MMSI., selaku dosen pembimbing yang telah banyak memberikan waktu dan sarannya selama pembuatan Penulisan Ilmiah ini.
6. Kedua orangtua dari penulis, Bapak Widodo Poerwanto dan Ibu Triana Ardiyanti, yang memberikan semagat, berkat doa serta restu keduanya kepada penulis, maka Penulisan Ilmiah ini dapat terselesaikan.
7. Teman-teman dekat penulis, Febri, Doni, Alip, Daffa, Baskara, Jeki, Fazrin, dan Giyas, yang telah menjadi tempat berbagi cerita, memberikan semangat, dan insight yang membantu selama pengerjaan Penulisan Ilmiah ini.
8. Teman-teman Kelas 3IA20 serta pihak lainnya yang secara langsung ikut memberikan dukungan.
9. Semua pihak yang tidak dapat penulis sebutkan satu persatu. Terima kasih atas bantuan, saran, kritik, dan masukan yang telah diberikan kepada penulis

Penulisan Ilmiah ini diharapkan dapat bermanfaat bagi aktivitas akademis Program Studi Informatika Fakultas Teknologi Industri, Universitas Gunadarma, dan menyadari atas segala keterbatasan pengetahuan dan kemampuan diharapkan adanya kritik yang membangun dan semoga Penulisan Ilmiah ini dapat bermanfaat untuk orang banyak.

Penulis menyadari bahwa Penulisan Ilmiah ini masih memiliki banyak kekurangan, baik dari sisi materi, teknis penulisan, maupun cara penyajiannya. Oleh karena itu, segala bentuk kritik dan saran yang membangun sangat penulis nantikan sebagai bahan evaluasi dan perbaikan di kemudian hari.

Akhir kata, penulis berharap Penulisan Ilmiah ini dapat memberikan manfaat, baik bagi masyarakat luas maupun secara khusus bagi rekan-rekan mahasiswa Universitas Gunadarma. Penulis juga memohon maaf sebesar-besarnya apabila terdapat kekeliruan kata atau penyampaian yang kurang berkenan selama penulisan ini.

Bekasi, 19 Juli 2026

Farhan Harits Prakoso

DAFTAR ISI

UNIVERSITAS GUNADARMA 1
PERNYATAAN ORISINALITAS DAN PUBLIKASI i
LEMBAR PENGESAHAN ii
ABSTRAK iii
KATA PENGANTAR iv
DAFTAR ISI vi
DAFTAR TABEL viii
DAFTAR GAMBAR ix
DAFTAR LAMPIRAN x
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Batasan Masalah 4
1.3 Tujuan Penelitian 4
1.4 Metode Penelitian 5
1.5 Sistematika Penulisan Ilmiah 6
BAB II 7
TINJAUAN PUSTAKA 7
2.1 Metode Kuantitatif Komparatif 7
2.2 Landing Page 7
2.3 React JS 9
2.4 Astro JS 10
2.5 Web Vitals 12
2.5.1 First Contentful Paint (FCP) 12
2.5.2 Largest Contentful Paint (LCP) 13
2.5.3 Total Blocking Time (TBT) 14
2.5.4 Cumulative Layout Shift (CLS) 14
2.5.5 Speed Index 15
2.6 Google Lighthouse 15
2.7 PageSpeed Insights 17

2.8 Apache JMeter 18
BAB III 21
PEMBAHASAN 21
3.1 Gambaran Umum 21
3.2 Analisis Kebutuhan Sistem 24
3.2.1 Kebutuhan Fungsional 24
3.2.2 Kebutuhan Non-Fungsional 25
3.2.3 Spesifikasi Perangkat Keras 25
3.2.4 Spesifikasi Perangkat Lunak 26
3.3 Perancangan Prototipe 26
3.3.1 Deskripsi dan Tema Prototipe 27
3.3.2 Spesifikasi Teknis Prototipe 27
3.4 Implementasi Prototipe 29
3.4.1 Implementasi Prototipe Astro JS 29
3.4.2 Implementasi React JS 32
3.4.3 Perbandingan Build Output 35
3.4.4 Deployment 36
3.5 Hasil Prototipe 37
3.6 Skenario Pengujian Peforma 43
3.6.1 Skenario Pengujian Google Lighthouse 43
3.6.2 Skenario Pengujian Pagespeed Insights 44
3.6.3 Skenario Pengujian Apache JMeter 45
3.7 Hasil & Analisis Pengujian 46
3.7.1 Hasil Pengujian Google Lighthouse 46
3.7.2 Hasil Pengujian Pagespeed Insights 49
3.7.3 Hasil Pengujian JMeter 52
BAB IV 59
PENUTUP 59
4.1 Kesimpulan 59
4.2 Saran 61
DAFTAR PUSTAKA 63
LAMPIRAN L-1

DAFTAR TABEL

Tabel 3.1 Spesifikasi Perangkat Keras 25
Tabel 3.2 Spesifikasi Perangkat Lunak 26
Tabel 3.3 Struktur Konten Prototipe Landing Page 28
Tabel 3.4 Perbandingan Build Output 35
Tabel 3.5 Konfigurasi Pengujian Google Lighthouse 43
Tabel 3.6 Konfigurasi Pengujian Pagespeed Insights 44
Tabel 3.7 Konfigurasi Pengujian Apache Jmeter 45
Tabel 3.8 Hasil Pengujian Lighthouse Astro JS Mode Mobile 46
Tabel 3.9 Hasil Pengujian Lighthouse React JS Mode Mobile 47
Tabel 3.10 Hasil Pengujian Lighthouse Astro JS Mode Desktop 48
Tabel 3.11 Hasil Pengujian Lighthouse React JS Mode Desktop 48
Tabel 3.12 Hasil Pengujian Pagespeed Astro JS Mode Mobile 49
Tabel 3.13 Hasil Pengujian Pagespeed React JS Mode Mobile 49
Tabel 3.14 Hasil Pengujian Pagespeed Astro JS Mode Desktop 51
Tabel 3.15 Hasil Pengujian Pagespeed React JS Mode Desktop 51
Tabel 3.16 Hasil Pengujian Jmeter Skenario 1 Astro JS 52
Tabel 3.17 Hasil Pengujian Jmeter Skenario 1 React JS 52
Tabel 3.18 Hasil Pengujian Jmeter Skenario 2 Astro JS 54
Tabel 3.19 Hasil Pengujian Jmeter Skenario 2 React JS 54
Tabel 3.20 Hasil Pengujian Jmeter Skenario 3 Astro JS 55
Tabel 3.21 Hasil Pengujian Jmeter Skenario 3 React JS 55
Tabel 3.22 Rekapitulasi Hasil Pengujian JMeter 57

DAFTAR GAMBAR

Gambar 1.1 Penggunaan Web Framework dan Teknologi Web (2025) 1
Gambar 1.2 Tren Penggunaan Front-end Framework (2016–2024) 2
Gambar 1.3 Penggunaan Front-end Framework used at Work (2024) 2
Gambar 3.1 Tahapan Perancangan 22
Gambar 3.2 Struktur Direktori Astro JS 32
Gambar 3.3 Struktur Direktori React JS 34
Gambar 3.4 Dashboard Vercel 37
Gambar 3.5 Hero Section 37
Gambar 3.6 Service Section 38
Gambar 3.7 Gallery Section 38
Gambar 3.8 Video Tour Section 39
Gambar 3.9 Team Section 39
Gambar 3.10 Testimonials Section 40
Gambar 3.11 Stats Section 40
Gambar 3.12 Virtual Tour Section 41
Gambar 3.13 Map & Contact Section 41
Gambar 3.14 FAQ Section 42
Gambar 3.15 Footer Section 42
Gambar 3.16 Visualisasi Hasil Pengujian Lighthouse Mode Mobile 47
Gambar 3.17 Visualisasi Hasil pengujian Lighthouse Mode Desktop 48
Gambar 3.18 Visualisasi Hasil Pengujian Pagespeed Mode Mobile 50
Gambar 3.19 Visualisasi Hasil Pengujian Pagespeed Mode Desktop 51
Gambar 3.20 Visualisasi Hasil Pengujian Jmeter Skenario 1 53
Gambar 3.21 Visualisasi Hasil Pengujian Jmeter Skenario 2 54
Gambar 3.22 Visualisasi Hasil Pengujian Jmeter Skenario 3 56

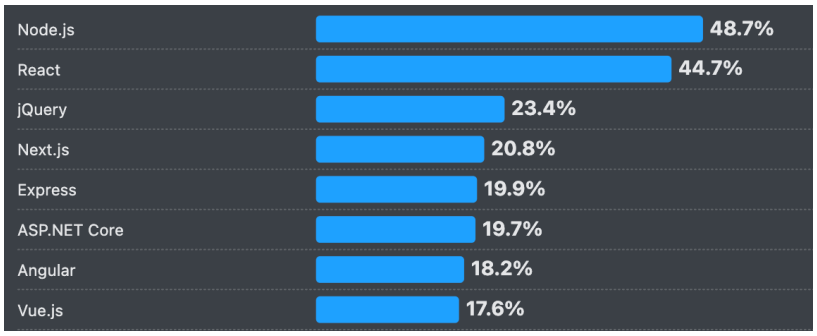
DAFTAR LAMPIRAN

Lampiran 1. Listing Program L-1

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi *Web* membuat *landing page* menjadi salah satu komponen terpenting dalam strategi digital sebuah produk atau layanan. Halaman ini adalah titik pertama yang dilihat pengguna, dan kecepatan pemuatannya sangat menentukan apakah mereka akan bertahan atau langsung meninggalkan halaman. Menurut Google & SOASTA (2017), keterlambatan pemuatan halaman sebesar satu detik saja sudah cukup untuk menurunkan tingkat konversi hingga 20%. Di sisi lain, *React JS* telah berkembang menjadi *library JavaScript* paling dominan yang digunakan oleh pengembang *frontend* global. Berdasarkan Gambar 1.1 *Stack Overflow Developer Survey* (2025), sebesar 44,7% *developer* profesional menggunakan *React JS* sebagai *framework* utama mereka, menjadikannya standar industri dalam membangun antarmuka web modern.

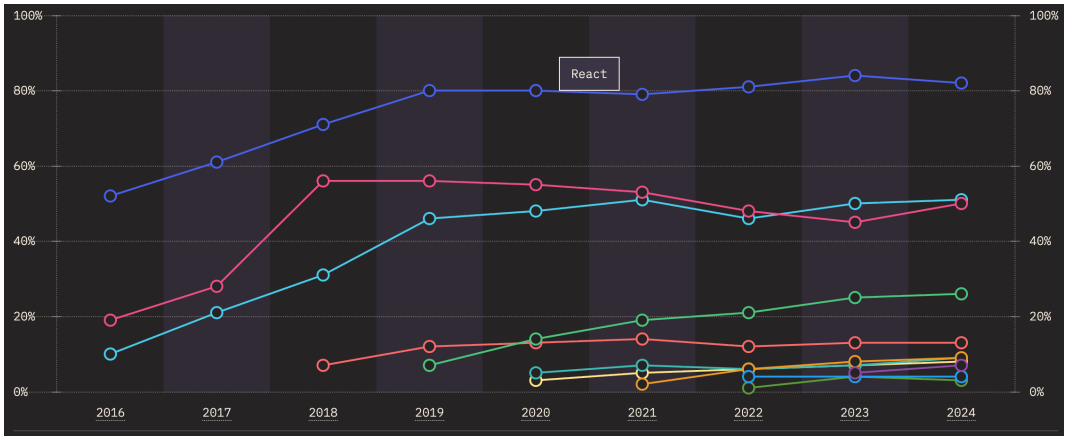


Gambar 1.1 Penggunaan Web Framework dan Teknologi Web (2025)

Sumber: *Stack Overflow Developer Survey*, 2025

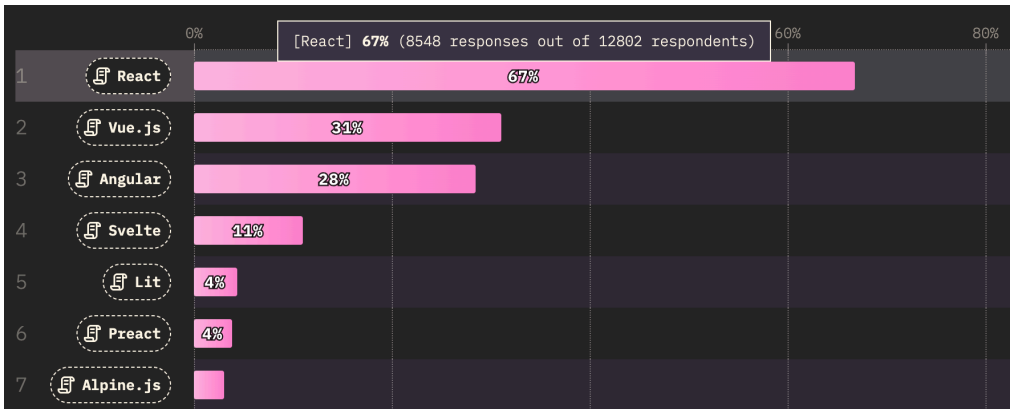
Hal ini diperkuat oleh *State of JavaScript* (2024), sebagaimana ditunjukkan pada Gambar 1.2 dan Gambar 1.3, bahwa *React JS* telah mempertahankan posisi dominannya secara konsisten sejak tahun 2016 hingga 2024 dengan tingkat penggunaan mencapai lebih dari 80% responden. Selain itu, *React JS* juga tercatat digunakan secara profesional oleh 67% responden atau setara 8.548 dari 12.802 *developer* yang disurvei lebih dari dua kali lipat *Vue.js* (31%) dan *Angular* (28%).

Tingginya angka adopsi ini kemudian mendorong qbanyak pengembang menggunakan *React JS* untuk membangun berbagai jenis *website* tanpa terkecuali, termasuk *landing page*, meskipun karakteristik arsitekturnya belum tentu sesuai dengan kebutuhan halaman tersebut.



Gambar 1.2 Tren Penggunaan Front-end Framework (2016–2024)

Sumber: State of JavaScript*, 2024*



Gambar 1.3 Penggunaan Front-end Framework used at Work (2024)

Sumber: State of JavaScript*, 2024*

Pendekatan *React JS* dalam merender halaman menggunakan metode *Client-Side Rendering* (CSR), di mana antarmuka dibangun sepenuhnya di sisi *browser* menggunakan JavaScript. Pendekatan ini unggul untuk aplikasi yang bersifat

interaktif dan dinamis, namun kurang optimal diterapkan pada *landing page* yang sebagian besar kontennya bersifat statis, karena *browser* harus mengunduh dan mengeksekusi seluruh *bundle* JavaScript sebelum konten dapat ditampilkan. Konsep CSR beserta dampaknya terhadap performa dijelaskan lebih lanjut pada sub-bab 2.3.

Persoalan ini bukan hal baru dalam pengembangan *web*. Muncul pendekatan rendering alternatif seperti *Static Site Generation* (SSG), di mana halaman dirender pada saat proses *build* sehingga *browser* langsung menerima HTML yang sudah jadi tanpa perlu menunggu eksekusi JavaScript (Zaidan & Suyatno, 2025). Riset *State of JavaScript* (2024) mencatat bahwa Astro JS menempati posisi pertama sebagai *framework* alternatif yang paling banyak disebut komunitas pengembang global, mengindikasikan adanya pergeseran kesadaran developer terhadap pentingnya memilih teknologi yang sesuai karakteristik proyek. Astro JS menerapkan pendekatan SSG melalui konsep *Islands Architecture*, yang meminimalkan pengiriman JavaScript ke sisi *client* (Astro, 2024). Konsep SSG dan *Islands Architecture* dijelaskan lebih rinci pada sub-bab 2.4.

Klaim keunggulan Astro JS tersebut secara teoritis menjanjikan payload yang lebih kecil dan pemuatan halaman yang lebih cepat, namun perlu dibuktikan secara empiris, bukan sekadar diasumsikan dari spesifikasi teknis semata. Oleh karena itu, penelitian ini dilakukan untuk membandingkan performa *landing page* yang dibangun menggunakan React JS dan Astro JS secara langsung melalui pengujian terstandar menggunakan Google Lighthouse, PageSpeed Insights, dan Apache JMeter pada studi kasus *landing page* dengan aset media berat, guna memberikan acuan teknis yang objektif bagi para pengembang dalam memilih teknologi *frontend* yang paling sesuai dengan kebutuhan proyeknya. Meskipun studi komparatif *framework frontend* telah banyak dilakukan, belum ada kajian empiris spesifik yang membandingkan arsitektur *SSG-Islands Architecture* (Astro JS) dan *CSR* (React JS) pada *landing page* berkonten media berat secara simultan menggunakan Web Vitals (sisi klien) dan JMeter (sisi peladen).

1.2 Batasan Masalah

Penelitian ini membatasi kajiannya pada beberapa aspek berikut :

a. Teknologi yang dibandingkan adalah Astro JS yang dikonfigurasi dalam mode *Static Site Generation* (SSG) dengan penerapan *Islands Architecture* untuk komponen interaktif, dan React JS yang dijalankan dengan pendekatan *Client-Side Rendering* (CSR) menggunakan Vite sebagai *build tool*.
b. Kedua teknologi diuji melalui pembangunan dua prototipe landing page yang identik dari sisi konten, struktur halaman, dan aset media yang digunakan, sehingga perbedaan hasil pengujian dapat sepenuhnya diatribusikan pada perbedaan arsitektur masing-masing framework.
c. Pengujian performa dilakukan menggunakan Google Lighthouse dan PageSpeed Insights untuk mengukur metrik Web Vitals, meliputi *First Contentful Paint* (FCP), *Largest Contentful Paint* (LCP), *Total Blocking Time* (TBT), *Cumulative Layout Shift* (CLS), dan *Speed Index* (SI).
d. Pengujian beban dilakukan menggunakan Apache JMeter untuk mengukur *Response Time*, *Throughput*, dan *Error Rate* melalui simulasi beban pengguna secara bersamaan.
e. Penelitian ini tidak mencakup pengujian pada kondisi jaringan yang bervariasi (seperti 3G/4G/5G), optimasi kode secara manual di luar konfigurasi default masing-masing framework, maupun perbandingan terhadap framework atau library lain di luar React JS dan Astro JS.

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk membuktikan secara empiris perbedaan performa antara *React JS* dan *Astro JS* dalam konteks pembangunan *landing page* dengan aset media berat. Secara lebih rinci, tujuan penelitian ini adalah sebagai berikut.

a. Mengukur dan membandingkan performa *landing page* yang dibangun menggunakan *React JS* dan *Astro JS* berdasarkan metrik *Web Vitals*, meliputi

First Contentful Paint (FCP), *Largest Contentful Paint* (LCP), *Total Blocking Time* (TBT), dan *Cumulative Layout Shift* (CLS) *Speed Index* (SI), melalui instrumen *Google Lighthouse* dan *PageSpeed Insights*.

b. Mengevaluasi ketahanan dan stabilitas sistem dari kedua *framework* ketika menghadapi simulasi beban pengguna secara bersamaan, diukur melalui metrik *Response Time*, *Throughput*, dan *Error Rate* menggunakan *Apache JMeter*.

c. Memberikan rekomendasi teknis berbasis data empiris kepada pengembang *frontend* mengenai kesesuaian penggunaan *React JS* dan *Astro JS* untuk kebutuhan pembangunan *landing page*, khususnya pada skenario yang melibatkan aset media berat.

1.4 Metode Penelitian

Penelitian ini menggunakan metode kuantitatif komparatif, yaitu dengan membangun dua prototipe landing page sebagai objek pengujian, kemudian membandingkan performa keduanya menggunakan instrumen pengujian yang telah ditentukan. Pendekatan ini dipilih karena memungkinkan pengontrolan variabel secara ketat, di mana kedua prototipe dibangun dengan konten, struktur, dan aset media yang identik, sehingga perbedaan hasil pengujian dapat secara sah diatribusikan pada perbedaan arsitektur *framework* yang digunakan.

Secara garis besar, penelitian dilakukan melalui enam tahap utama, yaitu studi literatur, perancangan dan pembangunan prototipe, *deployment* ke platform hosting, pengujian performa menggunakan Google Lighthouse dan PageSpeed Insights, pengujian beban menggunakan Apache JMeter, serta analisis dan komparasi data hasil pengujian. Keenam tahap tersebut dilaksanakan secara berurutan, di mana output dari satu tahap menjadi prasyarat bagi dimulainya tahap berikutnya, guna menjaga konsistensi metodologi sepanjang proses penelitian.

1.5 Sistematika Penulisan Ilmiah

Penulisan ilmiah ini terdiri dari empat bagian yang disusun secara berurutan dan saling berkesinambungan.

a. Bagian pertama merupakan pendahuluan yang memuat latar belakang pemilihan topik, ruang lingkup batasan penelitian, tujuan yang ingin dicapai, metode penelitian yang ditempuh, serta sistematika penulisan secara keseluruhan.
b. Bagian kedua merupakan tinjauan pustaka yang menguraikan landasan teori sebagai pijakan penelitian, meliputi konsep landing page dan karakteristiknya, arsitektur *Client-Side Rendering* dan *Static Site Generation*, konsep *Islands Architecture* pada Astro JS, penjelasan metrik Web Vitals, serta gambaran umum masing-masing instrumen pengujian yang digunakan dalam penelitian ini.
c. Bagian ketiga merupakan inti dari penulisan ilmiah ini, yaitu pembahasan yang memaparkan proses pembangunan kedua prototipe landing page, hasil pengujian menggunakan ketiga instrumen audit, analisis komparatif data yang diperoleh, serta interpretasi temuan penelitian secara menyeluruh.
d. Bagian keempat merupakan penutup yang memuat simpulan dari seluruh hasil penelitian beserta saran yang dapat menjadi pertimbangan bagi penelitian selanjutnya maupun bagi para pengembang dalam pengambilan keputusan pemilihan teknologi *frontend*.

BAB II
TINJAUAN PUSTAKA

2.1 Metode Kuantitatif Komparatif

Metode kuantitatif komparatif merupakan pendekatan penelitian yang bertujuan untuk membandingkan dua atau lebih objek, kondisi, atau perlakuan berdasarkan data numerik yang terukur, guna mengidentifikasi ada tidaknya perbedaan yang signifikan di antara objek yang dibandingkan. Pendekatan ini umum diterapkan dalam penelitian bidang teknologi informasi untuk menguji perbedaan performa antar framework melalui pembangunan prototipe yang identik, kemudian diuji menggunakan instrumen pengukuran performa dan beban yang terstandarisasi guna menghasilkan data yang objektif dan dapat direplikasi (Hermanto & Engel, 2025).

Dalam konteks penelitian ini, metode kuantitatif komparatif diterapkan dengan membandingkan performa dua framework frontend, yaitu Astro JS dan React JS, yang diuji pada kondisi eksperimen yang identik dan terkontrol. Data yang dikumpulkan berupa nilai numerik dari metrik Web Vitals (FCP, LCP, TBT, CLS, SI) melalui Google Lighthouse dan PageSpeed Insights, serta metrik Response Time, Throughput, dan Error Rate melalui Apache JMeter. Data tersebut kemudian dianalisis secara deskriptif melalui perbandingan nilai rata-rata untuk menarik kesimpulan mengenai perbedaan performa antara kedua framework yang diuji.

2.2 Landing Page

Landing page merupakan halaman web yang dirancang secara khusus dengan satu tujuan tunggal, yaitu mendorong pengunjung untuk melakukan tindakan spesifik yang disebut *call-to-action* (CTA). Tindakan tersebut dapat berupa pendaftaran akun, pembelian produk, pengunduhan konten, maupun pengisian formulir kontak. Berbeda dengan halaman utama (*homepage*) yang menyajikan informasi secara luas dan umum, *landing page* dirancang dengan fokus yang sempit

dan terarah agar pengunjung tidak terdistraksi oleh elemen-elemen yang tidak relevan dengan tujuan utama halaman (Krestianti et al., 2024). Dalam konteks pemasaran digital, *landing page* sering kali menjadi titik akhir dari sebuah kampanye iklan atau tautan yang disebarkan melalui media sosial maupun mesin pencari, sehingga kualitas dan kecepatan pemuatannya menjadi faktor penentu keberhasilan kampanye tersebut.

Menurut Trisnawati & Rusdianto (2024), *landing page* merupakan bagian dari halaman *website* yang menampilkan informasi khusus dan penawaran terbatas yang ditujukan untuk segmen audiens tertentu. Karakteristik inilah yang membedakan *landing page* dari jenis halaman web lainnya, kontennya bersifat terfokus, elemen navigasinya minimal, dan seluruh elemen visual diarahkan untuk mendukung satu pesan utama. Lebih lanjut, *landing page* yang dioptimalkan dengan baik juga berkontribusi pada peningkatan visibilitas di mesin pencari melalui penerapan *Search Engine Optimization* (SEO), sehingga mampu mendatangkan *organic traffic* sekaligus meningkatkan potensi konversi.

Dari sisi teknis, *landing page* memiliki karakteristik yang berbeda dengan aplikasi web pada umumnya. Halaman ini umumnya bersifat statis atau semi-statis, artinya konten yang disajikan tidak berubah secara dinamis berdasarkan interaksi pengguna secara *real-time*. Komponen yang lazim ditemukan pada sebuah *landing page* mencakup *hero section*, deskripsi produk atau layanan, galeri media, *testimonial*, serta tombol atau formulir CTA (Krestianti et al., 2024). Karakteristik statis inilah yang menjadi dasar pertimbangan dalam penelitian ini, karena pendekatan *Client-Side Rendering* (CSR) yang diterapkan *React JS* sejatinya dirancang untuk kebutuhan aplikasi yang jauh lebih dinamis dan interaktif dibandingkan karakteristik *landing page* pada umumnya.

Kecepatan pemuatan *landing page* memiliki korelasi langsung dengan pengalaman pengguna dan tingkat konversi. Hal ini menjadikan pemilihan arsitektur dan teknologi yang tepat dalam membangun *landing page* bukan sekadar pertimbangan teknis semata, melainkan juga berdampak langsung pada efektivitas bisnis dan pemasaran digital secara keseluruhan.

2.3 React JS

React JS adalah *library JavaScript* bersifat *open source* yang dikembangkan oleh Facebook pada tahun 2013 untuk membangun antarmuka pengguna (*user interface*) berbasis komponen yang bersifat deklaratif dan efisien. Berbeda dengan *framework* penuh seperti *Angular*, *React JS* berfokus secara spesifik pada lapisan tampilan (*view layer*) dalam arsitektur MVC (*Model-View-Controller*), sehingga memberikan fleksibilitas tinggi bagi pengembang dalam menentukan ekosistem pendukung yang akan digunakan (Haryono et al., 2025). Salah satu fitur fundamental *React JS* adalah penggunaan *Virtual DOM*, yaitu representasi virtual dari *Document Object Model* yang memungkinkan *React JS* memperbarui hanya bagian antarmuka yang berubah tanpa harus me-*render* ulang keseluruhan halaman, sehingga meningkatkan efisiensi pembaruan antarmuka pada aplikasi yang bersifat dinamis dan interaktif (Anggraeni et al., 2024).

Pendekatan utama yang digunakan *React JS* dalam merender halaman adalah *Client-Side Rendering* (CSR), di mana seluruh proses pembangunan antarmuka dilakukan di sisi *browser* pengguna menggunakan *JavaScript*. Ketika pengguna mengakses sebuah halaman berbasis *React JS*, *browser* terlebih dahulu mengunduh berkas *HTML* kosong beserta seluruh *bundle JavaScript* yang diperlukan, kemudian mengeksekusinya untuk menghasilkan tampilan halaman secara dinamis di sisi klien (Haryono et al., 2025). Proses ini menyebabkan adanya jeda waktu antara permintaan awal pengguna dan tampilnya konten pertama di layar, yang dikenal sebagai *First Contentful Paint* (FCP). Semakin besar ukuran *bundle JavaScript* yang harus diunduh dan dieksekusi, semakin lama pula waktu yang dibutuhkan sebelum pengguna dapat melihat dan berinteraksi dengan konten halaman.

Meskipun pendekatan CSR sangat unggul untuk aplikasi web yang memerlukan interaktivitas tinggi dan pembaruan data secara *real-time*, pendekatan ini memiliki keterbatasan mendasar ketika diterapkan pada halaman yang bersifat statis seperti *landing page*. Menurut Zulhijaya et al. (2025), hasil pengujian performa menunjukkan bahwa *React JS* secara konsisten menghasilkan nilai FCP dan *Largest Contentful Paint* (LCP) yang lebih tinggi dibandingkan *framework* yang menghasilkan *output* lebih ringan, dengan selisih waktu yang cukup signifikan

terutama pada halaman yang memuat aset media berat. Hal ini disebabkan oleh beban eksekusi *JavaScript* yang harus diselesaikan terlebih dahulu sebelum konten dapat dirender, menjadikan CSR kurang optimal untuk skenario di mana kecepatan pemuatan awal menjadi prioritas utama.

Popularitas *React JS* di kalangan pengembang *frontend* global tidak dapat dipungkiri. Berdasarkan *Stack Overflow Developer Survey* 2025, *React JS* digunakan oleh 44,7% pengembang profesional di seluruh dunia, menjadikannya *framework frontend* paling banyak digunakan secara global (Stack Overflow, 2025). Tingginya angka adopsi ini mendorong banyak pengembang menggunakan *React JS* sebagai pilihan *default* untuk berbagai jenis proyek, termasuk *landing page*, tanpa selalu mempertimbangkan apakah arsitektur CSR yang menjadi andalannya sesuai dengan karakteristik kebutuhan halaman yang dibangun. Fenomena ini diperkuat oleh hasil studi kualitatif (Fatikasari & Fargazzi 2025) yang menemukan bahwa *React JS* menjadi *framework* paling dominan dalam diskusi komunitas pengembang daring, namun sekaligus menjadi subjek keluhan terbanyak terkait kompleksitas *tooling* dan performa, mengindikasikan adanya kesenjangan antara popularitas dan kesesuaian penggunaannya untuk berbagai konteks proyek.

2.4 Astro JS

Astro JS adalah framework web modern yang dirancang khusus untuk membangun website yang cepat dan efisien dengan pendekatan berbasis konten statis. Berbeda dengan framework berbasis JavaScript pada umumnya yang mengirimkan seluruh bundle ke browser, Astro JS menggunakan filosofi "zero JavaScript by default" artinya secara default halaman yang dihasilkan berupa HTML murni tanpa JavaScript sama sekali, kecuali pada komponen yang memang secara eksplisit membutuhkan interaktivitas (Lucaciu, 2026). Pendekatan ini menjadikan Astro JS sangat relevan untuk kebutuhan pembangunan website yang bersifat statis atau semi-statis seperti landing page, blog, dan halaman dokumentasi, di mana sebagian besar konten tidak memerlukan pembaruan dinamis secara real-time.

Salah satu fitur teknis dari Astro JS adalah penerapan Static Site Generation (SSG) sebagai mode rendering utamanya. Dalam pendekatan SSG, seluruh halaman dirender pada saat proses build, sehingga ketika pengguna mengakses halaman tersebut, browser langsung menerima HTML yang sudah jadi tanpa perlu menunggu eksekusi JavaScript terlebih dahulu (Zaidan & Suyatno, 2025). Ini berbeda secara fundamental dengan Client-Side Rendering (CSR) yang digunakan React JS, di mana browser harus mengunduh dan mengeksekusi JavaScript terlebih dahulu sebelum konten dapat ditampilkan. Konsekuensi langsungnya adalah nilai First Contentful Paint (FCP) dan Largest Contentful Paint (LCP) pada halaman berbasis SSG cenderung lebih rendah, karena konten sudah tersedia sejak permintaan pertama diterima oleh browser.

Keunikan lain yang dimiliki Astro JS adalah konsep Islands Architecture. Dalam arsitektur ini, sebuah halaman web dipandang sebagai hamparan konten statis yang sesekali diselingi oleh "pulau-pulau" (islands) komponen interaktif yang membutuhkan JavaScript (Lucaciu, 2026). Komponen-komponen interaktif tersebut baru dihidrasi, yaitu diaktifkan oleh JavaScript ketika pengguna memang membutuhkannya, misalnya ketika elemen tersebut masuk ke area tampilan (viewport) atau ketika pengguna berinteraksi dengannya. Mekanisme ini dikenal sebagai partial hydration, dan dampaknya sangat signifikan terhadap performa: jumlah JavaScript yang dikirimkan ke browser berkurang drastis karena hanya komponen yang benar-benar interaktif yang mendapat script, sementara bagian statis lainnya tetap berupa HTML murni. Astro JS juga mendukung integrasi dengan berbagai framework frontend populer seperti React, Vue, dan Svelte, sehingga pengembang tidak harus meninggalkan ekosistem yang sudah mereka kuasai untuk mulai menggunakan Astro JS (Lucaciu, 2026). Fleksibilitas ini menjadikan Astro JS bukan sekadar alternatif sempit, melainkan sebuah lapisan orkestrasi yang dapat memanfaatkan kekuatan berbagai framework sekaligus tetap menjaga output halaman seringan mungkin. Dalam konteks penelitian ini, Astro JS digunakan dalam konfigurasi SSG penuh tanpa framework tambahan, sehingga seluruh halaman menghasilkan HTML statis murni yang dapat dibandingkan secara langsung dengan prototipe berbasis React JS CSR.

2.5 Web Vitals

Web Vitals adalah inisiatif yang diluncurkan oleh Google pada tahun 2020 sebagai upaya untuk menyediakan panduan terpadu mengenai sinyal kualitas yang penting dalam membangun pengalaman pengguna yang baik di web. Sebelum adanya *Web Vitals*, pengembang dihadapkan pada begitu banyak alat dan metrik performa yang tersedia sehingga sulit untuk menentukan mana yang paling relevan dan perlu diprioritaskan. *Web Vitals* hadir untuk menyederhanakan hal tersebut dengan menetapkan seperangkat metrik standar yang berfokus pada tiga aspek utama pengalaman pengguna, yaitu kecepatan pemuatan, interaktivitas, dan stabilitas visual halaman (Google, 2024). Metrik-metrik ini kemudian dijadikan acuan resmi oleh Google dan digunakan sebagai salah satu faktor penilaian dalam algoritma peringkat mesin pencari.

Dalam ekosistem *Web Vitals*, terdapat subset metrik yang disebut *Core Web Vitals*, yaitu metrik-metrik yang dianggap paling krusial dan berlaku universal untuk semua jenis halaman web. *Core Web Vitals* saat ini terdiri dari *Largest Contentful Paint* (LCP), *Interaction to Next Paint* (INP), dan *Cumulative Layout Shift* (CLS) (Google, 2024). Selain ketiga metrik inti tersebut, terdapat metrik diagnostik tambahan yang tidak termasuk dalam *Core Web Vitals* namun tetap digunakan secara luas dalam pengujian performa berbasis laboratorium, yaitu *First Contentful Paint* (FCP) dan *Total Blocking Time* (TBT). Dalam penelitian ini, metrik yang diukur meliputi FCP, LCP, TBT, CLS dan SI kombinasi yang umum digunakan dalam penelitian perbandingan performa *framework* berbasis *Web Vitals* (Zaidan & Suyatno, 2025; Khasanah & Suartana, 2024).

2.5.1 First Contentful Paint (FCP)

First Contentful Paint (FCP) adalah metrik yang mengukur waktu yang dibutuhkan sejak pengguna pertama kali meminta sebuah halaman hingga *browser* berhasil merender elemen konten pertama yang terlihat di layar, baik berupa teks, gambar, maupun elemen *canvas* (Google, 2024). FCP menjadi indikator awal seberapa cepat pengguna mulai melihat respons visual dari halaman yang sedang

dimuat. Semakin cepat nilai FCP, semakin cepat pula pengguna mendapatkan konfirmasi bahwa halaman sedang berjalan dan tidak mengalami kegagalan.

Google menetapkan ambang batas FCP sebagai berikut: nilai di bawah 1,8 detik dikategorikan sebagai cepat (*good*), nilai antara 1,9 hingga 3,0 detik dikategorikan sebagai perlu perbaikan (*needs improvement*), dan nilai di atas 3,0 detik dikategorikan sebagai lambat (*poor*) (Khasanah & Suartana, 2024). Dalam konteks penelitian ini, FCP menjadi salah satu metrik kunci karena perbedaan arsitektur antara CSR dan SSG secara langsung berdampak pada waktu kemunculan konten pertama, halaman berbasis SSG umumnya menghasilkan nilai FCP yang lebih rendah karena *HTML* sudah tersedia saat permintaan pertama diterima tanpa perlu menunggu eksekusi *JavaScript* (Zaidan & Suyatno, 2025).

2.5.2 Largest Contentful Paint (LCP)

Largest Contentful Paint (LCP) adalah metrik yang mengukur waktu yang dibutuhkan untuk merender elemen konten terbesar yang terlihat di *viewport* pengguna sejak halaman pertama kali dimuat (Google, 2024). Elemen yang dimaksud biasanya berupa gambar berukuran besar, video, atau blok teks berukuran besar yang mendominasi area tampilan. LCP menjadi representasi yang lebih akurat dari persepsi pengguna terhadap kecepatan pemuatan halaman dibandingkan metrik pemuatan tradisional, karena ia mengukur kapan konten utama, bukan sekadar elemen pertama benar-benar tampil di layar.

Google menetapkan ambang batas LCP sebagai berikut: nilai di bawah 2,5 detik dikategorikan sebagai **cepat**, nilai antara 2,6 hingga 4,0 detik dikategorikan sebagai **perlu perbaikan**, dan nilai di atas 4,0 detik dikategorikan sebagai **lambat** (Khasanah & Suartana, 2024). LCP menjadi metrik yang sangat relevan dalam penelitian ini mengingat objek pengujian adalah *landing page* dengan aset media berat kondisi di mana elemen LCP kemungkinan besar adalah gambar beresolusi tinggi atau video yang membutuhkan waktu unduh signifikan, sehingga perbedaan arsitektur *rendering* akan terlihat jelas pada nilai metrik ini (Zaidan & Suyatno, 2025).

2.5.3 Total Blocking Time (TBT)

Total Blocking Time (TBT) adalah metrik yang mengukur total waktu di mana halaman web tidak dapat merespons interaksi pengguna setelah FCP terjadi hingga halaman sepenuhnya interaktif (Zaidan & Suyatno, 2025). TBT dihitung berdasarkan akumulasi *long tasks*, yaitu tugas-tugas yang memblokir *main thread* selama lebih dari 50 milidetik. Semakin besar nilai TBT, semakin lama pengguna harus menunggu sebelum mereka dapat berinteraksi dengan halaman secara penuh, yang berujung pada persepsi halaman yang terasa "beku" atau tidak responsif.

Google menetapkan ambang batas TBT sebagai berikut: nilai di bawah 200 milidetik dikategorikan sebagai cepat, nilai antara 201 hingga 600 milidetik dikategorikan sebagai perlu perbaikan, dan nilai di atas 600 milidetik dikategorikan sebagai lambat (Khasanah & Suartana, 2024). TBT berkorelasi langsung dengan ukuran *bundle JavaScript* yang harus dieksekusi oleh *browser*, semakin besar *bundle* yang dikirimkan, semakin tinggi potensi terjadinya *long tasks* yang menghambat interaktivitas. Oleh karena itu, TBT menjadi metrik yang sangat sensitif terhadap perbedaan antara pendekatan CSR yang JavaScript-*heavy* dan SSG yang meminimalkan *JavaScript* di sisi klien.

2.5.4 Cumulative Layout Shift (CLS)

Cumulative Layout Shift (CLS) adalah metrik yang mengukur stabilitas visual sebuah halaman web dengan menghitung seberapa sering dan seberapa jauh elemen-elemen pada halaman berpindah atau bergeser dari posisi awalnya selama proses pemuatan berlangsung (Khasanah & Suartana, 2024). Pergeseran tata letak yang tidak terduga dapat sangat mengganggu pengalaman pengguna, misalnya ketika pengguna hendak mengklik sebuah tombol namun tiba-tiba konten bergeser sehingga klik tersebut mengenai elemen yang berbeda. CLS mengukur akumulasi skor pergeseran sepanjang siklus hidup halaman.

Google menetapkan ambang batas CLS sebagai berikut: nilai di bawah 0,1 dikategorikan sebagai **baik**, nilai antara 0,1 hingga 0,25 dikategorikan sebagai **perlu perbaikan**, dan nilai di atas 0,25 dikategorikan sebagai **buruk** (Khasanah & Suartana, 2024). Dalam konteks *landing page* dengan aset media berat, CLS

menjadi metrik yang perlu diperhatikan karena gambar dan video yang belum ditentukan dimensinya secara eksplisit dapat menyebabkan pergeseran tata letak yang signifikan saat konten tersebut selesai dimuat. Kedua prototipe dalam penelitian ini menggunakan dimensi aset yang didefinisikan secara eksplisit untuk memastikan bahwa perbedaan nilai CLS yang terukur benar-benar mencerminkan perbedaan arsitektur *framework*, bukan perbedaan dalam penanganan aset media.

2.5.5 Speed Index

Speed Index (SI) adalah metrik yang mengukur seberapa cepat konten halaman web ditampilkan secara visual kepada pengguna selama proses pemuatan berlangsung. Berbeda dengan FCP yang hanya mengukur kemunculan elemen pertama, *Speed Index* mengevaluasi kecepatan pengisian konten secara keseluruhan di area yang terlihat oleh pengguna (*viewport*) dari awal pemuatan hingga halaman terlihat lengkap secara visual (Hermanto & Engel, 2025). Semakin rendah nilai *Speed Index*, semakin cepat konten halaman terisi dan semakin baik pengalaman yang dirasakan oleh pengguna.

Speed Index diukur menggunakan analisis visual terhadap rekaman proses pemuatan halaman yang dilakukan oleh *Google Lighthouse*. Alat ini merekam tampilan halaman *frame* per *frame*, kemudian menghitung seberapa besar porsi halaman yang sudah terlihat lengkap pada setiap titik waktu. Nilai akhir *Speed Index* dinyatakan dalam satuan detik dan diperoleh dari integrasi area di antara kurva kemajuan visual tersebut (Google, 2024). Metrik ini sangat relevan dalam konteks penelitian komparatif *framework* karena mencerminkan persepsi kecepatan pengguna secara holistik, bukan hanya dari satu elemen tertentu.

2.6 Google Lighthouse

Google Lighthouse merupakan sebuah alat open source yang dikembangkan oleh Google dan dirancang untuk mengukur serta mengaudit kualitas halaman web secara otomatis. Alat ini mampu menganalisis berbagai aspek penting dari sebuah halaman web, mulai dari performa, aksesibilitas, penerapan praktik terbaik, optimasi mesin pencari (Search Engine Optimization/SEO), hingga kemampuan

sebagai Progressive Web App (PWA) (Google, 2024). Dalam konteks penelitian komparatif framework JavaScript, Lighthouse banyak digunakan sebagai instrumen pengujian karena kemampuannya menghasilkan data yang terstandarisasi dan dapat direplikasi, sehingga sangat cocok digunakan dalam perbandingan antara dua atau lebih sistem yang diuji secara bersamaan (Hermanto & Engel, 2025).

Lighthouse bekerja dengan cara mensimulasikan proses pemuatan halaman web dalam lingkungan yang terkontrol, yang biasa disebut sebagai lab data atau lab testing. Proses pengujian dapat dilakukan melalui beberapa cara, yaitu menggunakan Chrome DevTools yang tersedia langsung di browser Google Chrome, melalui antarmuka berbasis web di laman web.dev, atau menggunakan command line interface (CLI) untuk kebutuhan otomatisasi. Dalam penelitian ini, pengujian dilakukan melalui Chrome DevTools, sehingga Lighthouse memanfaatkan perangkat dan koneksi jaringan lokal peneliti sebagai baseline sebelum diterapkan simulasi throttling, berbeda dengan PageSpeed Insights yang dijalankan pada infrastruktur server Google sebagaimana dijelaskan pada sub-bab 2.7. Hasilnya kemudian disajikan dalam bentuk laporan lengkap yang mencakup skor untuk setiap kategori audit beserta rekomendasi perbaikan yang dapat langsung ditindaklanjuti oleh pengembang.

Dari kelima kategori audit yang tersedia pada Lighthouse, kategori Performance menjadi yang paling relevan dalam penelitian ini karena berfokus pada seberapa cepat dan stabil halaman web dimuat dari sudut pandang pengguna. Skor performa Lighthouse dihitung berdasarkan beberapa metrik utama yang masing-masing memiliki bobot kontribusi tertentu terhadap skor akhir, meliputi First Contentful Paint (FCP), Largest Contentful Paint (LCP), Total Blocking Time (TBT), Cumulative Layout Shift (CLS), dan Speed Index (SI). Skor akhir yang dihasilkan berada pada rentang 0 hingga 100, di mana skor 90 ke atas dikategorikan baik (good), skor 50 hingga 89 dikategorikan perlu perbaikan (needs improvement), dan skor di bawah 50 dikategorikan buruk (poor) (Zaidan & Suyatno, 2025). Penjelasan lebih mendalam mengenai masing-masing metrik tersebut telah diuraikan pada sub-bab 2.5 mengenai Web Vitals. Keandalan Lighthouse sebagai instrumen pengujian telah dibuktikan pada berbagai penelitian komparatif performa

web, di antaranya oleh Khasanah & Suartana (2024) dan Zaidan & Suyatno (2025) dalam membandingkan performa berbagai framework JavaScript.

2.7 PageSpeed Insights

PageSpeed Insights (PSI) adalah sebuah layanan analisis performa halaman web yang disediakan secara gratis oleh Google dan dapat diakses melalui laman pagespeed.web.dev. Layanan ini dirancang untuk membantu pengembang web dalam memahami dan meningkatkan performa halaman mereka, baik pada perangkat mobile maupun desktop (Google, 2024). Salah satu keunggulan utama PageSpeed Insights dibandingkan alat pengujian performa lainnya terletak pada cara pengambilan datanya yang menggabungkan dua jenis sumber data secara sekaligus, yaitu lab data yang dihasilkan dari simulasi terkontrol menggunakan mesin Lighthouse, dan field data yang bersumber dari data penggunaan nyata pengguna di seluruh dunia melalui Chrome User Experience Report (CrUX). Berbeda dengan Lighthouse yang dijalankan langsung pada perangkat dan koneksi jaringan pengguna, komponen lab data pada PageSpeed Insights dijalankan pada infrastruktur server Google dengan profil perangkat dan kalibrasi throttling yang telah ditetapkan secara baku, sehingga hasilnya tidak dipengaruhi oleh spesifikasi perangkat maupun kondisi jaringan pihak yang melakukan pengujian.

Kombinasi dua sumber data tersebut menjadikan PageSpeed Insights memiliki nilai analisis yang lebih komprehensif dibandingkan menjalankan Lighthouse secara langsung yang hanya mengandalkan lab data, karena field data mencerminkan pengalaman pengguna yang sesungguhnya berdasarkan kumpulan data historis dari pengguna Chrome yang pernah mengakses halaman tersebut (Google, 2024). Dalam penelitian yang dilakukan oleh Zaidan & Suyatno (2025), PageSpeed Insights digunakan sebagai instrumen pengujian karena kemampuannya memberikan gambaran performa yang lebih menyeluruh, tidak hanya dari sisi teknis laboratorium tetapi juga dari perspektif pengalaman pengguna yang nyata di berbagai kondisi jaringan.

Metrik yang ditampilkan oleh PageSpeed Insights selaras dengan metrik Core Web Vitals yang ditetapkan oleh Google, yaitu First Contentful Paint (FCP),

Largest Contentful Paint (LCP), Total Blocking Time (TBT), dan Cumulative Layout Shift (CLS). Setiap metrik ditampilkan beserta status penilaiannya berdasarkan ambang batas yang telah ditetapkan, dengan indikator warna hijau untuk kondisi baik, oranye untuk perlu perbaikan, dan merah untuk kondisi buruk, disertai daftar rekomendasi perbaikan yang spesifik beserta estimasi penghematan waktu yang dapat dicapai (Google, 2024). Dalam penelitian ini, PageSpeed Insights digunakan sebagai instrumen pengujian yang melengkapi Google Lighthouse, sehingga memungkinkan validasi silang hasil pengujian pada mode mobile dan desktop antara landing page berbasis Astro JS dan React JS.

2.8 Apache JMeter

Apache JMeter adalah perangkat lunak *open source* berbasis Java yang dikembangkan oleh *Apache Software Foundation* dan dirancang khusus untuk keperluan pengujian beban (*load testing*) serta pengukuran performa sistem secara menyeluruh. Awalnya *JMeter* dikembangkan untuk menguji aplikasi berbasis web melalui protokol HTTP dan HTTPS, namun seiring perkembangannya alat ini telah diperluas untuk mendukung berbagai jenis pengujian terhadap layanan *File Transfer Protocol* (FTP), *database*, dan berbagai protokol lainnya (Apache Software Foundation, 2024). Dalam penelitian performa web, *JMeter* digunakan secara luas untuk mensimulasikan kondisi beban pengguna yang tinggi guna mengevaluasi ketahanan dan kapasitas sistem dalam menghadapi lonjakan trafik secara bersamaan, sehingga kelemahan sistem yang tidak terdeteksi pada kondisi normal dapat teridentifikasi.

Cara kerja *JMeter* dalam pengujian beban pada dasarnya adalah dengan membuat sejumlah *virtual user* atau *thread* yang secara bersamaan mengirimkan *request* ke server target, kemudian merekam respons yang diterima untuk dianalisis lebih lanjut. Komponen utama dalam konfigurasi pengujian *JMeter* meliputi *Thread Group* yang menentukan jumlah pengguna virtual yang akan disimulasikan, *ramp-up period* yang mengatur lamanya waktu penambahan pengguna secara bertahap dari nol hingga jumlah yang ditentukan, dan *loop count* yang menentukan berapa kali setiap pengguna akan mengulangi rangkaian *request* yang telah

dikonfigurasi (Apache Software Foundation, 2024). Penerapan *ramp-up period* yang bertahap ini penting dilakukan agar simulasi mendekati kondisi yang lebih realistis, di mana pengguna tidak semuanya mengakses sistem pada saat yang persis bersamaan melainkan datang secara bertahap dalam rentang waktu tertentu (Permatasari et al., 2020).

Terdapat tiga metrik utama yang diukur dalam pengujian menggunakan *Apache JMeter*, yaitu *response time*, *throughput*, dan *error rate*. *Response time* mengukur rata-rata waktu yang dibutuhkan oleh server untuk merespons setiap *request* dari pengguna, di mana semakin rendah nilainya maka semakin baik kemampuan server dalam melayani pengguna secara cepat (Tejaya et al., 2023). *Throughput* mengukur jumlah *request* yang berhasil diproses oleh server dalam satuan waktu tertentu, umumnya dinyatakan dalam satuan *request* per detik, sehingga nilai yang lebih tinggi mencerminkan kapasitas server yang lebih besar dalam menangani beban secara bersamaan. Sementara itu, *error rate* mengukur persentase *request* yang gagal diproses dari keseluruhan *request* yang dikirimkan selama sesi pengujian berlangsung, dan nilai yang rendah menunjukkan bahwa sistem mampu menangani beban dengan stabil tanpa banyak mengalami kegagalan layanan (Hermanto & Engel, 2025).

Dalam penelitian ini, pengujian menggunakan *Apache JMeter* dilakukan untuk mengevaluasi ketahanan kedua *landing page* berbasis Astro JS dan React JS ketika menghadapi lonjakan jumlah pengguna secara bersamaan. Pengujian dirancang dalam tiga skenario beban yang berbeda, yaitu 100, 200, dan 300 pengguna virtual, dengan *ramp-up period* yang diatur secara bertahap agar simulasi mendekati kondisi trafik yang lebih realistis. Setiap skenario dijalankan beberapa kali untuk memastikan konsistensi hasil, kemudian nilai rata-rata dari setiap metrik dihitung sebagai dasar perbandingan antara kedua *framework* yang diuji. Pendekatan pengujian berlapis dengan skenario bertingkat seperti ini telah terbukti efektif dalam mengidentifikasi perbedaan performa yang signifikan antar sistem, sebagaimana ditunjukkan oleh Tejaya et al. (2023) dan Permatasari et al. (2020) yang menggunakan *JMeter* dalam skenario serupa untuk mengevaluasi ketahanan sistem web di bawah berbagai kondisi beban pengguna.

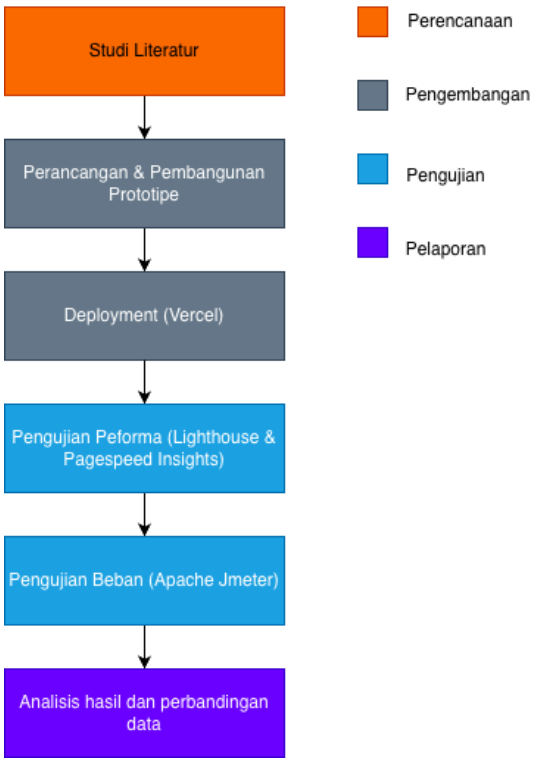
BAB III
PEMBAHASAN

3.1 Gambaran Umum

Penelitian ini diterapkan sebagai eksperimen dengan metode kuantitatif komparatif sebagaimana dijelaskan pada sub-bab 2.1, dengan tujuan mengukur dan membandingkan performa dua teknologi *frontend* modern, yaitu Astro JS dengan pendekatan *Static Site Generation* (SSG) dan React JS dengan pendekatan *Client-Side Rendering* (CSR), dalam konteks pembangunan *landing page* dengan aset media berat. Pendekatan ini dipilih karena memungkinkan pengontrolan variabel secara ketat, di mana kedua prototipe dibangun dengan konten yang sepenuhnya identik, sehingga perbedaan hasil pengujian dapat secara sah diatribusikan pada perbedaan arsitektur *framework*, bukan pada perbedaan konten atau infrastruktur.

Agar proses penelitian berjalan secara sistematis dan hasil yang diperoleh dapat dipertanggungjawabkan secara ilmiah, seluruh rangkaian kegiatan penelitian diorganisasi ke dalam empat fase utama yang mencakup enam tahapan kerja. Keempat fase tersebut meliputi fase perencanaan, fase pengembangan, fase pengujian, dan fase pelaporan. Masing-masing fase dirancang secara berurutan dan saling bergantung, di mana output dari satu fase menjadi prasyarat bagi dimulainya fase berikutnya. Pendekatan bertahap seperti ini penting untuk menjaga konsistensi metodologi sekaligus memastikan bahwa setiap keputusan teknis yang diambil selama penelitian memiliki dasar yang dapat ditelusuri kembali.

Alur dari keseluruhan tahapan penelitian beserta keterkaitannya secara metodologis dapat dilihat pada *flowchart Gambar 3.1* berikut. *Flowchart* tersebut menggambarkan bagaimana setiap tahapan mengalir secara sekuensial dari fase perencanaan hingga fase pelaporan, di mana setiap tahapan merupakan prasyarat bagi dimulainya tahapan berikutnya sehingga integritas dan konsistensi metodologi penelitian tetap terjaga sepanjang proses berlangsung.



Gambar 3.1 Tahapan Perancangan

Secara lebih detail, berikut adalah penjelasan dari setiap fase masing-masing keputusan yang diambil:

a) **Fase Perencanaan (Studi Literatur)** Fase ini merupakan fondasi dari seluruh penelitian. Studi literatur dilakukan secara sistematis dengan mengumpulkan referensi dari jurnal ilmiah, dokumentasi resmi *framework*, dan survei industri yang berkaitan dengan arsitektur CSR, SSG, *Islands Architecture*, metrik Web Vitals, serta karakteristik masing-masing instrumen pengujian. Pemahaman mendalam terhadap landasan teori pada fase ini menjadi krusial karena menentukan relevansi instrumen pengujian yang dipilih dan memastikan bahwa metrik yang diukur benar-benar merepresentasikan permasalahan yang ingin dipecahkan, yaitu perbedaan performa rendering antara dua paradigma arsitektur yang berbeda secara fundamental.
b) **Fase Pengembangan (Perancangan, Pembangunan, dan *Deployment*)** Pada fase ini, dua prototipe *landing page* dibangun secara paralel dengan konten,

aset media, dan struktur komponen yang sepenuhnya identik. Keidentikan ini merupakan syarat mutlak validitas komparasi, apabila terdapat perbedaan konten atau aset di antara kedua prototipe, maka perbedaan performa yang terukur tidak dapat secara eksklusif diatribusikan pada perbedaan *framework*. Setelah pembangunan selesai, kedua prototipe di-*deploy* ke platform *hosting* yang sama, yaitu Vercel, agar seluruh pengujian berlangsung dalam lingkungan infrastruktur yang setara.

c) **Fase Pengujian (Pengujian Performa dan Pengujian Beban)** Fase pengujian merupakan inti dari eksperimen yang dilakukan dan dibagi menjadi dua bagian yang saling melengkapi. Bagian pertama adalah pengujian performa pemuatan halaman menggunakan Google Lighthouse dan PageSpeed Insights, yang menghasilkan data metrik Web Vitals seperti FCP, LCP, TBT, CLS, dan SI. Bagian kedua adalah pengujian beban menggunakan Apache JMeter untuk mengevaluasi ketahanan dan stabilitas sistem ketika menghadapi simulasi lonjakan pengguna secara bersamaan. Pemisahan dua jenis pengujian ini disengaja karena keduanya mengukur dimensi performa yang berbeda. Lighthouse dan PSI mengukur kecepatan pemuatan dari perspektif satu pengguna, sementara JMeter mengukur kapasitas sistem dalam melayani banyak pengguna sekaligus.

d) **Fase Pelaporan (Analisis Hasil dan Perbandingan Data)** Fase terakhir adalah pengolahan dan interpretasi seluruh data mentah yang diperoleh dari fase pengujian. Pada fase ini dilakukan analisis komparatif untuk mengidentifikasi pola perbedaan performa antara kedua *framework* di seluruh instrumen pengujian, kemudian hasil analisis tersebut diinterpretasikan dalam konteks teoritis yang telah dibangun pada Bab II. Output akhir dari fase ini adalah kesimpulan berbasis data dan rekomendasi teknis yang dapat langsung dimanfaatkan oleh pengembang *web* dalam pengambilan keputusan pemilihan teknologi *frontend*.

3.2 Analisis Kebutuhan Sistem

Sebelum proses pembangunan prototipe dimulai, dilakukan tahap analisis kebutuhan sistem secara menyeluruh untuk memastikan bahwa seluruh komponen yang diperlukan telah teridentifikasi dan terdefinisi dengan jelas. Tahap ini penting dilakukan agar proses pengembangan dan pengujian dapat berjalan tanpa hambatan teknis yang tidak terduga, sekaligus memastikan bahwa kondisi eksperimen benar-benar terkontrol dan adil bagi kedua *framework* yang dibandingkan. Analisis kebutuhan dalam penelitian ini dibagi ke dalam empat aspek, yaitu kebutuhan fungsional, kebutuhan non-fungsional, spesifikasi perangkat keras, dan spesifikasi perangkat lunak.

3.2.1 Kebutuhan Fungsional

Kebutuhan fungsional mendefinisikan kapabilitas minimum yang harus dipenuhi oleh kedua prototipe agar tujuan penelitian dapat tercapai. Kebutuhan ini tidak berfokus pada performa, melainkan pada kelengkapan fitur yang menjamin bahwa kedua prototipe merepresentasikan sebuah *landing page* yang utuh dan dapat diuji secara setara. Terdapat empat kebutuhan fungsional utama yang ditetapkan dalam penelitian ini.

a. **Representasi Konten:** Kedua prototipe harus mampu menampilkan 11 *section* landing page secara utuh, mulai dari *Hero Section* hingga *Footer*, dengan muatan media yang identik.
b. **Aksesibilitas Publik:** Prototipe harus bisa diakses secara publik melalui URL yang valid agar instrumen pengujian eksternal seperti PageSpeed Insights bisa melakukan audit.
c. **Interaktivitas Dasar:** Sistem harus bisa menjalankan navigasi dan elemen interaktif seperti *accordion* pada bagian FAQ serta filter pada galeri foto.
d. **Responsivitas:** Prototipe harus mampu menyesuaikan tata letak pada berbagai ukuran layar (desktop dan mobile) sesuai dengan skenario pengujian yang ditetapkan.

3.2.2 Kebutuhan Non-Fungsional

Berbeda dengan kebutuhan fungsional yang berfokus pada *apa* yang harus bisa dilakukan sistem, kebutuhan non-fungsional berfokus pada *bagaimana* kualitas dan batasan eksperimen harus dijaga agar hasil perbandingan yang diperoleh benar-benar valid dan bebas dari bias. Dalam penelitian komparatif seperti ini, kebutuhan non-fungsional justru menjadi aspek yang paling kritis karena satu variabel yang tidak terkontrol dapat menggugurkan seluruh validitas hasil pengujian.

a. **Keidentikan Aset:** Jumlah, format, dan ukuran berkas gambar serta video yang digunakan pada Astro JS dan React JS harus sama persis.
b. **Isolasi Variabel:** Seluruh animasi antarmuka wajib menggunakan CSS murni tanpa bantuan *library* pihak ketiga agar perbedaan performa benar-benar berasal dari framework-nya, bukan dari beban pustaka luar.
c. **Konsistensi Infrastruktur:** Kedua prototipe harus berada di platform *hosting* yang sama dengan spesifikasi peladen yang setara untuk menghindari perbedaan waktu respon akibat infrastruktur jaringan.
d. **Stabilitas:** Sistem harus tetap stabil dan tidak mengalami kegagalan koneksi (*error rate* 0%) saat menerima beban simulasi pengguna dari Apache JMeter.

3.2.3 Spesifikasi Perangkat Keras

Untuk menjaga konsistensi selama pengembangan dan pengujian beban, digunakan satu perangkat keras yang sama. Spesifikasi perangkat keras yang digunakan dalam penelitian ini, ditunjukan pada Tabel 3.1.

Tabel 3.1 Spesifikasi Perangkat Keras

Komponen	Spesifikasi
Laptop	MacBook Air
Prosesor	Apple M2
Memori (RAM)	16 GB
Media Penyimpanan	SSD
Sistem Operasi	macOS Tahoe 26.3

3.2.4 Spesifikasi Perangkat Lunak

Pemilihan perangkat lunak dalam penelitian ini dilakukan dengan mempertimbangkan dua kriteria utama, yaitu relevansi terhadap tujuan penelitian dan kemampuan untuk menghasilkan lingkungan pengujian yang setara bagi kedua *framework*. Seluruh perangkat lunak yang digunakan, mulai dari *code editor* hingga instrumen pengujian, dijalankan pada perangkat dan sistem operasi yang sama untuk menghindari variasi hasil yang disebabkan oleh perbedaan lingkungan eksekusi, Daftar lengkap perangkat lunak yang digunakan ditujunkan pada Tabel 3.2.

Tabel 3.2 Spesifikasi Perangkat Lunak

Kategori	Nama Perangkat Lunak
Code Editor	Visual Studio Code
Web Browser	Google Chrome
Runtime Environment	Node.js
Package Manager	npm
Version Control	Git & GitHub
Deployment Platform	Vercel
Framework 1	Astro JS (Mode SSG + Island Architecture)
Framework 2	React JS (Mode CSR dengan Vite)
Testing Tool 1	Google Lighthouse
Testing Tool 2	PageSpeed Insights
Testing Tool 3	Apache JMeter v5.6.3

3.3 Perancangan Prototipe

Tahap perancangan prototipe merupakan tahap yang paling menentukan validitas seluruh eksperimen ini. Pada tahap ini, seluruh keputusan desain yang menyangkut konten, struktur, aset media, dan batasan teknis kedua prototipe ditetapkan sebelum proses pembangunan dimulai. Penetapan keputusan desain di awal ini penting untuk memastikan bahwa tidak ada penyesuaian yang dilakukan di tengah proses yang berpotensi menciptakan asimetri antara kedua prototipe. Seluruh keputusan perancangan yang dijelaskan pada sub-bab ini berlaku secara identik untuk prototipe Astro JS maupun React JS, kecuali pada aspek yang secara inheren berbeda akibat perbedaan arsitektur *framework* masing-masing.

3.3.1 Deskripsi dan Tema Prototipe

Prototipe yang dibangun dalam penelitian ini adalah *landing page* milik layanan klinik hewan peliharaan fiktif bernama "PawCare - Premium Pet Care & Clinic". Tema ini dipilih karena memungkinkan penggunaan konten yang cukup kaya dan kompleks, mencakup banyak gambar, video lokal, *embed* peta, *Google Fonts*, serta animasi sehingga perbedaan performa antara SSG dan CSR dapat terukur secara signifikan melalui instrumen pengujian yang digunakan (Zaidan & Suyatno, 2025).

Kedua prototipe dibangun dengan konten yang sepenuhnya identik, mulai dari struktur *section*, jumlah dan format aset, teks, hingga elemen antarmuka. Satu-satunya perbedaan terletak pada *framework* yang digunakan Astro JS dengan pendekatan *Static Site Generation* dan React JS dengan pendekatan *Client-Side Rendering* menggunakan Vite. Keidentikan ini menjadi syarat utama dalam penelitian komparatif agar perbedaan performa yang terukur benar-benar mencerminkan perbedaan *framework*, bukan perbedaan konten (Hermanto & Engel, 2025).

3.3.2 Spesifikasi Teknis Prototipe

Kedua prototipe terdiri dari sebelas *section* yang tersusun dalam satu halaman tunggal (*single page*) dengan konten yang sepenuhnya identik. Untuk menjaga objektivitas perbandingan, terdapat beberapa ketentuan teknis yang diterapkan secara konsisten pada keduanya.

Dari sisi *styling* dan animasi, kedua prototipe menggunakan *Tailwind CSS* sebagai *utility-first CSS framework* untuk kebutuhan *layout* dan tampilan antarmuka. Sementara seluruh efek animasi seperti *fade-in* pada kartu layanan, *counter* pada *section Stats*, dan transisi *accordion* pada *section FAQ* diimplementasikan menggunakan *CSS Keyframes* murni tanpa *library* eksternal seperti AOS atau GSAP. Pemisahan ini dilakukan agar perbedaan performa yang terukur benar-benar berasal dari perbedaan *framework*, bukan dari beban *library* animasi pihak ketiga (Anggraeni et al., 2024).

Dari sisi aset, seluruh gambar lokal menggunakan format PNG yang bersifat *lossless* sehingga ukuran *file*-nya lebih besar dibandingkan WebP. Hal ini disengaja untuk menciptakan kondisi *payload* yang lebih berat agar perbedaan kecepatan pemuatan antara SSG dan CSR lebih terasa. Khusus gambar *hero*, dimuat langsung dari URL eksternal Unsplash untuk menciptakan *cross-origin request* yang memengaruhi nilai FCP. Untuk elemen video, digunakan file MP4 lokal yang ditampilkan via tag *HTML5 Video* standar.

Selain itu, kedua prototipe memuat tipografi *Google Fonts* dari CDN Google (*Playfair Display* dan *Inter*) serta *embed* Google Maps *iframe* pada *section Map & Contact*. Kedua elemen ini menghasilkan *external network request* yang identik pada keduanya dan berkontribusi pada kompleksitas pemuatan halaman secara keseluruhan, Rincian struktur konten kesebelas section tersebut ditunjukkan pada Tabel 3.3.

Tabel 3.3 Struktur Konten Prototipe Landing Page

No	*Section*	Deskripsi Konten	Elemen Multimedia
1	*Hero*	*Headline*, *subheadline*, dua tombol CTA, empat *counter* statistik	Gambar *hero* berukuran besar (Unsplash), animasi *CSS Keyframes*
2	*Services*	Delapan kartu layanan klinik	Ikon SVG, gambar *thumbnail* PNG, animasi *fade-in*
3	*Gallery*	Galeri foto hewan peliharaan	16 foto PNG, *filter tabs*
4	*Video*	Tur virtual fasilitas klinik	*Embed* Video MP4 lokal (*HTML5 Video*)
5	*Team*	Profil 6 dokter hewan	Foto *portrait* PNG, ikon SVG
6	*Testimonials*	6 Testimoni pelanggan	Foto avatar PNG, *grid card* interaktif
7	*Stats*	Statistik utama klinik	Animasi *counter* CSS, *background* PNG

8	*Virtual Tour*	Foto-foto fasilitas klinik	5 foto PNG, galeri interaktif
No	*Section*	Deskripsi Konten	*Elemen Multimedia*
9	*Map & Contact*	Lokasi dan informasi kontak	*Embed* Google Maps *iframe*
10	*FAQ*	Delapan pertanyaan umum	Komponen *accordion* CSS
11	*Footer*	Navigasi, kontak, hak cipta	Tautan dan teks

3.4 Implementasi Prototipe

Tahap implementasi merupakan tahap di mana seluruh keputusan perancangan yang telah ditetapkan pada sub-bab sebelumnya diwujudkan secara teknis menjadi dua prototipe *landing page* yang dapat berjalan dan diakses secara publik. Bagian ini menjelaskan secara rinci proses pembangunan kedua prototipe, mencakup konfigurasi arsitektur *rendering*, cara kerja masing-masing pendekatan dalam menghasilkan halaman, struktur direktori, serta implementasi Islands Architecture pada prototipe Astro JS. Pemahaman terhadap perbedaan implementasi teknis antara keduanya menjadi konteks yang penting sebelum membaca hasil pengujian, karena perbedaan itulah yang secara langsung menghasilkan perbedaan performa yang terukur pada tahap selanjutnya.

Prototipe pertama dibangun menggunakan Astro JS versi 5.1.8 dengan pendekatan *Static Site Generation* (SSG) yang diperkuat oleh penerapan Islands Architecture, sedangkan prototipe kedua menggunakan React JS versi 18.3.1 dengan pendekatan *Client-Side Rendering* (CSR) berbasis Vite versi 6.0.5. Setelah proses implementasi selesai, kedua prototipe di-*deploy* ke platform Vercel agar dapat diakses melalui URL publik yang digunakan sebagai target pengujian pada seluruh instrumen yang ditetapkan.

3.4.1 Implementasi Prototipe Astro JS

Prototipe Astro JS dibangun dengan menggabungkan *Static Site Generation* (SSG) sebagai metode rendering dasar dan *Islands Architecture* sebagai mekanisme penanganan interaktivitas, sebagaimana telah dijelaskan pada sub-bab 2.3.

Konfigurasi utama prototipe Astro JS ditetapkan melalui berkas astro.config.mjs sebagai berikut.

```
import { defineConfig } from 'astro/config';
import tailwind from '@astrojs/tailwind';
import react from '@astrojs/react';

export default defineConfig({
  output: 'static',
  integrations: [tailwind(), react()],
});
```

Terdapat tiga elemen konfigurasi yang perlu mendapat perhatian khusus. Pertama, baris output: 'static' merupakan instruksi eksplisit kepada Astro JS untuk beroperasi dalam mode SSG penuh. Kedua, tailwind() mengaktifkan Tailwind CSS melalui integrasi resmi @astrojs/tailwind sehingga seluruh *styling* diproses sepenuhnya pada saat *build time*. Ketiga, react() melalui @astrojs/react merupakan integrasi yang ditambahkan sebagai konsekuensi dari penerapan *Islands Architecture*; tanpa integrasi ini, Astro JS tidak dapat memproses komponen React sebagai *island* interaktif sebagaimana dijelaskan pada sub-bab 2.3. Hal yang menarik dari konfigurasi ini adalah bahwa kedua prototipe pada dasarnya sama-sama menggunakan React sebagai *library* komponen, perbedaannya bukan pada *library* yang digunakan, melainkan pada cara pengirimannya ke *browser*.

Cara kerja SSG pada prototipe ini dapat diamati secara langsung melalui berkas index.astro yang merupakan halaman utama aplikasi.

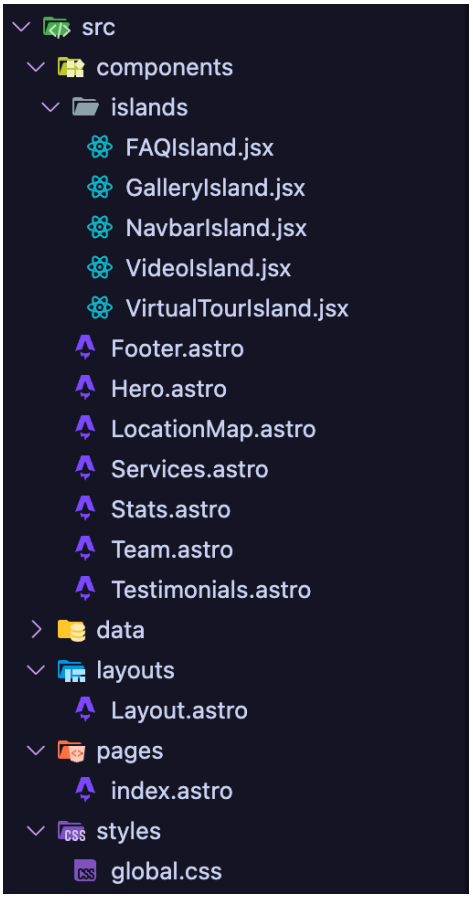
```
---
import Layout from "../layouts/Layout.astro";
import Hero from "../components/Hero.astro";
import Services from "../components/Services.astro";
import Team from "../components/Team.astro";
import Testimonials from "../components/Testimonials.astro";
import Stats from "../components/Stats.astro";
import LocationMap from "../components/LocationMap.astro";
import Footer from "../components/Footer.astro";
// Islands komponen interaktif dengan client:* directive
import NavbarIsland from "../components/islands/NavbarIsland.jsx";
import GalleryIsland from "../components/islands/GalleryIsland.jsx";
import VideoIsland from "../components/islands/VideoIsland.jsx";
import VirtualTourIsland from "../components/islands/VirtualTourIsland.jsx";
```

```
import FAQIsland from "../components/islands/FAQIsland.jsx";
---

<Layout>
  <NavbarIsland client:load />
  <main>
    <Hero />
    <Services />
    <GalleryIsland client:visible />
    <VideoIsland client:visible />
    <Team />
    <Testimonials />
    <Stats />
    <VirtualTourIsland client:visible />
    <LocationMap />
    <FAQIsland client:visible />
  </main>
  <Footer />
</Layout>
```

Berkas index.astro tersebut menggambarkan implementasi *Islands Architecture* pada prototipe ini secara langsung. Dari dua belas komponen yang menyusun *landing page*, tujuh komponen bersifat statis dan dirender sebagai HTML murni pada saat *build*, yaitu Hero, Services, Team, Testimonials, Stats, LocationMap, dan Footer. Lima komponen sisanya difungsikan sebagai *island* interaktif menggunakan komponen React (.jsx) dengan *client directive* yang disesuaikan berdasarkan posisinya di halaman. NavbarIsland menggunakan client:load agar langsung interaktif sejak halaman dimuat, sedangkan GalleryIsland, VideoIsland, VirtualTourIsland, dan FAQIsland menggunakan client:visible karena keempatnya berada di bawah *fold*, sesuai dengan mekanisme *partial hydration* yang telah dijelaskan pada sub-bab 2.3.

Selanjutnya pada Struktur direktori prototipe Astro JS, sebagaimana ditunjukkan pada Gambar 3.2, mencerminkan pemisahan yang jelas antara komponen statis dan komponen interaktif, sekaligus menggambarkan Islands Architecture secara visual melalui Struktur atau organisasi berkasnya.



Gambar 3.2 Struktur Direktori Astro JS

Direktori components/ terbagi menjadi dua kelompok, komponen .astro untuk elemen statis dan subdirektori islands/ yang memuat lima komponen React (.jsx) untuk elemen interaktif. Direktori layouts/ memuat Layout.astro sebagai pembungkus HTML dasar yang bertanggung jawab atas pemuatan Google Fonts, pengaturan *meta tag*, dan struktur dokumen HTML secara keseluruhan. Direktori pages/ memuat index.astro sebagai satu-satunya halaman dalam aplikasi. Direktori styles/ memuat global.css yang mendefinisikan seluruh animasi CSS *Keyframes*. Direktori data/ memuat dua belas berkas JavaScript sebagai sumber data terpusat yang diimpor oleh setiap komponen.

3.4.2 Implementasi React JS

Prototipe React JS dibangun menggunakan Vite sebagai *build tool* dengan pendekatan *Client-Side Rendering* (CSR) sebagaimana telah dijelaskan pada sub-bab 2.3. Konfigurasi proyek ditetapkan melalui vite.config.js sebagai berikut.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwind from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwind()],
});
```

Konfigurasi Vite pada prototipe ini hanya memuat dua *plugin*, yaitu react() untuk mendukung sintaks JSX dan pemrosesan komponen React, serta tailwindcss() untuk mengintegrasikan Tailwind CSS versi 4. Tidak ada *plugin* tambahan seperti SSR, *code splitting* manual, atau optimasi *bundle* di luar konfigurasi *default* Vite yang ditambahkan. Keputusan ini disengaja untuk mempertahankan pendekatan CSR murni dalam kondisi standar yang umum digunakan oleh pengembang pada umumnya, sehingga hasil pengujian benar-benar merepresentasikan performa CSR sebagaimana yang lazim ditemukan di lingkungan produksi nyata.

Perbedaan fundamental antara kedua pendekatan *rendering* terlihat paling jelas pada berkas main.jsx yang merupakan *entry point* dari seluruh aplikasi React JS. Jika pada prototipe Astro JS titik masuk aplikasi adalah berkas index.astro yang memproses seluruh komponen pada saat *build time* dan menghasilkan HTML lengkap sebelum ada satu pun pengguna yang mengaksesnya, maka pada prototipe React JS titik masuk aplikasinya bekerja dengan cara yang sepenuhnya berbeda.

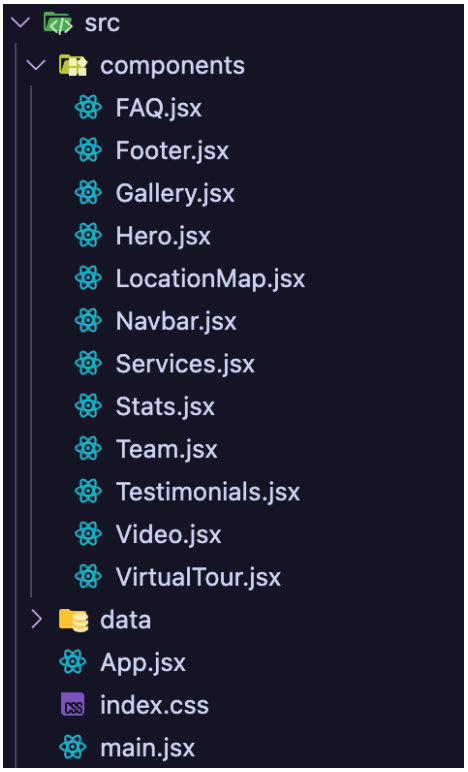
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Fungsi createRoot pada berkas tersebut bertugas memasang seluruh aplikasi React ke dalam elemen DOM dengan ID root. Proses pemasangan ini baru dapat dieksekusi setelah *bundle* JavaScript berhasil diunduh dan di-*parse* oleh *browser*, sebuah prasyarat yang tidak dimiliki oleh Astro JS karena Astro JS telah

menyelesaikan seluruh proses *rendering* pada saat *build time*. Berbeda dengan prototipe Astro JS yang hanya meng-*hydrate* lima komponen interaktif secara selektif, React JS memperlakukan seluruh dua belas komponen penyusun halaman secara setara, semuanya dirender oleh JavaScript di sisi *browser* tanpa terkecuali, tidak peduli apakah komponen tersebut bersifat statis atau interaktif. Hero, Services, Team, Testimonials, Stats, LocationMap, dan Footer yang pada prototipe Astro JS cukup dikirim sebagai HTML statis, pada prototipe React JS tetap harus melalui proses *rendering* JavaScript di *browser* meskipun kontennya tidak pernah berubah secara dinamis. Inilah inefisiensi mendasar CSR ketika diterapkan pada halaman yang sebagian besar kontennya bersifat statis.

Struktur direktori prototipe React JS, sebagaimana ditunjukkan pada Gambar 3.3, mencerminkan pendekatan CSR yang memperlakukan seluruh komponen secara setara tanpa pemisahan antara elemen statis dan interaktif, sekaligus menggambarkan bagaimana React JS mengorganisasi kode dalam paradigma *single bundle* yang dikirimkan sepenuhnya ke *browser*.



Gambar 3.3 Struktur Direktori React JS

Direktori components/ memuat dua belas komponen berekstensi .jsx dengan nama yang identik dengan komponen pada prototipe Astro JS. Berbeda dengan prototipe Astro JS yang memisahkan komponen statis dan interaktif ke dalam direktori yang berbeda, seluruh komponen React JS berada dalam satu direktori yang sama tanpa pemisahan, mencerminkan bahwa dalam paradigma CSR, tidak ada perbedaan perlakuan antara komponen statis dan interaktif karena keduanya sama-sama dirender oleh JavaScript di *browser*. Direktori data/ memuat dua belas berkas JavaScript yang isinya persis sama dengan berkas data/ pada prototipe Astro JS, menjamin tidak ada perbedaan dalam hal data yang dirender. Di tingkat *root* direktori src/ terdapat App.jsx sebagai komponen induk, main.jsx sebagai *entry point*, dan index.css sebagai berkas *styling* global

3.4.3 Perbandingan Build Output

Sebelum masuk ke tahap pengujian eksternal, dilakukan analisis terhadap *build output* yang dihasilkan oleh masing-masing prototipe menggunakan perintah npm run build. Perbandingan ini penting karena ukuran dan komposisi *build output* berkorelasi langsung dengan metrik performa yang diukur pada tahap pengujian — semakin besar *bundle* JavaScript yang harus diproses *browser*, semakin lama pula konten dapat ditampilkan kepada pengguna (Haryono et al., 2025). Hasil perbandingan dirangkum pada Tabel 3.4 berikut.

Tabel 3.4 Perbandingan Build Output

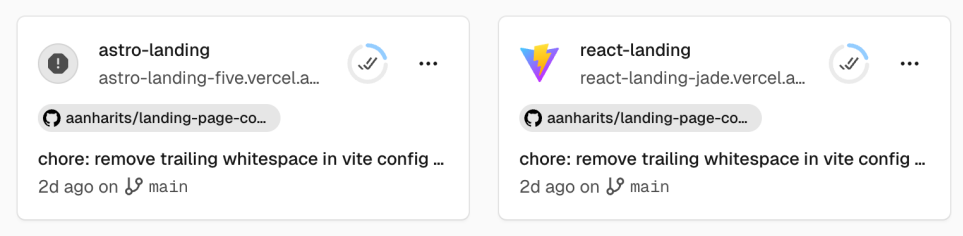
Metrik Build	Astro JS	React JS
Total ukuran HTML	85.52 KB	1.13 KB
Total ukuran JavaScript	211 KB	289.41 KB
Total ukuran CSS	23.86 KB	34.48 KB
Jumlah file JavaScript	8 file terpisah	1 file monolitik
Waktu build	1.62 detik	1.06 detik

Terdapat tiga perbedaan utama yang terlihat dari Tabel 3.4 tersebut.

a. Ukuran HTML Astro JS jauh lebih besar (85.52 KB) karena sudah berisi seluruh konten halaman secara lengkap, sementara React JS hanya menghasilkan HTML sebesar 1.13 KB yang berfungsi sebagai *shell* kosong untuk menampung eksekusi JavaScript. Hal ini menjadi bukti konkret bahwa Astro JS memindahkan pekerjaan rendering ke fase *build*, sementara React JS menyerahkannya sepenuhnya ke *browser* pengguna.
b. Meskipun total ukuran JavaScript Astro JS (±211 KB) tampak mendekati React JS (289.41 KB), tidak semuanya dikirim pada *initial load* — hanya NavbarIsland (2.44 KB) yang langsung dimuat, sementara empat *island* lainnya baru dikirim ketika komponen tersebut masuk *viewport* melalui client:visible. Dengan demikian, JavaScript yang benar-benar diproses *browser* pada *initial load* Astro JS hanya sekitar ±189 KB, sekitar 100 KB lebih ringan dibandingkan React JS yang mengirim seluruh *bundle* sekaligus.
c. Ukuran CSS Astro JS (23.86 KB) lebih kecil dibandingkan React JS (34.48 KB), disebabkan oleh mekanisme *tree-shaking* Tailwind CSS yang diproses lebih agresif pada saat *build time* di Astro JS, sehingga hanya *class* yang benar-benar digunakan yang disertakan dalam berkas CSS akhir (Zaidan & Suyatno, 2025).

3.4.4 Deployment

Kedua prototipe di-*deploy* ke Vercel dari satu repositori GitHub yang sama dengan struktur *monorepo*, di mana prototipe Astro JS berada di direktori astro-landing dan prototipe React JS di direktori react-landing. *Deployment* dilakukan secara terpisah dengan mengatur *root directory* pada konfigurasi masing-masing proyek di Vercel, sehingga setiap prototipe memiliki URL yang independent, sebagaimana ditunjukkan pada Gambar 3.4.

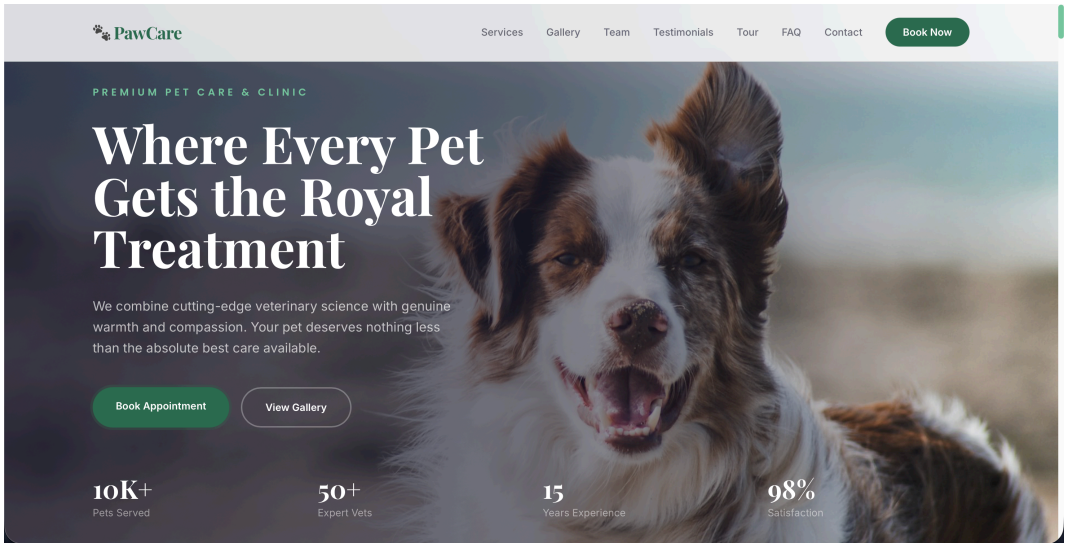


Gambar 3.4 Dashboard Vercel

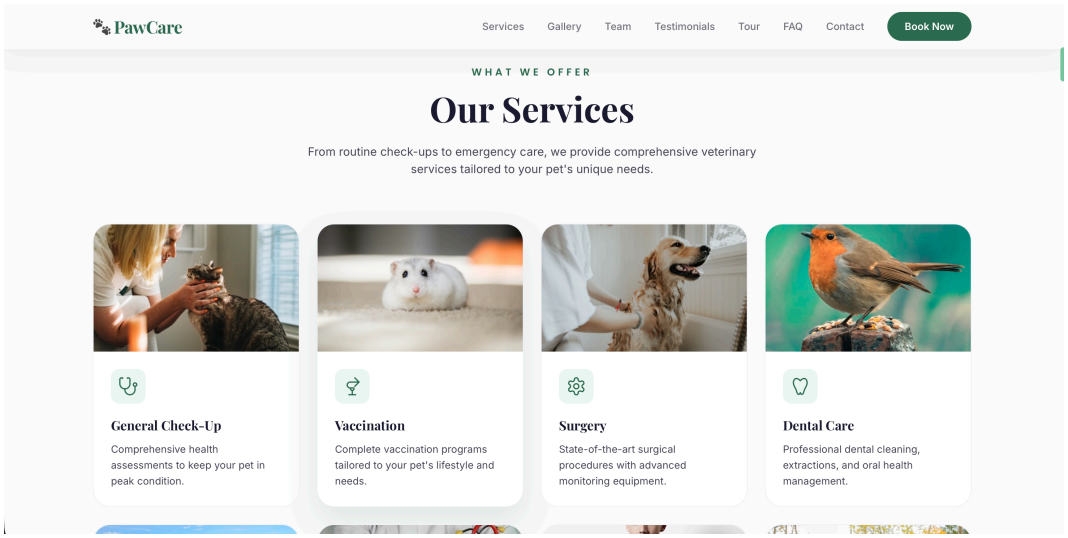
Prototipe Astro JS dapat diakses melalui *https://astro-landing-five.vercel.app* dan prototipe React JS melalui *https://react-landing-jade.vercel.app*. Kedua URL ini digunakan sebagai target pengujian pada seluruh sesi *Google Lighthouse, PageSpeed Insights*, dan *Apache JMeter.*

3.5 Hasil Prototipe

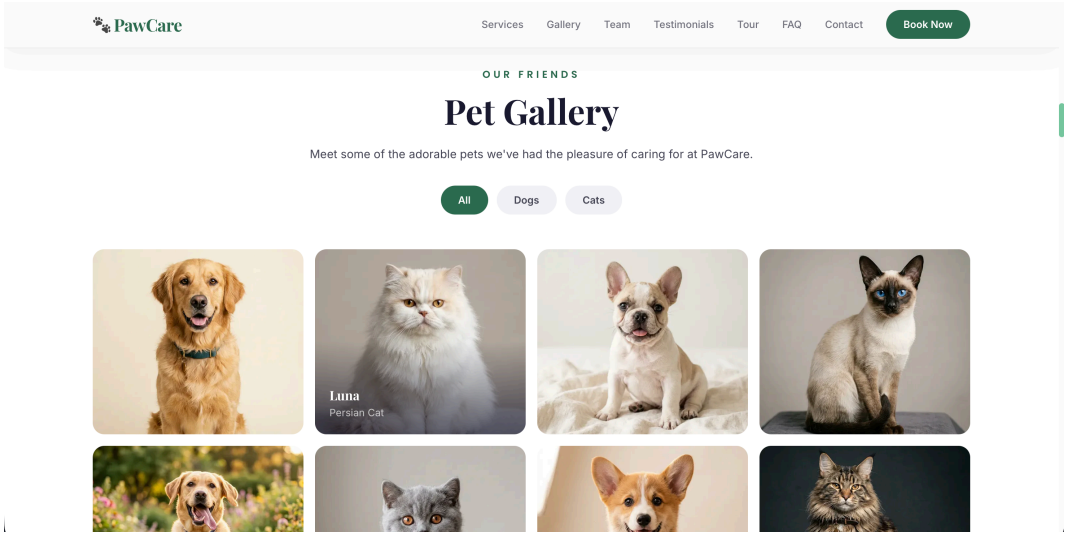
Setelah proses implementasi dan *deployment* selesai, kedua prototipe dapat diakses secara publik melalui URL Vercel masing-masing. Bagian ini menampilkan hasil tampilan visual prototipe untuk membuktikan bahwa konten dan antarmuka yang dibangun telah sesuai dengan struktur yang telah dirancang pada sub-bab 3.3, mencakup seluruh sebelas *section* mulai dari *Hero Section* hingga *Footer*.



Gambar 3.5 Hero Section

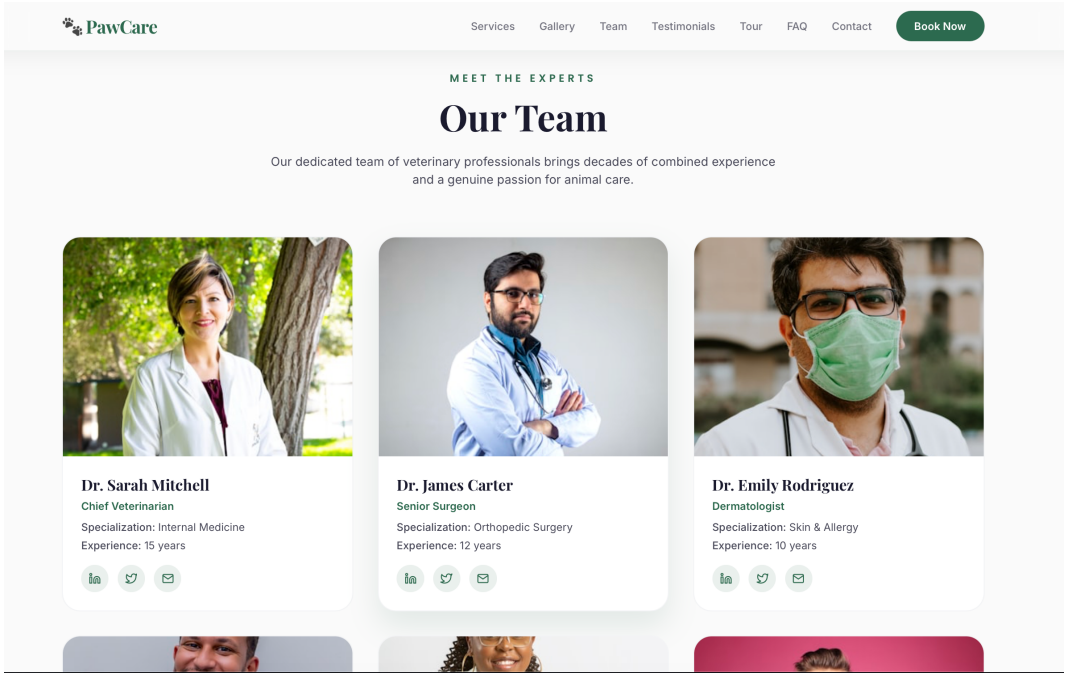


Gambar 3.6 Service Section

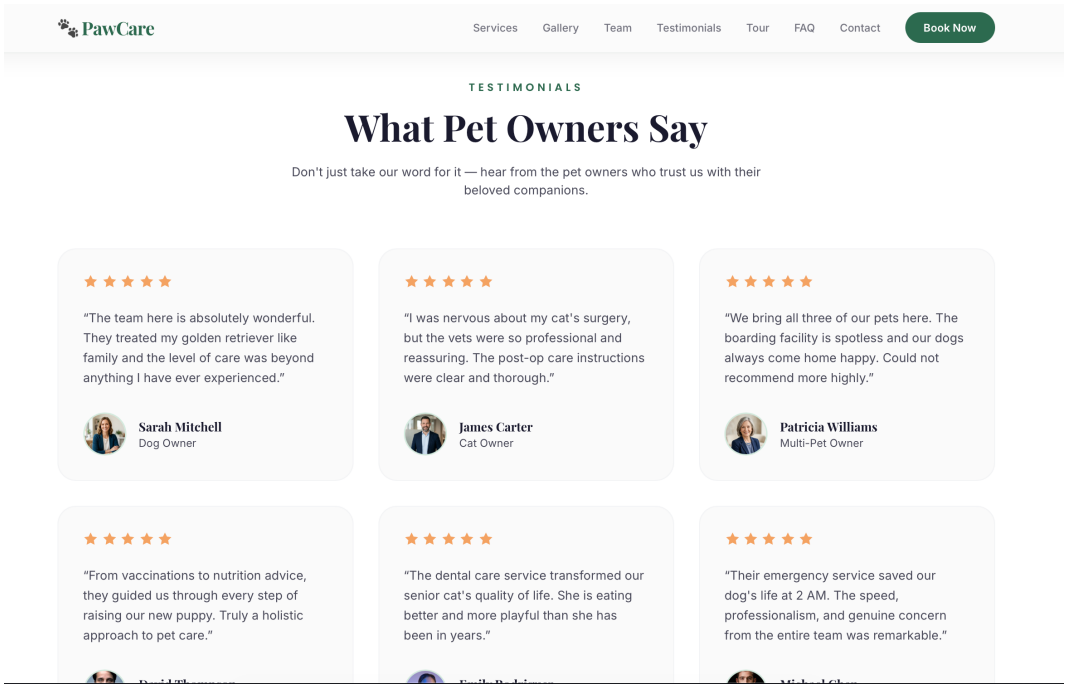


Gambar 3.7 Gallery Section

Gambar 3.8 Video Tour Section

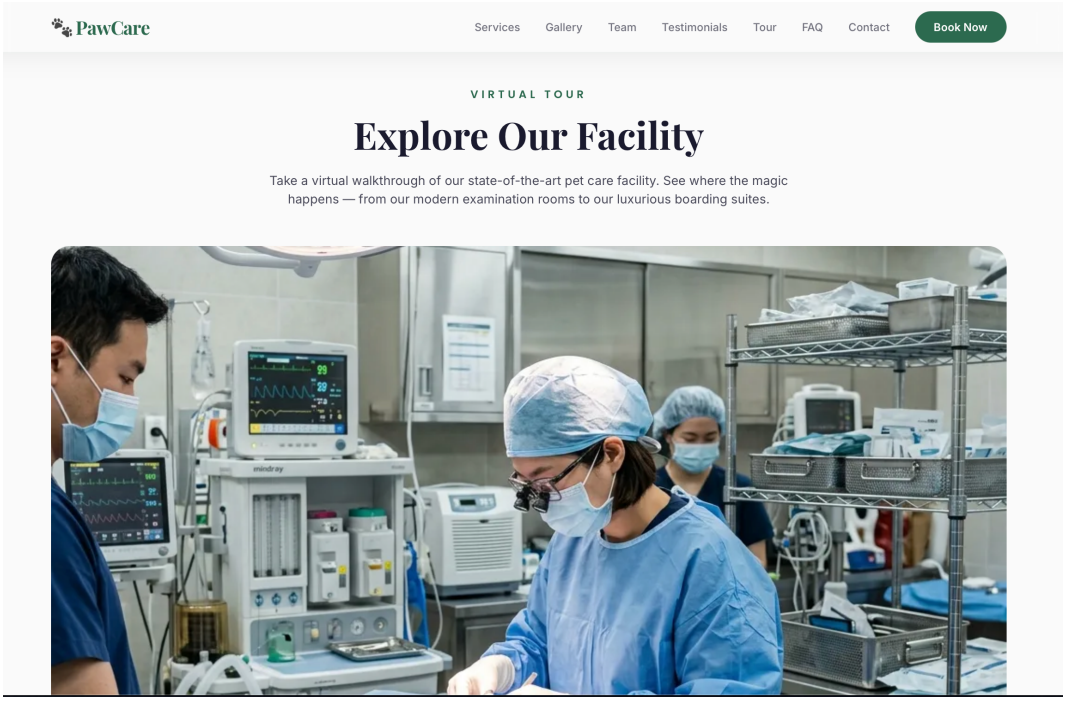


Gambar 3.9 Team Section

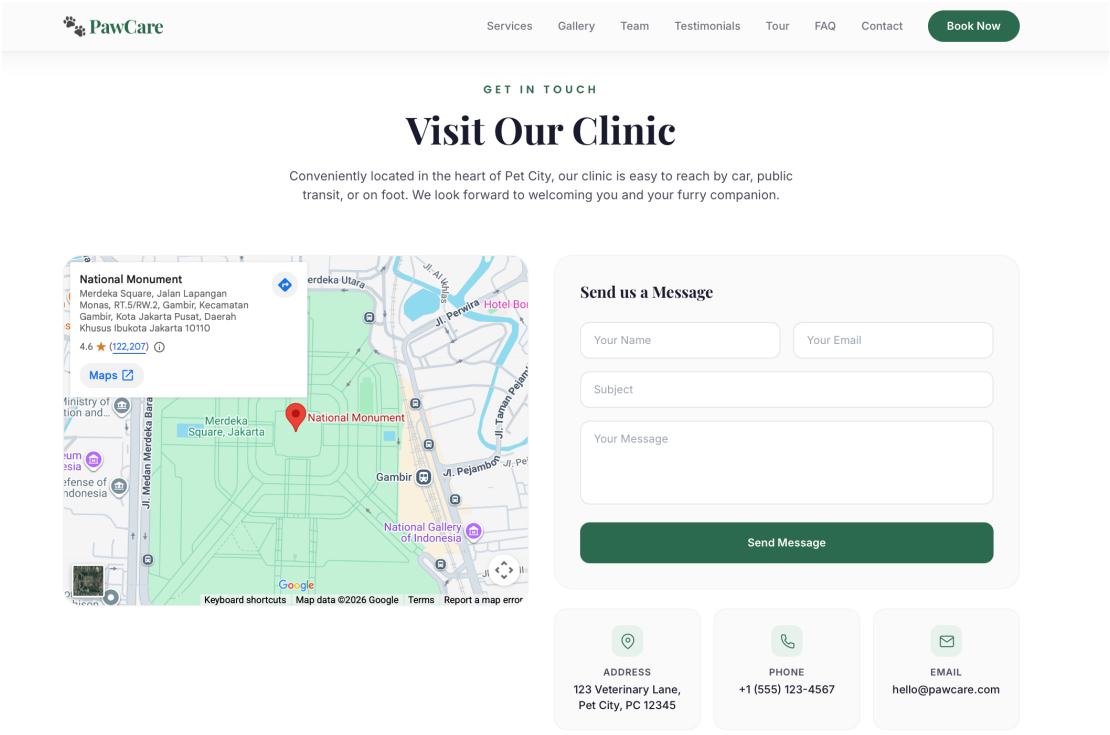


Gambar 3.10 Testimonials Section

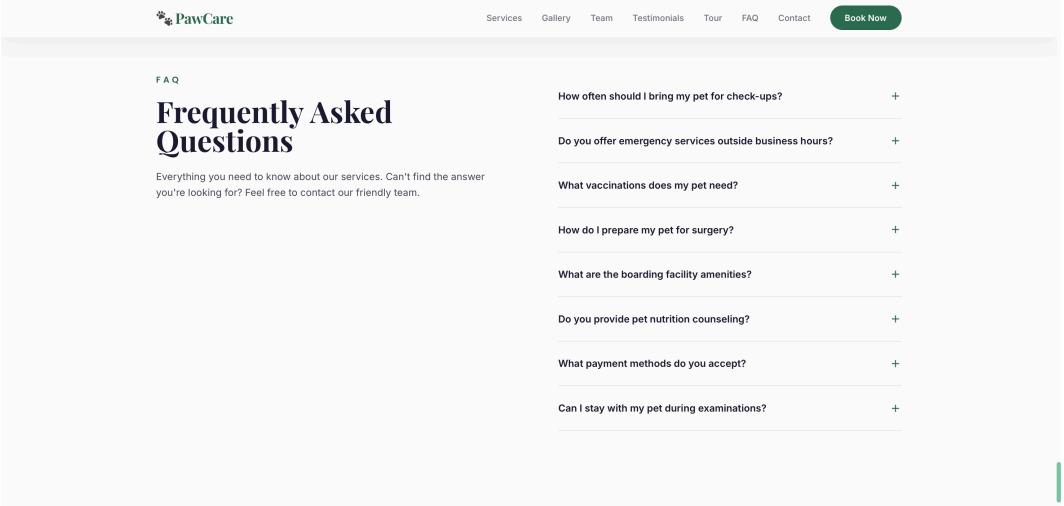
Gambar 3.11 Stats Section



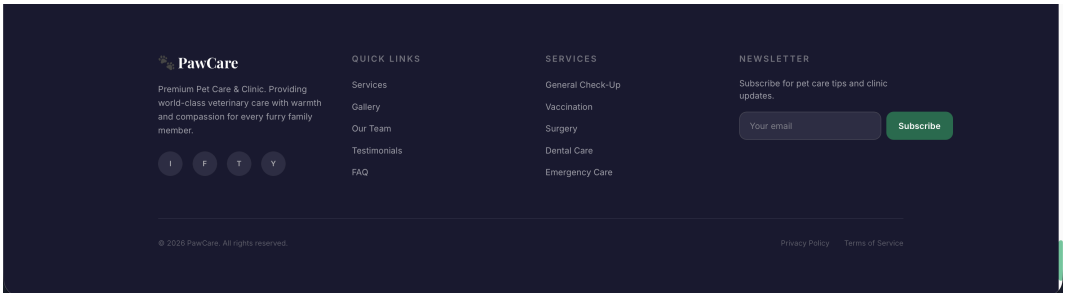
Gambar 3.12 Virtual Tour Section



Gambar 3.13 Map & Contact Section



Gambar 3.14 FAQ Section



Gambar 3.15 Footer Section

Meskipun secara visual kesebelas section pada Gambar 3.5 hingga Gambar 3.15 identik antara kedua prototipe, keduanya menghasilkan build output yang berbeda signifikan di balik tampilan yang sama tersebut, sebagaimana telah diukur pada Tabel 3.4. Prototipe Astro JS menghasilkan berkas HTML sebesar 85,52 KB yang sudah berisi seluruh konten kesebelas section secara lengkap, dengan JavaScript initial load hanya sekitar 189 KB karena hanya lima komponen island yang di-hydrate. Sebaliknya, prototipe React JS menghasilkan HTML shell kosong sebesar 1,13 KB, namun harus mengirimkan seluruh bundle JavaScript sebesar 289,41 KB sekaligus agar kesebelas section yang identik secara visual tersebut dapat dirender di browser. Perbedaan build output inilah yang menjadi dasar teoretis dari perbedaan performa yang diukur pada pengujian di sub-bab 3.6 dan 3.7.

3.6 Skenario Pengujian Peforma

Skenario Pengujian performa dilakukan terhadap kedua prototipe yang sudah *live* di Vercel menggunakan tiga instrumen secara berurutan, yaitu Google Lighthouse, PageSpeed Insights, dan Apache JMeter.

3.6.1 Skenario Pengujian Google Lighthouse

Google Lighthouse digunakan untuk mengukur performa *rendering* kedua prototipe melalui lima metrik utama, yaitu *First Contentful Paint* (FCP), *Largest Contentful Paint* (LCP), *Total Blocking Time* (TBT), *Cumulative Layout Shift* (CLS), dan *Speed Index* (SI). Pengujian dijalankan langsung melalui tab *Lighthouse* pada *Chrome DevTools* dengan mengakses URL Vercel masing-masing prototipe.

Untuk menjaga konsistensi hasil, beberapa prosedur standar diterapkan sebelum setiap sesi pengujian. Seluruh aplikasi lain di perangkat ditutup selama pengujian berlangsung dan jeda minimal 30 detik diberikan di antara setiap iterasi untuk memastikan kondisi *browser* kembali stabil. Pengujian dilakukan pada dua mode perangkat, yaitu *mobile* dan *desktop*, dengan konfigurasi *throttling* bawaan Lighthouse untuk mensimulasikan kondisi jaringan yang realistis, sebagaimana dirangkum pada Tabel 3.5.

Tabel 3.5 Konfigurasi Pengujian Google Lighthouse

Parameter	Keterangan
Mode Pengujian	*Navigation*
Kategori	*Performance*
Mode Perangkat	*Mobile* dan *Desktop*
Jumlah Iterasi	5 kali
Target	URL *Deployment* Vercel
Metrik	FCP, LCP, TBT, CLS, SI

Perlu dicatat bahwa hasil pengujian ini turut dipengaruhi oleh kondisi perangkat dan koneksi internet pada saat pengujian dilakukan, sehingga nilai yang diperoleh dapat sedikit berfluktuasi antar sesi pengujian meskipun dilakukan pada perangkat dan jaringan yang sama. Untuk meminimalisir dampak fluktuasi tersebut, setiap pengujian dilakukan sebanyak lima iterasi yang kemudian dirata-ratakan sebagai

nilai akhir. Perlu digarisbawahi bahwa pengujian Lighthouse pada penelitian ini dijalankan langsung pada perangkat dan koneksi jaringan lokal peneliti.

3.6.2 Skenario Pengujian Pagespeed Insights

PageSpeed Insights digunakan sebagai instrumen pengujian performa kedua yang melengkapi Google Lighthouse. Pengujian dilakukan melalui laman *pagespeed.web.dev* dengan memasukkan URL *deployment* Vercel masing-masing prototipe. Keunggulan *PageSpeed Insights* terletak pada kemampuannya menggabungkan *lab data* dari simulasi Lighthouse dengan *field data* dari *Chrome User Experience Report* (CrUX), sehingga memberikan gambaran performa yang lebih komprehensif dibandingkan pengujian Lighthouse secara langsung (Google, 2024).

Metrik yang diukur sama dengan pengujian Lighthouse, yaitu FCP, LCP, TBT, CLS, dan SI, pada mode *mobile* dan *desktop*. Setiap kombinasi prototipe dan mode perangkat diuji sebanyak lima kali iterasi dengan jeda antar pengujian untuk menghindari hasil yang bias akibat *caching*. Konfigurasi lengkap pengujian ditunjukkan pada Tabel 3.6.

Tabel 3.6 Konfigurasi Pengujian Pagespeed Insights

Parameter	Keterangan
Platform	pagespeed.web.dev
Kategori	Performance
Mode Perangkat	*Mobile* dan *Desktop*
Jumlah Iterasi	5 kali
Target	URL *Deployment* Vercel
Metrik	FCP, LCP, TBT, CLS, SI

Perlu dicatat bahwa berbeda dengan Lighthouse yang dijalankan pada perangkat dan koneksi jaringan lokal peneliti, PageSpeed Insights dijalankan pada infrastruktur server Google dengan profil perangkat dan kalibrasi throttling yang telah ditetapkan secara baku, sehingga kedua instrumen berpotensi menghasilkan nilai yang sedikit berbeda meskipun mengukur mode dan metrik yang sama.

3.6.3 Skenario Pengujian Apache JMeter

Apache JMeter digunakan untuk mengevaluasi ketahanan kedua prototipe terhadap lonjakan beban pengguna secara bersamaan. Pengujian dirancang dalam tiga skenario beban yang berbeda, yaitu 100, 200, dan 300 pengguna virtual (*threads*), untuk mensimulasikan kondisi trafik dari ringan hingga sedang. *Ramp-up period* diatur secara proporsional, 10 detik untuk 100 *threads*, 20 detik untuk 200 *threads*, dan 30 detik untuk 300 *threads*, agar simulasi mendekati kondisi trafik yang lebih realistis di mana pengguna tidak semuanya mengakses halaman pada waktu yang bersamaan (Permatasari et al., 2020).

Setiap skenario dikonfigurasi menggunakan *Thread Group* dengan *loop count* satu dan protokol HTTPS yang mengarah ke URL *deployment* Vercel masing-masing prototipe. Hasil pengujian diambil dari *Aggregate Report* pada panel *listener* JMeter yang mencakup tiga metrik utama, yaitu *average response time* (ms), *throughput* (req/s), dan *error rate* (%), Sebagaimana dirangkum pada Tabel 3.7.

Tabel 3.7 Konfigurasi Pengujian Apache Jmeter

Parameter	Skenario 1	Skenario 2	Skenario 3
Number of Threads	100	200	300
Ramp-Up Period	10 detik	20 detik	30 detik
Loop Count	1	1	1
Protocol	HTTPS	HTTPS	HTTPS
Method	GET	GET	GET
Jumlah Iterasi	5 kali	5 kali	5 kali
Metrik	*Response Time*, *Throughput*, *Error Rate*		

Perlu dicatat bahwa hasil pengujian ini turut dipengaruhi oleh kondisi infrastruktur eksternal di luar kendali penelitian, seperti kondisi jaringan internet pada saat pengujian dilakukan dan kondisi *platform hosting* Vercel yang bersifat *shared infrastructure* dengan pengguna lain di seluruh dunia. Selain itu, karena kedua prototipe diakses melalui *deployment* Vercel yang menerapkan mekanisme *caching*

pada *Edge Network*-nya, hasil pengujian JMeter perlu dipahami sebagai representasi waktu pengambilan berkas HTML dari CDN (*content delivery network*), bukan sebagai waktu *rendering* penuh yang dialami pengguna di sisi *browser*. Hal ini berlaku khususnya pada React JS, di mana JMeter hanya mengukur waktu pengambilan berkas *HTML shell* kosong dari CDN, tanpa mencakup proses eksekusi JavaScript dan *rendering* komponen yang baru terjadi setelah berkas tersebut diterima oleh *browser* pengguna, Untuk meminimalisir dampak variabel-variabel tersebut, dan setiap skenario pengujian dilakukan sebanyak lima iterasi yang kemudian dirata-ratakan sebagai nilai akhir.

3.7 Hasil & Analisis Pengujian

Bagian ini menyajikan seluruh data hasil pengujian yang diperoleh dari tiga instrumen, yaitu Google Lighthouse, PageSpeed Insights, dan Apache JMeter, beserta analisis komparatifnya. Setiap instrumen menghasilkan data dari lima iterasi pengujian yang kemudian hasilnya dirata-ratakan untuk mendapatkan nilai representatif masing-masing prototipe. Hasil pengujian disajikan dalam bentuk tabel per iterasi, dan visualisasi grafik batang bedasarkan rata-rata yang sudah di kalkulasikan untuk memudahkan perbandingan performa secara visual.

3.7.1 Hasil Pengujian Google Lighthouse

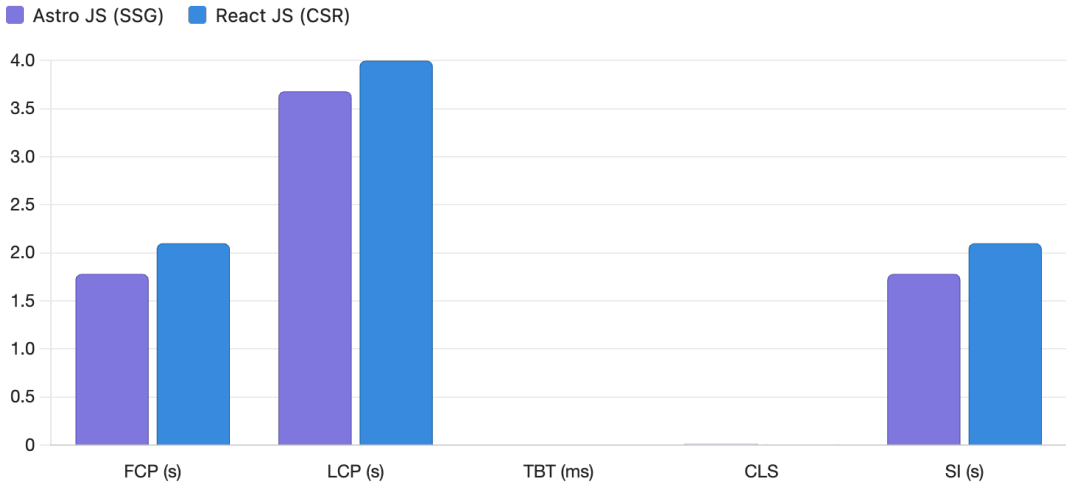
Pengujian Google Lighthouse dilakukan sebanyak lima iterasi untuk setiap prototipe serta Mode *Mobile* dan *Desktop*. Nilai rata-rata dari lima iterasi tersebut digunakan sebagai dasar perbandingan performa antara Astro JS dan React JS.

Tabel 3.8 Hasil Pengujian Lighthouse Astro JS Mode Mobile

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	1.7s	4.1s	0ms	0.013	1.7s
2	1.6s	3.6s	0ms	0.013	1.6s
3	1.6s	3.4s	0ms	0.013	1.6s
4	1.6s	4.0s	0ms	0.013	1.6s
5	2.4s	3.3s	0ms	0.013	2.4s
Rata-rata	1.78	3.68	0	0.013	1.78

Tabel 3.9 Hasil Pengujian Lighthouse React JS Mode Mobile

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	2.5s	3.7s	0ms	0.043	2.5s
2	2.6s	4.1s	0ms	0.013	2.6s
3	2.4s	4.1s	0ms	0.013	2.4s
4	2.4s	3.5s	0ms	0.013	2.4s
5	1.6s	4.6s	0ms	0	1.6s
Rata-rata	2.30	4.00	0	0.016	2.30



Gambar 3.16 Visualisasi Hasil Pengujian Lighthouse Mode Mobile

Berdasarkan hasil pengujian Google Lighthouse mode *mobile* pada Tabel 3.8 dan Tabel 3.9, serta divisualisasikan pada Gambar 3.16, Astro JS mengungguli React JS pada metrik FCP, LCP, dan SI dengan nilai masing-masing 1.78s, 3.68s, dan 1.78s dibandingkan React JS sebesar 2.30s, 4.00s, dan 2.30s. Keunggulan ini disebabkan oleh pendekatan SSG yang memungkinkan *browser* langsung menerima HTML lengkap tanpa menunggu eksekusi JavaScript (Lazuardy & Anggraini, 2022). Untuk metrik TBT, kedua prototipe sama-sama mencatat 0ms karena Astro JS tidak mengirimkan JavaScript ke *browser* sama sekali, sementara *bundle* React JS yang dihasilkan Vite cukup ringan sehingga tidak menimbulkan *blocking time*. Perbedaan CLS antara keduanya yakni 0.013 dan 0.016 tidak signifikan.

Setelah pengujian mode *mobile* selesai dilakukan, pengujian dilanjutkan pada mode *desktop* untuk melihat apakah pola keunggulan Astro JS tetap konsisten pada

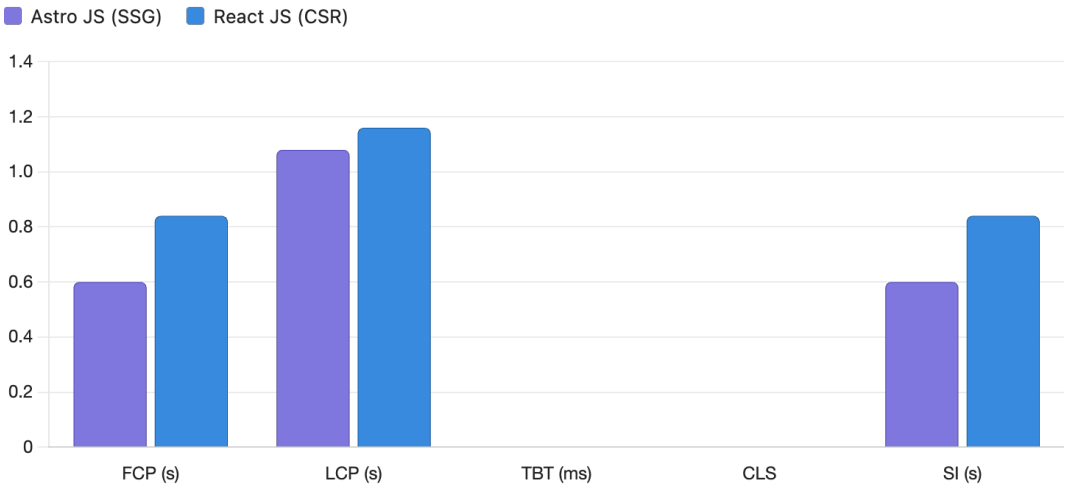
kondisi perangkat yang lebih *powerful*. Perbedaan mendasar antara kedua mode pada Google Lighthouse terletak pada simulasi kondisi perangkat, mode *mobile* menerapkan *CPU throttling* dan simulasi jaringan yang terbatas, sementara mode *desktop* tidak menerapkan pembatasan tersebut sehingga seluruh kemampuan pemrosesan perangkat dapat dimanfaatkan secara penuh. Kondisi ini secara teoritis akan mempersempit gap performa antara SSG dan CSR karena *bundle* JavaScript React JS dapat dieksekusi lebih cepat tanpa hambatan *throttling*.

Tabel 3.10 Hasil Pengujian Lighthouse Astro JS Mode Desktop

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	0.8s	1.0s	0ms	0	0.8s
2	0.5s	1.1s	0ms	0	0.5s
3	0.5s	1.1s	0ms	0	0.5s
4	0.7s	1.0s	0ms	0	0.7s
5	0.5s	1.2s	0ms	0	0.5s
Rata-rata	0.60	1.08	0	0	0.60

Tabel 3.11 Hasil Pengujian Lighthouse React JS Mode Desktop

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	1.0s	1.1s	0ms	0	1.0s
2	0.9s	1.1s	0ms	0	0.9s
3	0.8s	1.2s	0ms	0	0.8s
4	0.9s	1.2s	0ms	0	0.9s
5	0.6s	1.2s	0ms	0	0.6s
Rata-rata	0.84	1.16	0	0	0.84



Gambar 3.17 Visualisasi Hasil pengujian Lighthouse Mode Desktop

Bedasarkan Tabel 3.10 dan Tabel 3.11, serta divisualisasikan pada Gambar 3.17, Pada mode *desktop*, Astro JS tetap konsisten unggul dengan FCP 0.60s, LCP 1.08s, dan SI 0.60s dibandingkan React JS sebesar 0.84s, 1.16s, dan 0.84s. Selisih antar kedua prototipe pada mode *desktop* lebih kecil dibandingkan mode *mobile* karena tidak adanya *CPU throttling* membuat *bundle* JavaScript React JS dapat dieksekusi lebih cepat, sehingga gap performa antara SSG dan CSR menjadi lebih sempit (Khasanah & Suartana, 2024).

3.7.2 Hasil Pengujian Pagespeed Insights

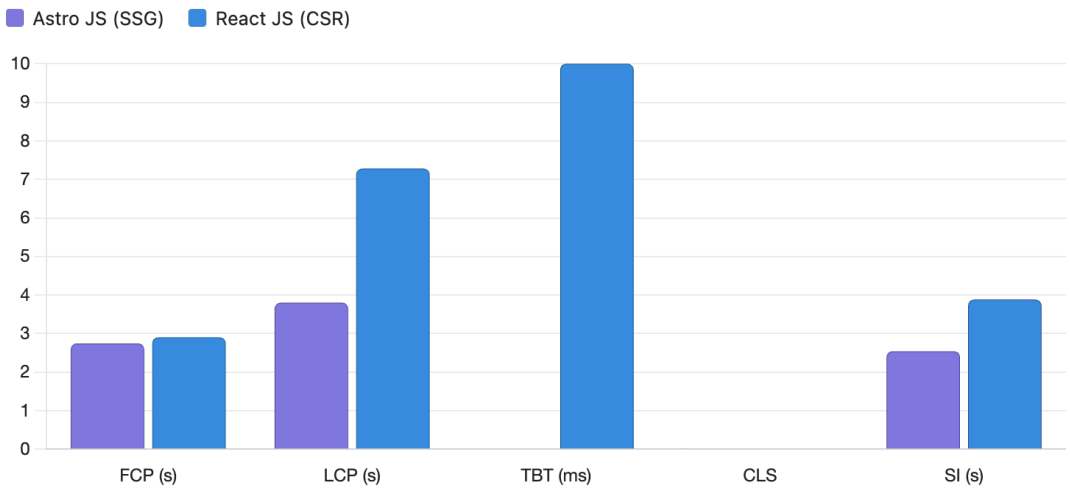
Pengujian PageSpeed Insights dilakukan sebanyak lima iterasi untuk setiap prototipe pada mode *Mobile* dan *Desktop*. Berbeda dengan Lighthouse yang sepenuhnya menggunakan *lab data*, PageSpeed Insights menggabungkan *lab data* dengan *field data* dari *Chrome User Experience Report (CrUX)* yang mencerminkan kondisi penggunaan nyata pengguna di berbagai jaringan dan perangkat, sehingga hasil yang diperoleh cenderung lebih tinggi dari Lighthouse dan lebih merepresentasikan kondisi di lapangan (Google, 2024).

Tabel 3.12 Hasil Pengujian Pagespeed Astro JS Mode Mobile

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	2.7s	3.6s	0ms	0.012	2.7s
2	2.6s	3.6s	0ms	0.012	1.6s
3	2.7s	4.1s	0ms	0.012	2.7s
4	2.7s	4.0s	0ms	0.012	2.7s
5	3.0s	3.7s	0ms	0.012	3.0s
Rata-rata	2.74	3.80	0ms	0.012	2.54

Tabel 3.13 Hasil Pengujian Pagespeed React JS Mode Mobile

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	2.9s	4.0s	0ms	0	3.2s
2	2.9s	9.7s	0ms	0	4.8s
3	2.9s	4.0s	10ms	0	3.8s
4	2.9s	9.0s	20ms	0	2.9s
5	2.9s	9.7s	20ms	0	4.7s
Rata-rata	2.90	7.28	10ms	0	3.88



Gambar 3.18 Visualisasi Hasil Pengujian Pagespeed Mode Mobile

Hasil pengujian PageSpeed Insights mode *mobile pada Tabel 3.12 dan Tabel 3.13, serta divisulisasikan pada Gambar 3.18,* menunjukkan Astro JS lebih unggul pada metrik FCP (2.74s vs 2.90s), LCP (3.80s vs 7.28s), TBT (0ms vs 10ms), dan SI (2.54s vs 3.88s). Perbedaan yang paling mencolok terlihat pada metrik LCP React JS yang mencapai 7.28s, jauh lebih tinggi dibandingkan hasil Lighthouse sebesar 4.00s dan kemunculan TBT sebesar 10ms pada React JS yang konsisten di PSI, sementara 0ms pada Lighthouse, hal ini disebabkan oleh perbedaan lingkungan eksekusi lab data antara kedua instrumen, Lighthouse dijalankan pada perangkat dan koneksi jaringan lokal peneliti, sedangkan PageSpeed Insights dijalankan pada infrastruktur server Google dengan baseline kecepatan pemrosesan yang berbeda, sehingga satu proses JavaScript yang nyaris mencapai ambang blocking time dapat tercatat berbeda pada kedua lingkungan tersebut

Setelah pengujian mode mobile selesai dilakukan, pengujian dilanjutkan pada mode desktop untuk memverifikasi konsistensi temuan pada kondisi perangkat yang berbeda. Sebagaimana pada Lighthouse, perbedaan nilai metrik antara mode mobile dan desktop pada PageSpeed Insights disebabkan oleh perbedaan simulasi kondisi perangkat, mode mobile menerapkan CPU throttling dan simulasi jaringan yang lebih terbatas untuk merepresentasikan perangkat kelas menengah, sementara mode desktop tidak menerapkan pembatasan tersebut sehingga nilai metrik yang dihasilkan cenderung lebih rendah dan lebih stabil. Dengan demikian, pengujian

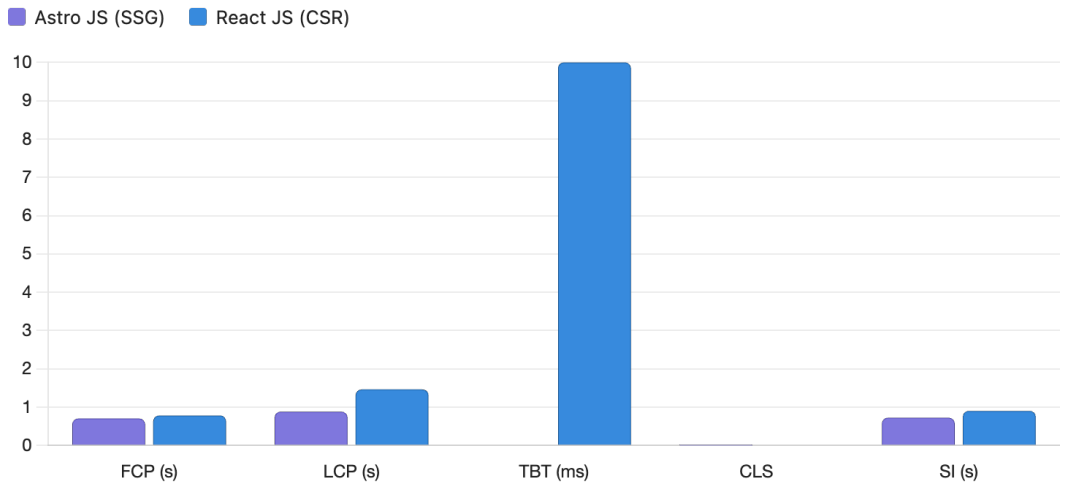
mode desktop pada PSI memberikan gambaran performa kedua prototipe pada kondisi perangkat yang lebih optimal, konsisten dengan prinsip pengujian pada Google Lighthouse.

Tabel 3.14 Hasil Pengujian Pagespeed Astro JS Mode Desktop

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	0.7s	0.9s	0ms	0.001	0.7s
2	0.7s	0.9s	0ms	0.001	0.7s
3	0.7s	0.9s	0ms	0.001	0.8s
4	0.7s	0.9s	0ms	0.001	0.7s
5	0.7s	0.8s	0ms	0.001	0.7s
Rata-rata	0.70	0.88	0ms	0.001	0.72

Tabel 3.15 Hasil Pengujian Pagespeed React JS Mode Desktop

Iterasi	FCP (s)	LCP (s)	TBT (ms)	CLS	SI (s)
1	0.8s	1.8s	20ms	0	1.0s
2	0.7s	1.0s	0ms	0	0.7s
3	0.8s	1.8s	10ms	0	0.9s
4	0.8s	0.9s	0ms	0	0.8s
5	0.8s	1.8s	20ms	0	1.1s
Rata-rata	0.78	1.46	10ms	0	0.90



Gambar 3.19 Visualisasi Hasil Pengujian Pagespeed Mode Desktop

Bedasarkan Tabel 3.14 dan Tabel 3.15, serta divisulisasikan pada Gambar 3.19, Pada mode desktop, Astro JS kembali unggul pada metrik FCP (0.70s vs 0.78s), LCP (0.88s vs 1.46s), TBT (0ms vs 10ms), dan SI (0.72s vs 0.90s). Pola kemunculan TBT sebesar 10ms pada React JS yang konsisten pada mode mobile

maupun desktop di PSI, sementara 0ms pada Lighthouse, disebabkan oleh perbedaan lingkungan eksekusi lab data antara kedua instrumen sebagaimana telah dijelaskan pada hasil pengujian mode mobile. Nilai LCP React JS pada mode desktop juga lebih tinggi di PSI (1.46s) dibandingkan Lighthouse (1.16s) dengan selisih 0.30s, yang turut mencerminkan perbedaan lingkungan eksekusi tersebut. Secara keseluruhan hasil PSI mengonfirmasi temuan Lighthouse bahwa Astro JS secara konsisten mengungguli React JS pada metrik performa rendering, khususnya pada LCP dan SI yang berkaitan langsung dengan kecepatan pemuatan konten berat seperti gambar PNG dan video MP4 lokal (Zaidan & Suyatno, 2025).

3.7.3 Hasil Pengujian JMeter

Pengujian Apache JMeter dilakukan untuk mengevaluasi dimensi performa yang tidak dapat diukur oleh Google Lighthouse maupun PageSpeed Insights, yaitu ketahanan sistem ketika diakses banyak pengguna secara bersamaan, mengingat kedua instrumen tersebut hanya mensimulasikan satu pengguna dalam satu waktu. Ketiga skenario beban sebagaimana telah dijelaskan pada sub-bab 3.6.3 dijalankan sebanyak lima iterasi, dan hasilnya dirata-ratakan untuk memastikan konsistensi data.

Skenario pertama mensimulasikan kondisi trafik ringan dengan 100 pengguna virtual yang mengakses kedua prototipe secara bersamaan dalam rentang ramp-up selama 10 detik.

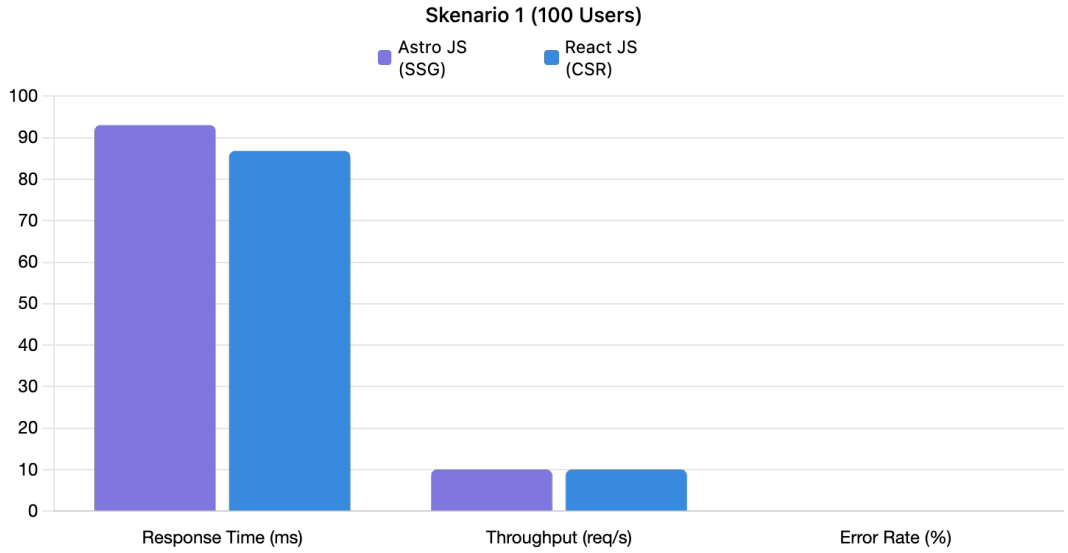
Tabel 3.16 Hasil Pengujian Jmeter Skenario 1 Astro JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	92	10.0	0.00
2	94	10.0	0.00
3	93	10.0	0.00
4	92	10.0	0.00
5	94	10.0	0.00
Rata-rata	93	10.0	0.00

Tabel 3.17 Hasil Pengujian Jmeter Skenario 1 React JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	88	10.0	0.00

2	86	10.0	0.00
3	84	10.0	0.00
4	84	10.0	0.00
5	92	10.0	0.00
Rata-rata	86.8	10.0	0.00



Gambar 3.20 Visualisasi Hasil Pengujian Jmeter Skenario 1

Bedasarkan Tabel 3.16 dan Tabel 3.17, serta divisualisasikan pada Gambar 3.20, Pada skenario beban ringan dengan 100 pengguna virtual, kedua prototipe menunjukkan performa yang hampir setara dari sisi *response time* server. Astro JS mencatat rata-rata 93 ms sementara React JS mencatat 86.8 ms, selisih hanya 6.2 ms yang secara praktis tidak signifikan. Kedua prototipe juga mencatat *throughput* yang identik sebesar 10.0/sec dan *error rate* 0.00% di seluruh iterasi. Kesetaraan performa pada skenario ini wajar terjadi karena beban 100 *user* masih tergolong ringan bagi infrastruktur CDN Vercel, sehingga perbedaan ukuran berkas HTML antara kedua *framework* belum memberikan dampak yang berarti terhadap kecepatan respons server.

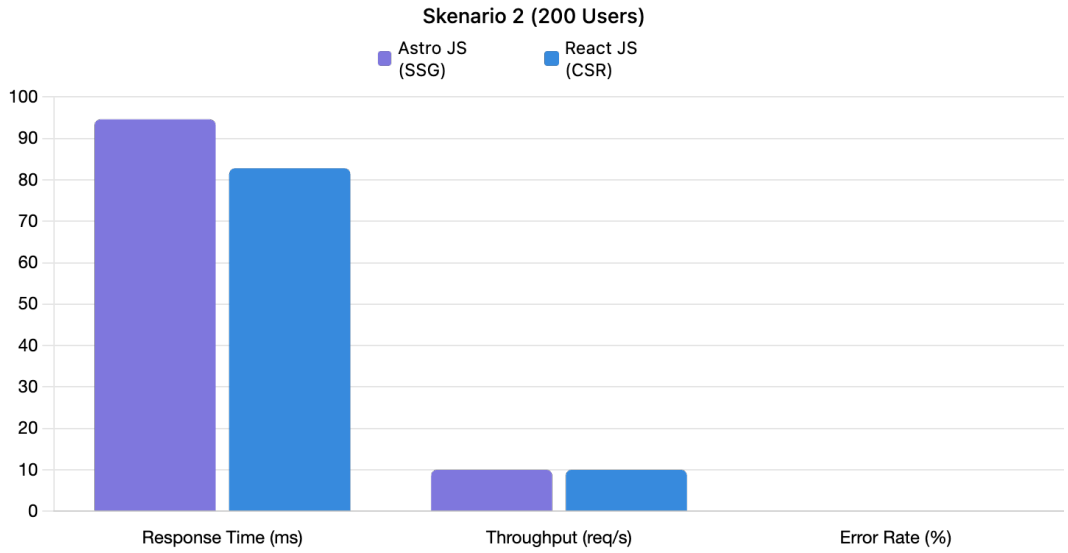
Setelah mengetahui bahwa kedua prototipe sama-sama stabil pada beban ringan, pengujian dilanjutkan pada skenario kedua dengan jumlah pengguna virtual yang dua kali lipat lebih besar untuk melihat apakah kesetaraan performa ini tetap konsisten atau performanya menurun atau meningkat ketika beban mulai meningkat ke kondisi trafik sedang.

Tabel 3.18 Hasil Pengujian Jmeter Skenario 2 Astro JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	98	10.0	0.00
2	94	10.0	0.00
3	94	10.0	0.00
4	95	10.0	0.00
5	92	10.0	0.00
Rata-rata	94.6	10.0	0.00

Tabel 3.19 Hasil Pengujian Jmeter Skenario 2 React JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	81	10.0	0.00
2	83	10.0	0.00
3	81	10.0	0.00
4	83	10.0	0.00
5	86	10.0	0.00
Rata-rata	82.8	10.0	0.00



Gambar 3.21 Visualisasi Hasil Pengujian Jmeter Skenario 2

Bedasarkan Tabel 3.18 dan 3.19, serta divisualisasikan pada Gambar 3.21, Pada skenario beban sedang dengan 200 pengguna virtual, pola yang serupa dengan skenario sebelumnya kembali terlihat React JS mencatat rata-rata *response time* sebesar 82.8 ms**,** sedikit lebih cepat dibandingkan Astro JS sebesar 94.6 ms, dengan selisih 11.8 ms. Kedua prototipe tetap mencatat *throughput* yang identik sebesar

10.0/sec dan *error rate* 0.00% di seluruh iterasi tanpa satu pun kegagalan *request*. Konsistensi *error rate* 0% pada beban yang dua kali lebih besar dari skenario sebelumnya menunjukkan bahwa kedua prototipe masih sangat mampu menangani peningkatan jumlah pengguna virtual tanpa mengalami penurunan stabilitas, sementara selisih *response time* yang sedikit melebar dari skenario sebelumnya (6.2 ms menjadi 11.8 ms) mengindikasikan bahwa ukuran berkas HTML mulai memberikan dampak yang sedikit lebih terasa terhadap kecepatan respons server seiring meningkatnya beban.

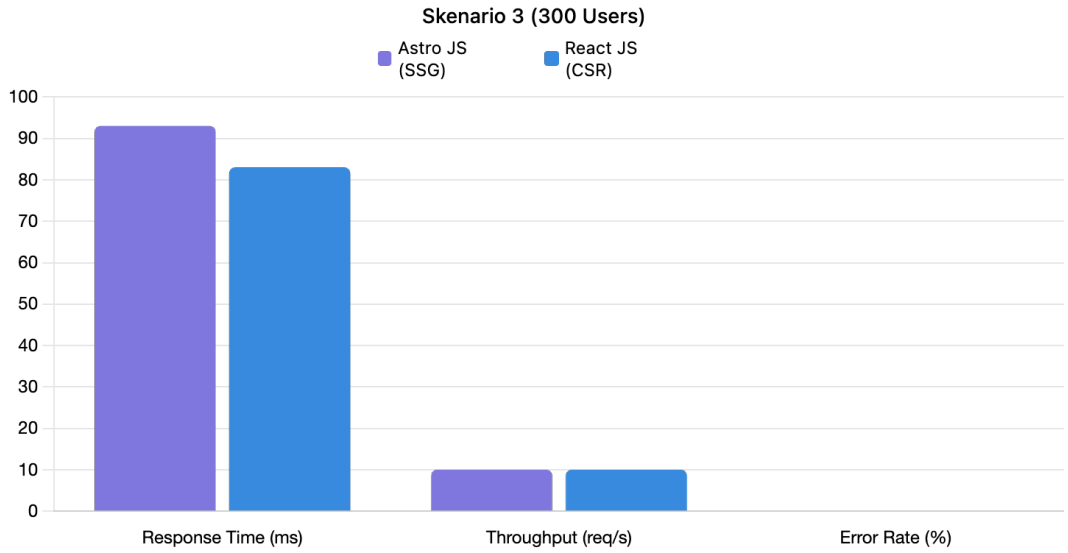
Untuk menguji batas ketahanan kedua prototipe pada kondisi beban yang lebih berat, pengujian dilanjutkan pada skenario ketiga dengan jumlah pengguna virtual tertinggi dalam penelitian ini, guna melihat apakah pola kestabilan ini tetap konsisten atau justru mulai menunjukkan tanda-tanda penurunan performa pada salah satu maupun kedua prototipe.

Tabel 3.20 Hasil Pengujian Jmeter Skenario 3 Astro JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	94	10.0	0.00
2	95	10.0	0.00
3	92	10.0	0.00
4	91	10.0	0.00
5	93	10.0	0.00
Rata-rata	93	10.0	0.00

Tabel 3.21 Hasil Pengujian Jmeter Skenario 3 React JS

Iterasi	Response Time (ms)	Throughput (req/s)	Error Rate (%)
1	82	10.0	0.00
2	82	10.0	0.00
3	83	10.0	0.00
4	86	10.0	0.00
5	82	10.0	0.00
Rata-rata	83	10.0	0.00



Gambar 3.22 Visualisasi Hasil Pengujian Jmeter Skenario 3

Bedasarkan Tabel 3.20 dan Tabel 3.21 serta divisualisasikan pada Gambar 3.22, Pada skenario beban tertinggi dengan 300 pengguna virtual, pola yang konsisten dengan dua skenario sebelumnya tetap terlihat, React JS mencatat rata-rata *response time* sebesar 83 ms, sedikit lebih cepat dibandingkan Astro JS sebesar 93 ms**,** dengan selisih 10 ms. Kedua prototipe tetap mencatat *throughput* yang identik sebesar 10.0/sec dan yang paling penting, *error rate* tetap 0.00% di seluruh iterasi meskipun beban telah ditingkatkan tiga kali lipat dari skenario pertama. Konsistensi *error rate* 0% pada beban tertinggi yang diuji dalam penelitian ini membuktikan bahwa kedua prototipe baik yang dibangun dengan pendekatan SSG dan Islands Architecture maupun CSR, sama-sama memiliki ketahanan yang baik terhadap lonjakan trafik hingga 300 pengguna secara bersamaan, tanpa mengalami satu pun kegagalan *request* pada infrastruktur Vercel.

Untuk melihat pola perubahan performa kedua prototipe secara menyeluruh seiring meningkatnya beban pengguna, hasil rata-rata dari ketiga skenario dirangkum pada Tabel 3.22 berikut.

Tabel 3.22 Rekapitulasi Hasil Pengujian JMeter

Skenario	Metrik	Astro JS	React JS
100 Users	Response Time (ms)	93	86.8
	Throughput (req/s)	10.0	10.0
	Error Rate (%)	0.00	0.00
200 Users	Response Time (ms)	94.6	82.8
	Throughput (req/s)	10.0	10.0
	Error Rate (%)	0.00	0.00
300 Users	Response Time (ms)	93	83
	Throughput (req/s)	10.0	10.0
	Error Rate (%)	0.00	0.00

Secara keseluruhan, hasil pengujian Apache JMeter menunjukkan bahwa baik Astro JS maupun React JS sama-sama mampu menangani peningkatan beban pengguna virtual dari 100 hingga 300 secara stabil, dibuktikan dengan error rate yang konsisten 0.00% di seluruh skenario dan iterasi. React JS secara konsisten mencatat response time yang sedikit lebih rendah dibandingkan Astro JS di ketiga skenario, dengan selisih yang relatif kecil dan stabil (6.2 ms hingga 11.8 ms). Keunggulan ini disebabkan oleh ukuran berkas HTML React JS yang jauh lebih kecil (1.13 KB) dibandingkan Astro JS (85.52 KB), karena React JS hanya mengirimkan shell HTML kosong tanpa proses rendering apa pun di server, sementara Astro JS mengirimkan HTML lengkap hasil rendering pada saat build. Dengan demikian, response time yang lebih rendah pada React JS tidak dapat diinterpretasikan sebagai keunggulan performa secara menyeluruh, melainkan konsekuensi dari pendekatan Client-Side Rendering (CSR) yang memindahkan seluruh beban rendering ke sisi browser pengguna, Sejalan dengan batasan yang telah dijelaskan pada sub-bab 3.6.3, hasil pengujian JMeter pada penelitian ini merepresentasikan waktu pengambilan berkas dari CDN, bukan waktu rendering di sisi browser yang telah diukur secara terpisah melalui instrumen Lighthouse dan PageSpeed Insights.

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan hasil pengujian dan analisis komparatif yang telah dilakukan terhadap dua prototipe *landing page* berbasis Astro JS dengan pendekatan *Static Site Generation* (SSG) yang diperkuat Islands Architecture, dan React JS dengan pendekatan *Client-Side Rendering* (CSR), dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, terkait perbandingan performa berdasarkan metrik Web Vitals melalui Google Lighthouse dan PageSpeed Insights, Astro JS secara konsisten mengungguli React JS pada hampir seluruh metrik di kedua mode pengujian. Pada Google Lighthouse mode mobile, Astro JS mencatat First Contentful Paint (FCP) dan Speed Index (SI) yang lebih cepat 0.52 detik (22.6%) serta Largest Contentful Paint (LCP) lebih cepat 0.32 detik (8.0%) dibandingkan React JS. Pada mode desktop, keunggulan serupa tetap konsisten dimiliki oleh Astro JS dengan FCP dan SI lebih cepat 0.24 detik (28.6%) serta LCP lebih cepat 0.08 detik (6.9%). Keunggulan yang lebih signifikan terlihat pada hasil PageSpeed Insights, di mana pada mode mobile Astro JS mencatat FCP lebih cepat 0.16 detik (5.5%), LCP lebih cepat 3.48 detik (47.8%), dan SI lebih cepat 1.34 detik (34.5%), sementara pada mode desktop FCP lebih cepat 0.08 detik (10.3%), LCP lebih cepat 0.58 detik (39.7%), dan SI lebih cepat 0.18 detik (20.0%). Hasil ini membuktikan secara empiris bahwa pendekatan SSG yang dikombinasikan dengan Islands Architecture pada Astro JS, di mana JavaScript hanya dikirimkan secara selektif untuk komponen interaktif melalui mekanisme partial hydration, memberikan keunggulan nyata dalam kecepatan pemuatan konten dan pengalaman pengguna pada landing page dengan aset media berat, dibandingkan pendekatan CSR murni yang mengharuskan browser mengunduh dan mengeksekusi seluruh bundle JavaScript sebelum konten dapat ditampilkan.

Kedua, terkait evaluasi ketahanan dan stabilitas sistem melalui Apache JMeter pada simulasi beban 100, 200, dan 300 pengguna virtual, kedua prototipe

menunjukkan tingkat stabilitas yang sama baiknya, dibuktikan dengan *error rate* sebesar 0.00% secara konsisten di seluruh skenario dan iterasi pengujian. Pada dimensi *response time* level server, React JS justru mencatat nilai yang sedikit lebih rendah dibandingkan Astro JS pada ketiga skenario, dengan selisih yang relatif kecil dan stabil antara 6 hingga 12 milidetik. Temuan ini disebabkan oleh karakteristik *build output* React JS yang menghasilkan berkas HTML berukuran sangat kecil (1.13 KB) karena hanya berfungsi sebagai *shell* kosong, sementara Astro JS menghasilkan berkas HTML yang jauh lebih besar (85.52 KB) karena sudah berisi konten lengkap hasil *rendering* pada saat *build*. Hasil ini menunjukkan bahwa pengujian JMeter mengevaluasi dimensi performa yang berbeda dari Lighthouse dan PageSpeed Insights. JMeter mengukur kecepatan dan ketahanan *server* dalam men-*deliver* berkas statis di level jaringan, sementara Lighthouse dan PSI mengukur kecepatan *rendering* konten dari perspektif pengalaman pengguna di *browser*. Perbedaan arah hasil antara kedua jenis pengujian ini bukan merupakan kontradiksi, melainkan bukti bahwa kedua arsitektur memiliki karakteristik *trade-off* yang berbeda pada level yang berbeda pula.

Ketiga, berdasarkan keseluruhan temuan tersebut, dapat disimpulkan bahwa Astro JS dengan pendekatan SSG dan Islands Architecture lebih sesuai digunakan untuk pembangunan *landing page* dengan konten yang sebagian besar bersifat statis dan memuat aset media berat, karena memberikan keunggulan signifikan pada kecepatan pemuatan konten dan pengalaman pengguna akhir, faktor yang secara langsung berkorelasi dengan tingkat konversi sebagaimana dijelaskan pada latar belakang penelitian ini. Meskipun demikian, React JS tetap menunjukkan performa yang kompetitif dari sisi ketahanan *server* dan *response time* di level jaringan, sehingga tetap relevan digunakan untuk proyek yang membutuhkan interaktivitas tinggi dan pembaruan data secara dinamis, sesuai dengan kekuatan utama arsitektur CSR yang telah dijelaskan pada tinjauan pustaka. Pemilihan teknologi *frontend* yang tepat pada akhirnya harus disesuaikan dengan karakteristik dan kebutuhan spesifik proyek yang akan dibangun, bukan semata-mata berdasarkan tingkat popularitas atau adopsi pasar suatu *framework*.

4.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemui selama proses penelitian, terdapat beberapa saran yang dapat menjadi pertimbangan bagi penelitian selanjutnya maupun bagi pengembang *web* dalam pengambilan keputusan pemilihan teknologi *frontend*.

Bagi penelitian selanjutnya, pengujian dapat dikembangkan dengan menggunakan lebih dari satu studi kasus *landing page* dengan karakteristik konten yang berbeda-beda, mengingat penelitian ini hanya menggunakan satu studi kasus berupa *landing page* layanan klinik hewan peliharaan dengan sebelas *section* dan aset media yang relatif berat. Penggunaan studi kasus yang lebih beragam akan memberikan gambaran yang lebih komprehensif mengenai konsistensi temuan pada berbagai jenis dan kompleksitas konten *landing page*. Selain itu, penelitian selanjutnya juga dapat mempertimbangkan pengujian pada kondisi simulasi jaringan yang bervariasi, seperti kondisi 3G atau 4G dengan *throttling* tertentu, mengingat penelitian ini hanya melakukan pengujian dalam satu kondisi jaringan tanpa simulasi variasi kecepatan akses sebagaimana telah dibatasi pada ruang lingkup penelitian ini.

Penelitian selanjutnya juga dapat mempertimbangkan penggunaan infrastruktur hosting yang lebih terkontrol, seperti Virtual Private Server (VPS) atau server fisik milik sendiri, untuk meminimalisir pengaruh variabel infrastruktur eksternal yang sulit diisolasi pada platform shared hosting seperti Vercel. Meski demikian, perlu disadari bahwa setiap jenis infrastruktur tetap memiliki karakteristik variabelnya masing-masing, seperti kapasitas bandwidth dan stabilitas koneksi pada server yang dikelola secara mandiri.

Bagi pengembang *web*, hasil penelitian ini dapat dijadikan acuan teknis dalam memilih teknologi *frontend* yang sesuai dengan karakteristik proyek yang akan dibangun. Apabila proyek yang dikembangkan berupa *landing page* atau halaman dengan konten yang sebagian besar bersifat statis, penggunaan Astro JS dengan pendekatan SSG dan Islands Architecture dapat menjadi pilihan yang lebih optimal untuk memaksimalkan kecepatan pemuatan dan pengalaman pengguna. Sebaliknya, apabila proyek yang dikembangkan membutuhkan interaktivitas tinggi

dan pembaruan data secara *real-time*, React JS dengan pendekatan CSR tetap menjadi pilihan yang relevan dan kompetitif, khususnya dari sisi ketahanan *server* dalam menangani beban pengguna secara bersamaan.

DAFTAR PUSTAKA

Anggraeni, O. S. I., Sugiarto, L., & Agustin, T. (2024). Studi komparatif performa framework JavaScript modern dalam pengembangan aplikasi web. *Modem: Jurnal Informatika dan Sains Teknologi, 2*(4), 162–177. https://doi.org/10.62951/modem.v2i4.239

Apache Software Foundation. (2024). *Apache JMeter - User's Manual*. https://jmeter.apache.org/usermanual/index.html

Astro (2024). *Why Astro?* Astro Docs. https://docs.astro.build/en/concepts/why-astro/

Fatikasari, P., & Fargazzi, M. (2025). Dominasi javascript dalam pengembangan web: Studi kualitatif terhadap opini publik di platform daring. *Kohesi: Jurnal Multidisiplin Saintek, 10*(3), 1–8. https://doi.org/10.8734/Kohesi.v1i2.365

Google. (2024). *About PageSpeed Insights*. Google for Developers. https://developers.google.com/speed/docs/insights/v5/about

Google, & SOASTA. (2017). *Business growth through mobile site speed*. Think with Google. https://business.google.com/uk/think/marketing-strategies/mobile-site-speed-importance/

Haryono, M. N. F., Kharisma, A. P., & Rahman, M. A. (2025). Analisis perbandingan efisiensi dan kebergunaan aplikasi berbasis React.js dan Next.js (studi kasus: Aplikasi web game corner FILKOM UB). *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer, 9*(8). https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/15177

Hermanto, R. R., & Engel, M. M. (2025). Analisis komparatif kinerja Next.js, Nuxt.js, dan Remix.js dalam implementasi server side rendering website berita. *TIN: Terapan Informatika Nusantara, 6*(5), 450-463. https://doi.org/10.47065/tin.v6i5.8315

Khasanah, M. U., & Suartana, I. M. (2024). Perbandingan performa client side rendering pada implementasi static website (studi kasus Barokah RentCar). *JINACS (Journal of Informatics and Computer Science), 6*(2), 541–550.

Krestianti, Rr. A., Utomo, R. B., Akbar, A., Hendrianu, R., Prananingrum, L., Agustin, S. K., & Irawaty. (2024). Pengembangan landing page sebagai alat pemasaran digital yang efektif bagi umkm (studi kasus pada umkm

Dechefdefinzs). *Jurnal Digital Teknologi Informasi, 7*(1), 6–17. https://doi.org/10.32502/digital.v7i1.7155

Lazuardy, M. F. S., & Anggraini, D. (2022). Modern front end web architectures with React.js and Next.js. *International Research Journal of Advanced Engineering and Science, 7*(1).

Lucaciu, D.-L. (2026). Comparative study of React, Astro and Eleventy technologies in developing a lottery statistical analysis application. *Open Journal of Applied Sciences, 16*(3), 758–769. https://doi.org/10.4236/ojapps.2026.163046

Permatasari, D. I., Ardani, M., Ma'ulfa, A. Y., Ilhami, N., Pratama, S. G., Astuti, S. R. D., & Naufalita, N. W. (2020). Pengujian aplikasi menggunakan metode load testing dengan Apache JMeter pada sistem informasi pertanian. *JUSTIN (Jurnal Sistem dan Teknologi Informasi), 8*(1).

Stack Overflow. (2025). *Stack Overflow Developer Survey 2025: Technology*. https://survey.stackoverflow.co/2025/technology

State of JS. (2024). *Front-end frameworks*. State of JavaScript 2024. https://2024.stateofjs.com/en-US/libraries/front-end-frameworks/

Tejaya, W., Rahman, S., & Munir, A. (2023). Pengujian website invitees menggunakan metode load testing dengan Apache JMeter. *Jurnal KHARISMA Tech, 18*(1).

Trisnawati, E., & Rusdianto, R. Y. (2024). Transformasi pemasaran di era digital: Optimalisasi landing page untuk meningkatkan brand awareness pada jasa pengepul sampah-Karung Sampah. *Maeswara: Jurnal Riset Ilmu Manajemen dan Kewirausahaan, 2*(2), 6–13. https://doi.org/10.61132/maeswara.v2i2.676

Zaidan, A. J., & Suyatno, D. F. (2025). Rendering performance analysis of Astro JS, Next JS, Nuxt JS, and SvelteKit frameworks using Google Lighthouse, PageSpeed Insight, and JMeter. *Journal of Emerging Information System and Business Intelligence, 6*(1), 1–13.

Zulhijaya, Lamada, M. S., & Wahid, A. (2025). Perbandingan framework Svelte JS dengan React JS terhadap performa web application berbasis javascript. *Jurnal Riset Teknik Komputer, 2*(2), 66–81.

LAMPIRAN

Lampiran 1. Listing Program

astro.config.mjs

```
import { defineConfig } from 'astro/config';
import tailwind from '@astrojs/tailwind';
import react from '@astrojs/react';

export default defineConfig({
  output: 'static',
  integrations: [tailwind(), react()],
});
```

src/pages/index.astro (entry point)

```
---
import Layout from "../layouts/Layout.astro";
import Hero from "../components/Hero.astro";
import Services from "../components/Services.astro";
import Team from "../components/Team.astro";
import Testimonials from "../components/Testimonials.astro";
import Stats from "../components/Stats.astro";
import LocationMap from "../components/LocationMap.astro";
import Footer from "../components/Footer.astro";

// Islands — komponen interaktif dengan client:* directive
import NavbarIsland from "../components/islands/NavbarIsland.jsx";
import GalleryIsland from "../components/islands/GalleryIsland.jsx";
import VideoIsland from "../components/islands/VideoIsland.jsx";
import VirtualTourIsland from "../components/islands/VirtualTourIsland.jsx";
import FAQIsland from "../components/islands/FAQIsland.jsx";
---

<Layout>
  <NavbarIsland client:load />
  <main>
    <Hero />
    <Services />
```

```
    <GalleryIsland client:visible />
    <VideoIsland client:visible />
    <Team />
    <Testimonials />
    <Stats />
    <VirtualTourIsland client:visible />
    <LocationMap />
    <FAQIsland client:visible />
  </main>
  <Footer />
</Layout>
```

src/layouts/Layout.astro

```
---
// src/layouts/Layout.astro
import "../styles/global.css";
---

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta
      name="description"
      content="PawCare — Premium Pet Care & Clinic. Trusted veterinary
services with 10K+ pets served, 50+ expert vets, and 15 years of excellence."
    />
    <title>PawCare — Premium Pet Care & Clinic</title>

    <!-- Preconnect to Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />

    <!-- Google Fonts: Playfair Display, Inter, Poppins (render-blocking for
research) -->
    <link

href="https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;
```

```
600;700&family=Inter:wght@300;400;500;600&family=Poppins:wght@400;
600&display=swap"
     rel="stylesheet"
   />
  </head>
  <body class="bg-background text-text font-body antialiased">
   <slot />

   <script is:inline>
    document.addEventListener("DOMContentLoaded", function () {
     // Intersection Observer for scroll animations
     const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
       if (entry.isIntersecting) {
        entry.target.classList.add('is-visible');
        observer.unobserve(entry.target);
       }
      });
     }, { threshold: 0.1, rootMargin: '0px 0px -50px 0px' });

     document.querySelectorAll('.animate-on-scroll').forEach((el) => {
      observer.observe(el);
     });
    });
   </script>
  </body>
</html>
```

Footer.astro

```
---
import { FOOTER_DATA } from '../data/footer.js';
---

<footer class="bg-text py-20">
  <div class="max-w-7xl mx-auto px-6 lg:px-8">
   <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-12 mb-16">
    <div class="lg:col-span-1">
     <h3 class="text-2xl font-heading font-bold text-white mb-4">🐾
{FOOTER_DATA.brand}</h3>
```

```
      <p class="text-sm text-white/60 leading-relaxed mb-
6">{FOOTER_DATA.tagline}. Providing world-class veterinary care with
warmth and compassion for every furry family member.</p>
      <div class="flex gap-4">
        {FOOTER_DATA.social.map((s) => (
          <a href={s.href} class="w-10 h-10 rounded-full bg-white/10 flex
items-center justify-center text-white/60 hover:bg-primary hover:text-white
transition-all duration-300" aria-label={s.name}>
            <span class="text-xs font-bold">{s.name.charAt(0)}</span>
          </a>
        ))}
      </div>
    </div>

    <div>
      <h4 class="text-sm font-semibold uppercase tracking-[0.15em] text-
white/40 mb-5">Quick Links</h4>
      <ul class="space-y-3">
        {FOOTER_DATA.quickLinks.map((link) => (
          <li><a href={link.href} class="text-sm text-white/60 hover:text-
secondary transition-colors duration-300">{link.label}</a></li>
        ))}
      </ul>
    </div>

    <div>
      <h4 class="text-sm font-semibold uppercase tracking-[0.15em] text-
white/40 mb-5">Services</h4>
      <ul class="space-y-3">
        {FOOTER_DATA.services.map((link) => (
          <li><a href={link.href} class="text-sm text-white/60 hover:text-
secondary transition-colors duration-300">{link.label}</a></li>
        ))}
      </ul>
    </div>

    <div>
      <h4 class="text-sm font-semibold uppercase tracking-[0.15em] text-
white/40 mb-5">Newsletter</h4>
      <p class="text-sm text-white/60 mb-4">Subscribe for pet care tips and
clinic updates.</p>
      <form class="flex gap-2">
```

```
        <input type="email" placeholder="Your email" class="newsletter-input flex-1 px-4 py-3 rounded-xl bg-white/10 border border-white/10 text-white text-sm placeholder-white/40" />
        <button type="submit" class="px-5 py-3 bg-primary text-white text-sm font-semibold rounded-xl hover:bg-secondary transition-colors duration-300">Subscribe</button>
       </form>
     </div>
   </div>

   <div class="pt-8 border-t border-white/10 flex flex-col sm:flex-row justify-between items-center gap-4">
     <p class="text-xs text-white/30">&copy; 2026 {FOOTER_DATA.brand}. All rights reserved.</p>
     <div class="flex gap-6">
       <a href="#" class="text-xs text-white/30 hover:text-white/60 transition-colors duration-300">Privacy Policy</a>
       <a href="#" class="text-xs text-white/30 hover:text-white/60 transition-colors duration-300">Terms of Service</a>
     </div>
   </div>
  </div>
</footer>
```

Hero.astro

```
---
import { HERO_DATA } from '../data/hero.js';
---

<section id="hero" class="relative min-h-screen flex items-center overflow-hidden">
  <!-- Background Image with Overlay -->
  <div class="absolute inset-0">
    <img
      src={HERO_DATA.image}
      alt="Happy pets at PawCare clinic"
      class="w-full h-full object-cover"
      width="1920"
      height="1080"
      loading="eager"
```

```
      fetchpriority="high"
      decoding="async"
    />
    <div class="absolute inset-0 bg-gradient-to-r from-text/80 via-text/60 to-transparent"></div>
  </div>

  <!-- Content -->
  <div class="relative z-10 max-w-7xl mx-auto px-6 lg:px-8 py-32 lg:py-40 w-full">
    <div class="max-w-2xl">
      <p class="hero-animate text-sm font-accent font-semibold uppercase tracking-[0.3em] text-secondary mb-6">
        {HERO_DATA.subtitle}
      </p>

      <h1 class="hero-animate-delay-1 text-5xl sm:text-6xl lg:text-7xl font-heading font-bold leading-[1.1] text-white mb-8">
        {HERO_DATA.title}
      </h1>

      <p class="hero-animate-delay-2 text-lg text-white/70 leading-relaxed max-w-lg mb-10 font-body">
        {HERO_DATA.description}
      </p>

      <!-- CTA Buttons with CSS keyframe animation -->
      <div class="hero-animate-delay-3 flex flex-wrap gap-4">
        <a
          href="#contact"
          class="cta-pulse px-8 py-4 bg-primary text-white text-sm font-semibold rounded-full hover:bg-primary-dark transition-colors duration-300"
        >
          {HERO_DATA.cta}
        </a>
        <a
          href="#gallery"
          class="px-8 py-4 border-2 border-white/30 text-white text-sm font-semibold rounded-full hover:border-white/60 hover:bg-white/10 transition-all duration-300"
        >
          {HERO_DATA.ctaSecondary}
```

```
      </a>
    </div>
  </div>

  <!-- Counter Stats -->
  <div class="mt-16 grid grid-cols-2 md:grid-cols-4 gap-8">
    {HERO_DATA.counters.map((counter, index) => (
      <div
        class="hero-animate-delay-3 text-center md:text-left"
        style={`animation-delay: ${0.6 + index * 0.15}s`}
      >
        <p class="text-3xl lg:text-4xl font-heading font-bold text-white mb-1">
          {counter.value}
        </p>
        <p class="text-sm text-white/50 font-body">
          {counter.label}
        </p>
      </div>
    ))}
  </div>
 </div>
</section>
```

LocationMap.astro

```
---
import { LOCATION_MAP } from "../data/location.js";
---

<section id="contact" class="py-24 lg:py-32 bg-white">
 <div class="max-w-7xl mx-auto px-6 lg:px-8">
  <div class="text-center mb-16 animate-on-scroll fade-up">
   <p class="text-sm font-accent font-semibold uppercase tracking-[0.3em]
text-primary mb-4">GET IN TOUCH</p>
   <h2 class="text-4xl lg:text-5xl font-heading font-bold text-text mb-
6">Visit Our Clinic</h2>
   <p class="text-text-light max-w-2xl mx-
auto">{LOCATION_MAP.description}</p>
  </div>

  <div class="grid grid-cols-1 lg:grid-cols-2 gap-8">
```

```
    <div class="animate-on-scroll fade-right">
      <div class="relative w-full rounded-3xl overflow-hidden bg-gray-100 border border-gray-100" style="padding-bottom: 75%">
        <iframe src={LOCATION_MAP.mapEmbed} title="PawCare Clinic Location" class="absolute inset-0 w-full h-full" style="border: 0;" allowfullscreen loading="lazy" referrerpolicy="no-referrer-when-downgrade"></iframe>
      </div>
    </div>

    <div class="animate-on-scroll fade-left">
      <form class="bg-background rounded-3xl p-8 border border-gray-100 mb-6">
        <h3 class="text-xl font-heading font-bold text-text mb-6">Send us a Message</h3>
        <div class="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-4">
          <input type="text" placeholder="Your Name" class="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm" id="contact-name" />
          <input type="email" placeholder="Your Email" class="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm" id="contact-email" />
        </div>
        <input type="text" placeholder="Subject" class="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm mb-4" id="contact-subject" />
        <textarea placeholder="Your Message" rows="4" class="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm mb-4 resize-none" id="contact-message"></textarea>
        <button type="submit" class="w-full px-8 py-4 bg-primary text-white text-sm font-semibold rounded-xl hover:bg-primary-dark transition-colors duration-300">Send Message</button>
      </form>

      <div class="grid grid-cols-1 sm:grid-cols-3 gap-4">
        <div class="bg-background rounded-2xl p-5 border border-gray-100 text-center">
          <div class="w-10 h-10 rounded-xl bg-secondary/15 flex items-center justify-center text-primary mx-auto mb-3">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5" class="w-5 h-5"><path
```

```
d="M21 10c0 7-9 13-9 13s-9-6-9-13a9 9 0 0118 0z"/><circle cx="12" cy="10"
r="3"/></svg>
          </div>
          <p class="text-xs font-semibold text-text-light uppercase tracking-
wider mb-1">Address</p>
          <p class="text-sm text-text font-
medium">{LOCATION_MAP.address}</p>
        </div>
        <div class="bg-background rounded-2xl p-5 border border-gray-100
text-center">
          <div class="w-10 h-10 rounded-xl bg-secondary/15 flex items-center
justify-center text-primary mx-auto mb-3">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" class="w-5 h-5"><path
d="M22 16.92v3a2 2 0 01-2.18 2 19.79 19.79 0 01-8.63-3.07 19.5 19.5 0 01-6-
6A19.79 19.79 0 013.12 4.18 2 2 0 014.11 2h3a2 2 0 012 1.72c.127.96.361
1.903.7 2.81a2 2 0 01-.45 2.11L8.09 9.91a16 16 0 006 6l1.27-1.27a2 2 0
012.11-.45c.907.339 1.85.573 2.81.7A2 2 0 0122 16.92z"/></svg>
          </div>
          <p class="text-xs font-semibold text-text-light uppercase tracking-
wider mb-1">Phone</p>
          <p class="text-sm text-text font-
medium">{LOCATION_MAP.phone}</p>
        </div>
        <div class="bg-background rounded-2xl p-5 border border-gray-100
text-center">
          <div class="w-10 h-10 rounded-xl bg-secondary/15 flex items-center
justify-center text-primary mx-auto mb-3">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" class="w-5 h-5"><path
d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-2-2V6c0-1.1.9-2
2-2z"/><polyline points="22,6 12,13 2,6"/></svg>
          </div>
          <p class="text-xs font-semibold text-text-light uppercase tracking-
wider mb-1">Email</p>
          <p class="text-sm text-text font-
medium">{LOCATION_MAP.email}</p>
        </div>
      </div>
    </div>
  </div>
</div>
```

```
</section>
```

Services.astro

```
---
import { SERVICES } from '../data/services.js';

const icons = {
  stethoscope:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M4.8 2.3A.3.3 0 105
2H4a2 2 0 00-2 2v5a6 6 0 006 6 6 6 0 006-6V4a2 2 0 00-2-2h-1a.2.2 0 10.3.3"
/><path d="M8 15v1a6 6 0 006 6 6 6 0 006-6v-4" /><circle cx="20" cy="10"
r="2" /></svg>',
  vaccine:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M15 2l5 5-5 5" /><path
d="M20 7H9.5a3.5 3.5 0 000 7h.5" /><path d="M12 22v-5" /><path d="M8
22h8" /><path d="M12 17a2 2 0 002-2v-1H10v1a2 2 0 002 2z" /></svg>',
  surgery:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M12.22 2h-.44a2 2 0 00-2
2v.18a2 2 0 01-1 1.73l-.43.25a2 2 0 01-2 0l-.15-.08a2 2 0 00-2.73.73l-.22.38a2
2 0 00.73 2.73l.15.1a2 2 0 011 1.72v.51a2 2 0 01-1 1.74l-.15.09a2 2 0 00-.73
2.73l.22.38a2 2 0 002.73.73l.15-.08a2 2 0 012 0l.43.25a2 2 0 011 1.73V20a2 2
0 002 2h.44a2 2 0 002-2v-.18a2 2 0 011-1.73l.43-.25a2 2 0 012 0l.15.08a2 2 0
002.73-.73l.22-.39a2 2 0 00-.73-2.73l-.15-.08a2 2 0 01-1-1.74v-.5a2 2 0 011-
1.74l.15-.09a2 2 0 00.73-2.73l-.22-.38a2 2 0 00-2.73-.73l-.15.08a2 2 0 01-2 0l-
.43-.25a2 2 0 01-1-1.73V4a2 2 0 00-2-2z" /><circle cx="12" cy="12" r="3"
/></svg>',
  dental:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M12 5.5C10 3 7 2 5.5
3.5S3 8 5 12c1.5 3 2.5 5 3 7 .5 1.5 1 2.5 2 2.5s1.5-.5 2-2c.5 1.5 1 2 2 2s1.5-1
2-2.5c.5-2 1.5-4 3-7 2-4 1-7-.5-8.5S14 3 12 5.5z" /></svg>',
  grooming:
```

```
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M7 20h10" /><path
d="M10 20c5.5-2.5.8-6.4 3-10" /><path d="M9.5 9.4c1.1.8 1.8 2.2 2.3 3.7-2
.4-3.5.4-4.8-.3-1.2-.6-2.3-1.9-3-4.2 2.8-.5 4.4 0 5.5.8z" /><path d="M14.1 6a7
7 0 00-1.1 4c1.9-.1 3.3-.6 4.3-1.4 1-1 1.6-2.3 1.7-4.6-2.7.1-4 1-4.9 2z"
/></svg>',
  emergency:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M8.5 14.5A2.5 2.5 0 0011
12c0-1.38-.5-2-1-3-1.07-2.14-1.5-3.5-1-5 .5 1.5 2 2.5 3 3 1.5 1 2 3.5 2 5.5a3 3
0 01-6 0" /><path d="M12 22v-3" /><path d="M4 22h16" /></svg>',
  nutrition:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M18 8h1a4 4 0 010 8h-1"
/><path d="M2 8h16v9a4 4 0 01-4 4H6a4 4 0 01-4-4V8z" /><line x1="6"
y1="1" x2="6" y2="4" /><line x1="10" y1="1" x2="10" y2="4" /><line
x1="14" y1="1" x2="14" y2="4" /></svg>',
  boarding:
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="1.5" stroke-linecap="round"
stroke-linejoin="round" class="w-7 h-7"><path d="M3 9l9-7 9 7v11a2 2 0 01-
2 2H5a2 2 0 01-2-2z" /><polyline points="9,22 9,12 15,12 15,22" /></svg>',
};
---

<section id="services" class="py-24 lg:py-32">
  <div class="max-w-7xl mx-auto px-6 lg:px-8">
    <!-- Section Header -->
    <div class="text-center mb-16 animate-on-scroll fade-up">
      <p class="text-sm font-accent font-semibold uppercase tracking-[0.3em]
text-primary mb-4">
        WHAT WE OFFER
      </p>
      <h2 class="text-4xl lg:text-5xl font-heading font-bold text-text mb-6">
        Our Services
      </h2>
      <p class="text-text-light max-w-2xl mx-auto">
        From routine check-ups to emergency care, we provide comprehensive
veterinary services tailored to your pet's unique needs.
```

```
    </p>
   </div>

   <!-- Services Grid -->
   <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
    {SERVICES.map((service, index) => (
     <div
      class="service-card bg-white rounded-3xl overflow-hidden border
border-gray-100 animate-on-scroll fade-up"
      style={`animation-delay: ${index * 80}ms`}
     >
      <!-- Thumbnail Image -->
      <div class="h-44 overflow-hidden">
       <img
        src={service.thumbnail}
        alt={service.title}
        class="w-full h-full object-cover transition-transform duration-500
hover:scale-110"
        width="400"
        height="176"
        loading="lazy"
        decoding="async"
       />
      </div>
      <div class="p-6">
       <div
        class="w-12 h-12 rounded-xl bg-secondary/15 flex items-center
justify-center text-primary mb-4"
        set:html={icons[service.icon]}
       ></div>
       <h3 class="text-lg font-heading font-bold text-text mb-
2">{service.title}</h3>
       <p class="text-sm text-text-light leading-relaxed">
        {service.description}
       </p>
      </div>
     </div>
    ))}
   </div>
  </div>
</section>
```

Stats.astro

```
---
import { STATS, STATS_BG } from '../data/stats.js';
---

<section class="relative py-24 lg:py-32 overflow-hidden">
  <!-- Parallax Background -->
  <div
    class="parallax-bg absolute inset-0"
    style={`background-image: url('${STATS_BG}')`}
  ></div>
  <div class="absolute inset-0 bg-primary-dark/85"></div>

  <!-- Content -->
  <div class="relative z-10 max-w-7xl mx-auto px-6 lg:px-8">
    <div class="text-center mb-12 animate-on-scroll fade-up">
      <p class="text-sm font-accent font-semibold uppercase tracking-[0.3em]
text-secondary mb-4">
        BY THE NUMBERS
      </p>
      <h2 class="text-4xl lg:text-5xl font-heading font-bold text-white">
        Trusted by Thousands
      </h2>
    </div>

    <div class="grid grid-cols-2 lg:grid-cols-4 gap-8 lg:gap-12">
      {STATS.map((stat, index) => (
        <div
          class="text-center animate-on-scroll fade-up"
          style={`animation-delay: ${index * 150}ms`}
        >
          <p class="text-5xl lg:text-6xl font-heading font-bold text-white mb-2">
            {stat.value}
          </p>
          <p class="text-sm font-accent font-medium uppercase tracking-[0.15em]
text-white/60">
            {stat.label}
          </p>
        </div>
      ))}
    </div>
```

```
  </div>
</section>
```

Team.astro

```
---
import { TEAM_MEMBERS } from '../data/team.js';
---

<section id="team" class="py-24 lg:py-32">
  <div class="max-w-7xl mx-auto px-6 lg:px-8">
    <!-- Section Header -->
    <div class="text-center mb-16 animate-on-scroll fade-up">
      <p class="text-sm font-accent font-semibold uppercase tracking-[0.3em]
text-primary mb-4">
        MEET THE EXPERTS
      </p>
      <h2 class="text-4xl lg:text-5xl font-heading font-bold text-text mb-6">
        Our Team
      </h2>
      <p class="text-text-light max-w-2xl mx-auto">
        Our dedicated team of veterinary professionals brings decades of
combined experience and a genuine passion for animal care.
      </p>
    </div>

    <!-- Team Grid -->
    <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-8">
      {TEAM_MEMBERS.map((member, index) => (
        <div
          class="team-card bg-white rounded-3xl overflow-hidden border border-
gray-100 animate-on-scroll fade-up"
          style={`animation-delay: ${index * 100}ms`}
        >
          <!-- Photo -->
          <div class="h-72 overflow-hidden">
            <img
              src={member.photo}
              alt={member.name}
              class="w-full h-full object-cover transition-transform duration-500
hover:scale-105"
```

```
            width="400"
            height="288"
            loading="lazy"
            decoding="async"
          />
        </div>
        <div class="p-6">
          <h3 class="text-xl font-heading font-bold text-text mb-
1">{member.name}</h3>
          <p class="text-primary font-semibold text-sm mb-
2">{member.role}</p>
          <p class="text-text-light text-sm mb-1">
            <span class="font-medium">Specialization:</span>
{member.specialization}
          </p>
          <p class="text-text-light text-sm mb-4">
            <span class="font-medium">Experience:</span>
{member.experience}
          </p>
          <!-- Social Icons -->
          <div class="flex gap-3">
            <a href={member.social.linkedin} class="w-9 h-9 rounded-full bg-
primary/10 flex items-center justify-center text-primary hover:bg-primary
hover:text-white transition-all duration-300" aria-label={`${member.name}
LinkedIn`}>
              <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round" class="w-4 h-4">
                <path d="M16 8a6 6 0 016 6v7h-4v-7a2 2 0 00-2-2 2 2 0 00-2
2v7h-4v-7a6 6 0 016-6z"></path>
                <rect x="2" y="9" width="4" height="12"></rect>
                <circle cx="4" cy="4" r="2"></circle>
              </svg>
            </a>
            <a href={member.social.twitter} class="w-9 h-9 rounded-full bg-
primary/10 flex items-center justify-center text-primary hover:bg-primary
hover:text-white transition-all duration-300" aria-label={`${member.name}
Twitter`}>
              <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round" class="w-4 h-4">
```

```
              <path d="M23 3a10.9 10.9 0 01-3.14 1.53 4.48 4.48 0 00-7.86
3v1A10.66 10.66 0 013 4s-4 9 5 13a11.64 11.64 0 01-7 2c9 5 20 0 20-11.5a4.5
4.5 0 00-.08-.83A7.72 7.72 0 0023 3z"></path>
            </svg>
          </a>
          <a href={`mailto:${member.social.email}`} class="w-9 h-9 rounded-
full bg-primary/10 flex items-center justify-center text-primary hover:bg-
primary hover:text-white transition-all duration-300" aria-label={`Email
${member.name}`}>
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round" class="w-4 h-4">
              <path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-
2-2V6c0-1.1.9-2 2-2z"></path>
              <polyline points="22,6 12,13 2,6"></polyline>
            </svg>
          </a>
        </div>
      </div>
    </div>
  ))}
 </div>
</section>
```

Testimonials.astro

```
---
import { TESTIMONIALS } from '../data/testimonials.js';
---

<section id="testimonials" class="py-24 lg:py-32 bg-white overflow-hidden">
  <div class="max-w-7xl mx-auto px-6 lg:px-8">
    <!-- Section Header -->
    <div class="text-center mb-16 animate-on-scroll fade-up">
      <p class="text-sm font-accent font-semibold uppercase tracking-[0.3em]
text-primary mb-4">
        TESTIMONIALS
      </p>
      <h2 class="text-4xl lg:text-5xl font-heading font-bold text-text mb-6">
        What Pet Owners Say
```

```
    </h2>
    <p class="text-text-light max-w-2xl mx-auto">
      Don't just take our word for it — hear from the pet owners who trust us with their beloved companions.
    </p>
  </div>

  <!-- Interactive Cards Grid -->
  <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8 animate-on-scroll fade-up">
    {TESTIMONIALS.map((item, index) => (
      <div class="bg-background rounded-3xl p-8 border border-gray-100 h-full hover:-translate-y-2 hover:shadow-xl hover:shadow-secondary/20 hover:border-secondary/30 transition-all duration-300 cursor-pointer flex flex-col">
        <!-- Stars -->
          <div class="flex gap-1 mb-6">
            {Array.from({ length: item.rating }).map(() => (
              <svg
                xmlns="http://www.w3.org/2000/svg"
                viewBox="0 0 24 24"
                fill="currentColor"
                class="w-5 h-5 text-accent"
              >
                <path d="M12 2l3.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 2z" />
              </svg>
            ))}
          </div>

          <!-- Quote -->
          <p class="text-text-light leading-relaxed mb-8 flex-grow">
            &ldquo;{item.quote}&rdquo;
          </p>

          <!-- Author -->
          <div class="flex items-center gap-4">
            <img
              src={item.avatar}
              alt={item.name}
              class="w-14 h-14 rounded-full object-cover border-2 border-secondary/30"
```

```
              width="56"
              height="56"
              loading="lazy"
              decoding="async"
             />
             <div>
              <p class="font-heading font-bold text-text">{item.name}</p>
              <p class="text-sm text-text-light">{item.role}</p>
             </div>
            </div>
           </div>
     ))}
    </div>
  </div>
</section>
```

islands/FAQIsland.jsx

```
import { useState } from 'react';
import { FAQ_ITEMS } from '../../data/faq_items.js';

export default function FAQIsland() {
  const [openIndex, setOpenIndex] = useState(null);

  const toggle = (index) => {
    setOpenIndex(openIndex === index ? null : index);
  };

  return (
    <section id="faq" className="py-24 lg:py-32">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-16 lg:gap-24">
          {/* Header */}
          <div className="animate-on-scroll fade-right">
            <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">FAQ</p>
            <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">
              Frequently Asked Questions
            </h2>
            <p className="text-text-light leading-relaxed">
```

```
          Everything you need to know about our services. Can&amp;apos;t find the
answer you&amp;apos;re
          looking for? Feel free to contact our friendly team.
         </p>
        </div>

        {/* Accordion */}
        <div className="animate-on-scroll fade-left" style={{ animationDelay:
'200ms' }}>
         {FAQ_ITEMS.map((item, index) => {
          const isOpen = openIndex === index;
          return (
           <div key={index} className="faq-item border-b border-gray-200">
            <button
             onClick={() => toggle(index)}
             aria-expanded={isOpen}
             className="w-full flex items-center justify-between py-6 text-
left"
            >
             <span className="text-base font-semibold text-text pr-
4">{item.question}</span>
             <span
              className={`text-primary flex-shrink-0 transition-transform
duration-300 ${isOpen ? 'rotate-45' : ''}`}
             >
              <svg
               xmlns="http://www.w3.org/2000/svg"
               viewBox="0 0 24 24"
               fill="none"
               stroke="currentColor"
               strokeWidth="2"
               className="w-5 h-5"
              >
               <line x1="12" y1="5" x2="12" y2="19" />
               <line x1="5" y1="12" x2="19" y2="12" />
              </svg>
             </span>
            </button>
            <div
             className={`overflow-hidden transition-all duration-300 ease-in-
out ${isOpen ? 'max-h-64' : 'max-h-0'}`}
            >
```

```
                <p className="pb-6 text-sm text-text-light leading-
relaxed">{item.answer}</p>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </section>
 );
}
```

islands/GalleryIsland.jsx

```
import { useState } from 'react';
import { GALLERY_ITEMS } from '../../data/gallery_items.js';
import { GALLERY_FILTERS } from '../../data/gallery_filters.js';

export default function GalleryIsland() {
 const [activeFilter, setActiveFilter] = useState('All');
 const [lightbox, setLightbox] = useState(null); // { src, name, breed }

 const filteredItems =
  activeFilter === 'All'
   ? GALLERY_ITEMS
   : GALLERY_ITEMS.filter((item) => item.category === activeFilter);

 const openLightbox = (item) => {
  setLightbox(item);
  document.body.style.overflow = 'hidden';
 };

 const closeLightbox = () => {
  setLightbox(null);
  document.body.style.overflow = '';
 };

 return (
  <section id="gallery" className="py-24 lg:py-32 bg-white">
   <div className="max-w-7xl mx-auto px-6 lg:px-8">
```

```
      {/* Section Header */}
      <div className="text-center mb-12 animate-on-scroll fade-up">
        <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">
          OUR FRIENDS
        </p>
        <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">Pet Gallery</h2>
        <p className="text-text-light max-w-2xl mx-auto mb-8">
          Meet some of the adorable pets we've had the pleasure of caring for at
PawCare.
        </p>

        {/* Filter Buttons */}
        <div className="flex flex-wrap justify-center gap-3">
          {GALLERY_FILTERS.map((filter) => (
            <button
              key={filter}
              onClick={() => setActiveFilter(filter)}
              className={`px-6 py-2.5 rounded-full text-sm font-semibold
transition-all duration-300 ${
                activeFilter === filter
                  ? 'bg-primary text-white'
                  : 'bg-gray-100 text-text-light hover:bg-gray-200'
              }`}
            >
              {filter}
            </button>
          ))}
        </div>
      </div>

      {/* Gallery Grid */}
      <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 gap-4">
        {filteredItems.map((item, index) => (
          <div
            key={item.src}
            onClick={() => openLightbox(item)}
            className="group rounded-2xl overflow-hidden relative cursor-
pointer animate-on-scroll zoom-in"
            style={{ animationDelay: `${index * 60}ms` }}
          >
```

```
            <img
              src={item.src}
              alt={`${item.name} - ${item.breed}`}
              className="w-full h-64 object-cover transition-transform duration-
500 group-hover:scale-110"
              width="400"
              height="256"
              loading="lazy"
              decoding="async"
            />
            <div className="absolute inset-0 bg-gradient-to-t from-text/70 via-
transparent to-transparent opacity-0 group-hover:opacity-100 transition-
opacity duration-300 flex items-end p-5">
              <div>
                <p className="text-white font-heading font-bold text-
lg">{item.name}</p>
                <p className="text-white/70 text-sm">{item.breed}</p>
              </div>
            </div>
          </div>
        ))}
      </div>
    </div>

    {/* Lightbox */}
    {lightbox && (
     <div
       className="lightbox-overlay active"
       role="dialog"
       aria-modal="true"
       onClick={(e) => e.target === e.currentTarget && closeLightbox()}
     >
       <button
         onClick={closeLightbox}
         className="absolute top-6 right-6 text-white text-4xl hover:text-
accent transition-colors z-10"
         aria-label="Close lightbox"
       >
         &times;
       </button>
       <div className="text-center">
```

```
          <img src={lightbox.src} alt={lightbox.name} className="rounded-
2xl" />
          <p className="text-white font-heading font-bold text-xl mt-
4">{lightbox.name}</p>
          <p className="text-white/60 text-sm">{lightbox.breed}</p>
         </div>
        </div>
      )}
    </section>
  );
}
```

islands/NavbarIsland.jsx

```
import { useState, useEffect } from 'react';
import { NAV_LINKS } from '../../data/navigation.js';

export default function NavbarIsland() {
  const [isOpen, setIsOpen] = useState(false);
  const [shrink, setShrink] = useState(false);

  useEffect(() => {
    const handleScroll = () => setShrink(window.scrollY > 50);
    window.addEventListener('scroll', handleScroll, { passive: true });
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  const closeMobileMenu = () => setIsOpen(false);

  return (
    <nav
      id="main-nav"
      className={`fixed top-0 left-0 right-0 z-50 glass-nav transition-all
duration-300 ${shrink ? 'nav-shrink py-3' : 'py-5'}`}
    >
      <div className="max-w-7xl mx-auto px-6 lg:px-8 flex items-center
justify-between">
        {/* Logo */}
        <a href="#" className="text-2xl font-heading font-bold tracking-tight
text-primary">
          🐾 PawCare
```

```
      </a>

      {/* Desktop Nav */}
      <div className="hidden lg:flex items-center gap-8">
        {NAV_LINKS.map((link) => (
          <a
            key={link.href}
            href={link.href}
            className="text-sm font-medium text-text/60 hover:text-primary
transition-colors duration-300"
          >
            {link.label}
          </a>
        ))}
        <a
          href="#contact"
          className="px-6 py-2.5 bg-primary text-white text-sm font-semibold
rounded-full hover:bg-primary-dark transition-colors duration-300"
        >
          Book Now
        </a>
      </div>

      {/* Mobile Hamburger */}
      <button
        onClick={() => setIsOpen(!isOpen)}
        className="lg:hidden flex flex-col gap-1.5 p-2"
        aria-label="Toggle menu"
        aria-expanded={isOpen}
      >
        <span
          className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? 'rotate-45 translate-y-2' : ''}`}
        />
        <span
          className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? 'opacity-0' : ''}`}
        />
        <span
          className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? '-rotate-45 -translate-y-2' : ''}`}
        />
```

```
      </button>
    </div>

    {/* Mobile Menu */}
    <div
      className={`lg:hidden overflow-hidden transition-all duration-500 ease-in-out ${isOpen ? 'max-h-96 opacity-100' : 'max-h-0 opacity-0'}`}
    >
      <div className="px-6 pb-6 pt-4 flex flex-col gap-4">
        {NAV_LINKS.map((link) => (
          <a
            key={link.href}
            href={link.href}
            onClick={closeMobileMenu}
            className="text-sm font-medium text-text/60 hover:text-primary transition-colors duration-300"
          >
            {link.label}
          </a>
        ))}
        <a
          href="#contact"
          onClick={closeMobileMenu}
          className="px-6 py-3 bg-primary text-white text-sm font-semibold rounded-full text-center hover:bg-primary-dark transition-colors duration-300"
        >
          Book Now
        </a>
      </div>
    </div>
  </nav>
 );
}
```

islands/VideoIsland.jsx

```
import { useState } from 'react';

export default function VideoIsland() {
  const [isPlaying, setIsPlaying] = useState(false);
```

```
  const handlePlay = () => {
    setIsPlaying(true);
  };

  return (
    <section id="video" className="py-24 lg:py-32 bg-primary-dark">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        {/* Section Header */}
        <div className="text-center mb-12 animate-on-scroll fade-up">
          <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-secondary mb-4">
            WATCH &amp;amp; DISCOVER
          </p>
          <h2 className="text-4xl lg:text-5xl font-heading font-bold text-white
mb-6">
            Tur Virtual Fasilitas Kami
          </h2>
          <p className="text-white/60 max-w-2xl mx-auto">
            Take a peek inside our world-class facility and see why thousands of
pet owners trust
            PawCare with their beloved companions.
          </p>
        </div>

        {/* Video Player */}
        <div className="animate-on-scroll fade-up max-w-4xl mx-auto">
          <div
            className="relative w-full rounded-3xl overflow-hidden bg-text/50
shadow-2xl"
            style={{ paddingBottom: '56.25%' }}
          >
            {/* Thumbnail overlay — shown until play */}
            {!isPlaying && (
              <button
                onClick={handlePlay}
                className="absolute inset-0 w-full h-full cursor-pointer group z-10"
                aria-label="Play video tour"
              >
                <img
                  src="/assets/video-thumbnail.png"
                  alt="PawCare Facility Tour Video Thumbnail"
                  className="absolute inset-0 w-full h-full object-cover"
```

```
              loading="lazy"
              decoding="async"
            />
            <div className="absolute inset-0 bg-text/40 group-hover:bg-text/30
transition-colors duration-300" />
            <div className="absolute inset-0 flex items-center justify-center">
              <div className="video-play-btn w-24 h-24 rounded-full bg-
white/90 group-hover:bg-white flex items-center justify-center transition-all
duration-300 group-hover:scale-110 shadow-2xl">
                <svg
                  xmlns="http://www.w3.org/2000/svg"
                  viewBox="0 0 24 24"
                  fill="#2D6A4F"
                  className="w-10 h-10 ml-1"
                >
                  <polygon points="5,3 19,12 5,21" />
                </svg>
              </div>
            </div>
          </button>
        )}

        {/* Video element — shown after play */}
        {isPlaying && (
          <video
            className="absolute inset-0 w-full h-full object-cover"
            controls
            autoPlay
            controlsList="nodownload"
            title="PawCare Virtual Tour"
          >
            <source src="/assets/video-embed/video-tour.mp4"
type="video/mp4" />
          </video>
        )}
      </div>
    </div>
  </section>
 );
}
```

islands/VirtualTourIsland.jsx

```
import { useState } from 'react';
import { VIRTUAL_TOUR } from '../../data/virtualTour.js';

export default function VirtualTourIsland() {
  const [activeIndex, setActiveIndex] = useState(0);
  const [fading, setFading] = useState(false);

  const activeFacility = VIRTUAL_TOUR.facilities[activeIndex];

  const selectFacility = (index) => {
    if (index === activeIndex) return;
    setFading(true);
    setTimeout(() => {
      setActiveIndex(index);
      setFading(false);
    }, 200);
  };

  return (
    <section id="virtual-tour" className="py-24 lg:py-32">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        {/* Section Header */}
        <div className="text-center mb-12 animate-on-scroll fade-up">
          <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">
            {VIRTUAL_TOUR.subtitle}
          </p>
          <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">
            {VIRTUAL_TOUR.title}
          </h2>
          <p className="text-text-light max-w-2xl mx-
auto">{VIRTUAL_TOUR.description}</p>
        </div>

        {/* Main Image Display */}
        <div className="animate-on-scroll fade-up">
          <div className="rounded-3xl overflow-hidden mb-6 bg-gray-100">
            <img
              src={activeFacility.image}
```

```
            alt={activeFacility.title}
            className="w-full h-[500px] lg:h-[600px] object-cover"
            style={{ opacity: fading ? 0 : 1, transition: 'opacity 0.2s ease-in-out'
}}
            width="800"
            height="600"
            loading="lazy"
            decoding="async"
          />
        </div>
        <p className="text-xl font-heading font-bold text-text mb-
2">{activeFacility.title}</p>
        <p className="text-text-light">{activeFacility.description}</p>
      </div>

      {/* Thumbnails */}
      <div
        className="grid grid-cols-5 gap-4 mt-6 animate-on-scroll fade-up"
        style={{ animationDelay: '200ms' }}
      >
        {VIRTUAL_TOUR.facilities.map((facility, index) => (
          <button
            key={index}
            onClick={() => selectFacility(index)}
            aria-label={`View ${facility.title}`}
            className={`tour-thumb rounded-xl overflow-hidden border-2
transition-all duration-300 ${
              activeIndex === index ? 'border-primary' : 'border-transparent'
            }`}
          >
            <img
              src={facility.image}
              alt={facility.title}
              className="w-full h-20 sm:h-24 lg:h-28 object-cover"
              width="200"
              height="112"
              loading="lazy"
              decoding="async"
            />
          </button>
        ))}
      </div>
```

```
      </div>
    </section>
  );
}
```

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: {
    minify: false,
  }
});
```

src/main.jsx (entry point)

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

src/App.jsx

```
import { useEffect } from 'react';
import Navbar from './components/Navbar.jsx';
import Hero from './components/Hero.jsx';
import Services from './components/Services.jsx';
import Gallery from './components/Gallery.jsx';
import Video from './components/Video.jsx';
import Team from './components/Team.jsx';
```

```
import Testimonials from './components/Testimonials.jsx';
import Stats from './components/Stats.jsx';
import VirtualTour from './components/VirtualTour.jsx';
import LocationMap from './components/LocationMap.jsx';
import FAQ from './components/FAQ.jsx';
import Footer from './components/Footer.jsx';

function App() {
 useEffect(() => {
  const observer = new IntersectionObserver((entries) => {
   entries.forEach((entry) => {
    if (entry.isIntersecting) {
     entry.target.classList.add('is-visible');
     observer.unobserve(entry.target);
    }
   });
  }, { threshold: 0.1, rootMargin: '0px 0px -50px 0px' });

  document.querySelectorAll('.animate-on-scroll').forEach((el) => {
   observer.observe(el);
  });

  return () => observer.disconnect();
 }, []);

 return (
  <>
   <Navbar />
   <main>
    <Hero />
    <Services />
    <Gallery />
    <Video />
    <Team />
    <Testimonials />
    <Stats />
    <VirtualTour />
    <LocationMap />
    <FAQ />
   </main>
   <Footer />
  </>
```

```
  );
}

export default App;
```

Navbar.jsx

```
import { useState, useEffect } from 'react';
import { NAV_LINKS } from '../data/navigation.js';

function Navbar() {
  const [isOpen, setIsOpen] = useState(false);
  const [scrolled, setScrolled] = useState(false);

  useEffect(() => {
    const handleScroll = () => setScrolled(window.scrollY > 50);
    window.addEventListener('scroll', handleScroll);
    return () => window.removeEventListener('scroll', handleScroll);
  }, []);

  const closeMenu = () => setIsOpen(false);

  return (
    <nav id="main-nav" className={`fixed top-0 left-0 right-0 z-50 glass-nav
transition-all duration-300 ${scrolled ? 'nav-shrink' : 'py-5'}`}>
      <div className="max-w-7xl mx-auto px-6 lg:px-8 flex items-center
justify-between">
        <a href="#" className="text-2xl font-heading font-bold tracking-tight
text-primary">🐾 PawCare</a>

        <div className="hidden lg:flex items-center gap-8">
          {NAV_LINKS.map((link) => (
            <a key={link.label} href={link.href} className="text-sm font-
medium text-text/60 hover:text-primary transition-colors duration-
300">{link.label}</a>
          ))}
          <a href="#contact" className="px-6 py-2.5 bg-primary text-white text-
sm font-semibold rounded-full hover:bg-primary-dark transition-colors
duration-300">Book Now</a>
        </div>
```

```
      <button onClick={() => setIsOpen(!isOpen)} className="lg:hidden flex
flex-col gap-1.5 p-2" aria-label="Toggle menu">
        <span className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? 'rotate-45 translate-y-2' : ''}`}></span>
        <span className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? 'opacity-0' : ''}`}></span>
        <span className={`block w-6 h-0.5 bg-text transition-all duration-300
${isOpen ? '-rotate-45 -translate-y-2' : ''}`}></span>
      </button>
    </div>

    <div className={`lg:hidden overflow-hidden transition-all duration-500
ease-in-out ${isOpen ? 'max-h-96 opacity-100' : 'max-h-0 opacity-0'}`}>
      <div className="px-6 pb-6 pt-4 flex flex-col gap-4">
        {NAV_LINKS.map((link) => (
          <a key={link.label} href={link.href} onClick={closeMenu}
className="text-sm font-medium text-text/60 hover:text-primary transition-
colors duration-300">{link.label}</a>
        ))}
        <a href="#contact" className="px-6 py-3 bg-primary text-white text-
sm font-semibold rounded-full text-center hover:bg-primary-dark transition-
colors duration-300">Book Now</a>
      </div>
    </div>
  </nav>
 );
}

export default Navbar;
```

Hero.jsx

```
import { HERO_DATA } from '../data/hero.js';


function Hero() {

 return (

  <section id="hero" className="relative min-h-screen flex items-center
overflow-hidden">
```

```
    <div className="absolute inset-0">
      <img src={HERO_DATA.image} alt="Happy pets at PawCare clinic" className="w-full h-full object-cover" width="1920" height="1080" loading="eager" fetchpriority="high" decoding="async" />
      <div className="absolute inset-0 bg-gradient-to-r from-text/80 via-text/60 to-transparent"></div>
    </div>

    <div className="relative z-10 max-w-7xl mx-auto px-6 lg:px-8 py-32 lg:py-40 w-full">
      <div className="max-w-2xl">
        <p className="hero-animate text-sm font-accent font-semibold uppercase tracking-[0.3em] text-secondary mb-6">{HERO_DATA.subtitle}</p>
        <h1 className="hero-animate-delay-1 text-5xl sm:text-6xl lg:text-7xl font-heading font-bold leading-[1.1] text-white mb-8">{HERO_DATA.title}</h1>
        <p className="hero-animate-delay-2 text-lg text-white/70 leading-relaxed max-w-lg mb-10 font-body">{HERO_DATA.description}</p>

        <div className="hero-animate-delay-3 flex flex-wrap gap-4">
          <a href="#contact" className="cta-pulse px-8 py-4 bg-primary text-white text-sm font-semibold rounded-full hover:bg-primary-dark transition-colors duration-300">{HERO_DATA.cta}</a>
          <a href="#gallery" className="px-8 py-4 border-2 border-white/30 text-white text-sm font-semibold rounded-full hover:border-white/60 hover:bg-white/10 transition-all duration-300">{HERO_DATA.ctaSecondary}</a>
        </div>
      </div>

      <div className="mt-16 grid grid-cols-2 md:grid-cols-4 gap-8">
```

```
        {HERO_DATA.counters.map((counter, index) => (
          <div key={counter.label} className="hero-animate-delay-3 text-
center md:text-left" style={{ animationDelay: `${0.6 + index * 0.15}s` }}>
            <p className="text-3xl lg:text-4xl font-heading font-bold text-white
mb-1">{counter.value}</p>
            <p className="text-sm text-white/50 font-
body">{counter.label}</p>
          </div>
        ))}
      </div>
    </div>
  </section>
 );
}

export default Hero;
```

Stats.jsx

```
import { STATS, STATS_BG } from '../data/stats.js';

function Stats() {
 return (
  <section className="relative py-24 lg:py-32 overflow-hidden">
    <div className="parallax-bg absolute inset-0" style={{
backgroundImage: `url('${STATS_BG}')` }}></div>
    <div className="absolute inset-0 bg-primary-dark/85"></div>

    <div className="relative z-10 max-w-7xl mx-auto px-6 lg:px-8">
      <div className="text-center mb-12 animate-on-scroll fade-up">
        <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-secondary mb-4">BY THE NUMBERS</p>
        <h2 className="text-4xl lg:text-5xl font-heading font-bold text-
white">Trusted by Thousands</h2>
```

```
      </div>

      <div className="grid grid-cols-2 lg:grid-cols-4 gap-8 lg:gap-12">
        {STATS.map((stat, index) => (
          <div key={stat.label} className="text-center animate-on-scroll fade-
up" style={{ animationDelay: `${index * 150}ms` }}>
            <p className="text-5xl lg:text-6xl font-heading font-bold text-white
mb-2">{stat.value}</p>
            <p className="text-sm font-accent font-medium uppercase tracking-
[0.15em] text-white/60">{stat.label}</p>
          </div>
        ))}
      </div>
    </div>
  </section>
 );
}

export default Stats;
```

Services.jsx

```
import { SERVICES } from '../data/services.js';

const icons = {
 stethoscope: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24
24" fill="none" stroke="currentColor" strokeWidth="1.5"
strokeLinecap="round" strokeLinejoin="round" className="w-7 h-7"><path
d="M4.8 2.3A.3.3 0 105 2H4a2 2 0 00-2 2v5a6 6 0 006 6 6 6 0 006-6V4a2 2 0
00-2-2h-1a.2.2 0 10.3.3" /><path d="M8 15v1a6 6 0 006 6 6 6 0 006-6v-4"
/><circle cx="20" cy="10" r="2" /></svg>,
 vaccine: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M15 2l5 5-5 5"
/><path d="M20 7H9.5a3.5 3.5 0 000 7h.5" /><path d="M12 22v-5" /><path
d="M8 22h8" /><path d="M12 17a2 2 0 002-2v-1H10v1a2 2 0 002 2z"
/></svg>,
 surgery: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M12.22 2h-.44a2 2 0
00-2 2v.18a2 2 0 01-1 1.73l-.43.25a2 2 0 01-2 0l-.15-.08a2 2 0 00-2.73.73l-
```

```
.22.38a2 2 0 00.73 2.73l.15.1a2 2 0 011 1.72v.51a2 2 0 01-1 1.74l-.15.09a2 2 0
00-.73 2.73l.22.38a2 2 0 002.73.73l.15-.08a2 2 0 012 0l.43.25a2 2 0 011
1.73V20a2 2 0 002 2h.44a2 2 0 002-2v-.18a2 2 0 011-1.73l.43-.25a2 2 0 012
0l.15.08a2 2 0 002.73-.73l.22-.39a2 2 0 00-.73-2.73l-.15-.08a2 2 0 01-1-1.74v-
.5a2 2 0 011-1.74l.15-.09a2 2 0 00.73-2.73l-.22-.38a2 2 0 00-2.73-.73l-
.15.08a2 2 0 01-2 0l-.43-.25a2 2 0 01-1-1.73V4a2 2 0 00-2-2z" /><circle
cx="12" cy="12" r="3" /></svg>,
  dental: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M12 5.5C10 3 7 2
5.5 3.5S3 8 5 12c1.5 3 2.5 5 3 7 .5 1.5 1 2.5 2 2.5s1.5-.5 2-2c.5 1.5 1 2 2 2s1.5-
1 2-2.5c.5-2 1.5-4 3-7 2-4 1-7-.5-8.5S14 3 12 5.5z" /></svg>,
  grooming: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M7 20h10" /><path
d="M10 20c5.5-2.5.8-6.4 3-10" /><path d="M9.5 9.4c1.1.8 1.8 2.2 2.3 3.7-2
.4-3.5.4-4.8-.3-1.2-.6-2.3-1.9-3-4.2 2.8-.5 4.4 0 5.5.8z" /><path d="M14.1 6a7
7 0 00-1.1 4c1.9-.1 3.3-.6 4.3-1.4 1-1 1.6-2.3 1.7-4.6-2.7.1-4 1-4.9 2z"
/></svg>,
  emergency: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24
24" fill="none" stroke="currentColor" strokeWidth="1.5"
strokeLinecap="round" strokeLinejoin="round" className="w-7 h-7"><path
d="M8.5 14.5A2.5 2.5 0 0011 12c0-1.38-.5-2-1-3-1.07-2.14-1.5-3.5-1-5 .5 1.5
2 2.5 3 3 1.5 1 2 3.5 2 5.5a3 3 0 01-6 0" /><path d="M12 22v-3" /><path
d="M4 22h16" /></svg>,
  nutrition: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M18 8h1a4 4 0 010
8h-1" /><path d="M2 8h16v9a4 4 0 01-4 4H6a4 4 0 01-4-4V8z" /><line
x1="6" y1="1" x2="6" y2="4" /><line x1="10" y1="1" x2="10" y2="4"
/><line x1="14" y1="1" x2="14" y2="4" /></svg>,
  boarding: <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round" className="w-7 h-7"><path d="M3 9l9-7 9 7v11a2 2
0 01-2 2H5a2 2 0 01-2-2z" /><polyline points="9,22 9,12 15,12 15,22"
/></svg>,
};

function Services() {
  return (
    <section id="services" className="py-24 lg:py-32">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
```

```
      <div className="text-center mb-16 animate-on-scroll fade-up">
        <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">WHAT WE OFFER</p>
        <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">Our Services</h2>
        <p className="text-text-light max-w-2xl mx-auto">From routine
check-ups to emergency care, we provide comprehensive veterinary services
tailored to your pet's unique needs.</p>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
        {SERVICES.map((service, index) => (
          <div key={service.title} className="service-card bg-white rounded-
3xl overflow-hidden border border-gray-100 animate-on-scroll fade-up"
style={{ animationDelay: `${index * 80}ms` }}>
            <div className="h-44 overflow-hidden">
              <img src={service.thumbnail} alt={service.title} className="w-full
h-full object-cover transition-transform duration-500 hover:scale-110"
width="400" height="176" loading="lazy" decoding="async" />
            </div>
            <div className="p-6">
              <div className="w-12 h-12 rounded-xl bg-secondary/15 flex items-
center justify-center text-primary mb-4">{icons[service.icon]}</div>
              <h3 className="text-lg font-heading font-bold text-text mb-
2">{service.title}</h3>
              <p className="text-sm text-text-light leading-
relaxed">{service.description}</p>
            </div>
          </div>
        ))}
      </div>
    </div>
  </section>
 );
}

export default Services;
```

Gallery.jsx

```
import { useState } from 'react';
import { GALLERY_ITEMS } from '../data/gallery_items.js';
import { GALLERY_FILTERS } from '../data/gallery_filters.js';

function Gallery() {
  const [activeFilter, setActiveFilter] = useState('All');
  const [lightbox, setLightbox] = useState(null);

  const filtered = activeFilter === 'All' ? GALLERY_ITEMS :
GALLERY_ITEMS.filter(i => i.category === activeFilter);

  return (
    <section id="gallery" className="py-24 lg:py-32 bg-white">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        <div className="text-center mb-12 animate-on-scroll fade-up">
          <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">OUR FRIENDS</p>
          <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">Pet Gallery</h2>
          <p className="text-text-light max-w-2xl mx-auto mb-8">Meet some of
the adorable pets we've had the pleasure of caring for at PawCare.</p>
          <div className="flex flex-wrap justify-center gap-3">
            {GALLERY_FILTERS.map(filter => (
              <button key={filter} onClick={() => setActiveFilter(filter)}
className={`px-6 py-2.5 rounded-full text-sm font-semibold transition-all
duration-300 ${activeFilter === filter ? 'bg-primary text-white' : 'bg-gray-100
text-text-light hover:bg-gray-200'}`}>{filter}</button>
            ))}
          </div>
        </div>

        <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 gap-4">
          {filtered.map((item, index) => (
            <div key={item.name} className="gallery-item group rounded-2xl
overflow-hidden relative cursor-pointer animate-on-scroll zoom-in" style={{
animationDelay: `${index * 60}ms` }} onClick={() => setLightbox(item)}>
              <img src={item.src} alt={`${item.name} - ${item.breed}`}
className="w-full h-64 object-cover transition-transform duration-500 group-
hover:scale-110" width="400" height="256" loading="lazy" decoding="async"
/>
```

```
          <div className="absolute inset-0 bg-gradient-to-t from-text/70 via-transparent to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-300 flex items-end p-5">
            <div>
              <p className="text-white font-heading font-bold text-lg">{item.name}</p>
              <p className="text-white/70 text-sm">{item.breed}</p>
            </div>
          </div>
        </div>
      ))}
    </div>
  </div>

  {lightbox && (
    <div className="lightbox-overlay active" onClick={() => setLightbox(null)}>
      <button className="absolute top-6 right-6 text-white text-4xl hover:text-accent transition-colors z-10" onClick={() => setLightbox(null)}>&times;</button>
      <div className="text-center" onClick={e => e.stopPropagation()}>
        <img src={lightbox.src} alt={lightbox.name} className="rounded-2xl max-w-[90vw] max-h-[85vh] object-contain" />
        <p className="text-white font-heading font-bold text-xl mt-4">{lightbox.name}</p>
        <p className="text-white/60 text-sm">{lightbox.breed}</p>
      </div>
    </div>
  )}
 </section>
 );
}

export default Gallery;
```

VirtualTour.jsx

```
import { useState } from 'react';
import { VIRTUAL_TOUR } from '../data/virtualTour.js';

function VirtualTour() {
```

```
  const [activeIndex, setActiveIndex] = useState(0);
  const [fading, setFading] = useState(false);

  const handleThumbClick = (index) => {
    setFading(true);
    setTimeout(() => {
      setActiveIndex(index);
      setFading(false);
    }, 200);
  };

  const active = VIRTUAL_TOUR.facilities[activeIndex];

  return (
    <section id="virtual-tour" className="py-24 lg:py-32">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        <div className="text-center mb-12 animate-on-scroll fade-up">
          <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">{VIRTUAL_TOUR.subtitle}</p>
          <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">{VIRTUAL_TOUR.title}</h2>
          <p className="text-text-light max-w-2xl mx-
auto">{VIRTUAL_TOUR.description}</p>
        </div>

        <div className="animate-on-scroll fade-up">
          <div className="rounded-3xl overflow-hidden mb-6 bg-gray-100">
            <img src={active.image} alt={active.title} className={`w-full h-
[500px] lg:h-[600px] object-cover transition-opacity duration-300 ${fading ?
'opacity-0' : 'opacity-100'}`} width="800" height="600" loading="lazy"
decoding="async" />
          </div>
          <p className="text-xl font-heading font-bold text-text mb-
2">{active.title}</p>
          <p className="text-text-light">{active.description}</p>
        </div>

        <div className="grid grid-cols-5 gap-4 mt-6 animate-on-scroll fade-up"
style={{ animationDelay: '200ms' }}>
          {VIRTUAL_TOUR.facilities.map((facility, index) => (
            <button key={facility.title} onClick={() =>
handleThumbClick(index)} className={`tour-thumb rounded-xl overflow-
```

```
hidden border-2 ${activeIndex === index ? 'active border-primary' : 'border-transparent'}`} aria-label={`View ${facility.title}`}>
            <img src={facility.image} alt={facility.title} className="w-full h-20 sm:h-24 lg:h-28 object-cover" width="200" height="112" loading="lazy" decoding="async" />
          </button>
        ))}
      </div>
    </div>
  </section>
 );
}

export default VirtualTour;
```

Video.jsx

```
import { useState } from 'react';

function Video() {
 const [playing, setPlaying] = useState(false);

 return (
  <section id="video" className="py-24 lg:py-32 bg-primary-dark">
   <div className="max-w-7xl mx-auto px-6 lg:px-8">
    <div className="text-center mb-12 animate-on-scroll fade-up">
     <p className="text-sm font-accent font-semibold uppercase tracking-[0.3em] text-secondary mb-4">WATCH & DISCOVER</p>
     <h2 className="text-4xl lg:text-5xl font-heading font-bold text-white mb-6">Tur Virtual Fasilitas Kami</h2>
     <p className="text-white/60 max-w-2xl mx-auto">Take a peek inside our world-class facility and see why thousands of pet owners trust PawCare with their beloved companions.</p>
    </div>

    <div className="animate-on-scroll fade-up max-w-4xl mx-auto">
     <div className="relative w-full rounded-3xl overflow-hidden bg-text/50 shadow-2xl" style={{ paddingBottom: '56.25%' }}>
      {!playing && (
       <button onClick={() => setPlaying(true)} className="absolute inset-0 w-full h-full cursor-pointer group z-10">
```

```
            <img src="/assets/video-thumbnail.png" alt="PawCare Facility
Tour" className="absolute inset-0 w-full h-full object-cover" loading="lazy"
decoding="async" />
            <div className="absolute inset-0 bg-text/40 group-hover:bg-text/30
transition-colors duration-300"></div>
            <div className="absolute inset-0 flex items-center justify-center">
              <div className="video-play-btn w-24 h-24 rounded-full bg-
white/90 group-hover:bg-white flex items-center justify-center transition-all
duration-300 group-hover:scale-110 shadow-2xl">
                <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="#2D6A4F" className="w-10 h-10 ml-1"><polygon points="5,3 19,12
5,21" /></svg>
              </div>
            </div>
          </button>
        )}
        {playing && (
          <video src="/assets/video-embed/video-tour.mp4" title="PawCare
Virtual Tour" className="absolute inset-0 w-full h-full object-cover"
autoPlay controls controlsList="nodownload"></video>
        )}
      </div>
    </div>
  </section>
 );
}

export default Video;
```

Team.jsx

```
import { TEAM_MEMBERS } from '../data/team.js';

function Team() {
 return (
  <section id="team" className="py-24 lg:py-32">
   <div className="max-w-7xl mx-auto px-6 lg:px-8">
    <div className="text-center mb-16 animate-on-scroll fade-up">
     <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">MEET THE EXPERTS</p>
```

```
        <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text mb-6">Our Team</h2>
        <p className="text-text-light max-w-2xl mx-auto">Our dedicated team of veterinary professionals brings decades of combined experience and a genuine passion for animal care.</p>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-8">
        {TEAM_MEMBERS.map((member, index) => (
          <div key={member.name} className="team-card bg-white rounded-3xl overflow-hidden border border-gray-100 animate-on-scroll fade-up" style={{ animationDelay: `${index * 100}ms` }}>
            <div className="h-72 overflow-hidden">
              <img src={member.photo} alt={member.name} className="w-full h-full object-cover transition-transform duration-500 hover:scale-105" width="400" height="288" loading="lazy" decoding="async" />
            </div>
            <div className="p-6">
              <h3 className="text-xl font-heading font-bold text-text mb-1">{member.name}</h3>
              <p className="text-primary font-semibold text-sm mb-2">{member.role}</p>
              <p className="text-text-light text-sm mb-1"><span className="font-medium">Specialization:</span> {member.specialization}</p>
              <p className="text-text-light text-sm mb-4"><span className="font-medium">Experience:</span> {member.experience}</p>
              <div className="flex gap-3">
                <a href={member.social.linkedin} className="w-9 h-9 rounded-full bg-primary/10 flex items-center justify-center text-primary hover:bg-primary hover:text-white transition-all duration-300" aria-label={`${member.name} LinkedIn`}>
                  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" className="w-4 h-4"><path d="M16 8a6 6 0 016 6v7h-4v-7a2 2 0 00-2-2 2 2 0 00-2 2v7h-4v-7a6 6 0 016-6z"/><rect x="2" y="9" width="4" height="12"/><circle cx="4" cy="4" r="2"/></svg>
                </a>
                <a href={member.social.twitter} className="w-9 h-9 rounded-full bg-primary/10 flex items-center justify-center text-primary hover:bg-primary hover:text-white transition-all duration-300" aria-label={`${member.name} Twitter`}>
```

```
                <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" className="w-4 h-
4"><path d="M23 3a10.9 10.9 0 01-3.14 1.53 4.48 4.48 0 00-7.86 3v1A10.66
10.66 0 013 4s-4 9 5 13a11.64 11.64 0 01-7 2c9 5 20 0 20-11.5a4.5 4.5 0 00-
.08-.83A7.72 7.72 0 0023 3z"/></svg>
              </a>
              <a href={`mailto:${member.social.email}`} className="w-9 h-9
rounded-full bg-primary/10 flex items-center justify-center text-primary
hover:bg-primary hover:text-white transition-all duration-300" aria-
label={`Email ${member.name}`}>
                <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" className="w-4 h-
4"><path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-2-
2V6c0-1.1.9-2 2-2z"/><polyline points="22,6 12,13 2,6"/></svg>
              </a>
            </div>
          </div>
        </div>
      ))}
    </div>
  </section>
 );
}

export default Team;
```

Testimonials.jsx

```
import { TESTIMONIALS } from '../data/testimonials.js';

function Testimonials() {
 return (
  <section id="testimonials" className="py-24 lg:py-32 bg-white overflow-
hidden">
    <div className="max-w-7xl mx-auto px-6 lg:px-8">
      <div className="text-center mb-16 animate-on-scroll fade-up">
        <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">TESTIMONIALS</p>
        <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">What Pet Owners Say</h2>
```

```
        <p className="text-text-light max-w-2xl mx-auto">Don't just take our word for it — hear from the pet owners who trust us with their beloved companions.</p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8 animate-on-scroll fade-up">
        {TESTIMONIALS.map((item) => (
          <div key={item.name} className="bg-background rounded-3xl p-8 border border-gray-100 h-full hover:-translate-y-2 hover:shadow-xl hover:shadow-secondary/20 hover:border-secondary/30 transition-all duration-300 cursor-pointer flex flex-col">
            <div className="flex gap-1 mb-6">
              {Array.from({ length: item.rating }).map((_, i) => (
                <svg key={i} xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="currentColor" className="w-5 h-5 text-accent">
                  <path d="M12 2l3.09 6.26L22 9.27l-5 4.87 1.18 6.88L12 17.77l-6.18 3.25L7 14.14 2 9.27l6.91-1.01L12 2z" />
                </svg>
              ))}
            </div>
            <p className="text-text-light leading-relaxed mb-8 flex-grow">&ldquo;{item.quote}&rdquo;</p>
            <div className="flex items-center gap-4">
              <img src={item.avatar} alt={item.name} className="w-14 h-14 rounded-full object-cover border-2 border-secondary/30" width="56" height="56" loading="lazy" decoding="async" />
              <div>
                <p className="font-heading font-bold text-text">{item.name}</p>
                <p className="text-sm text-text-light">{item.role}</p>
              </div>
            </div>
          </div>
        ))}
      </div>
    </div>
  </section>
 );
}

export default Testimonials;
```

FAQ.jsx

```
import { useState } from 'react';
import { FAQ_ITEMS } from '../data/faq_items.js';

function FAQ() {
  const [openIndex, setOpenIndex] = useState(null);

  const toggle = (index) => {
    setOpenIndex(openIndex === index ? null : index);
  };

  return (
    <section id="faq" className="py-24 lg:py-32">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        <div className="grid grid-cols-1 lg:grid-cols-2 gap-16 lg:gap-24">
          <div className="animate-on-scroll fade-right">
            <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">FAQ</p>
            <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">Frequently Asked Questions</h2>
            <p className="text-text-light leading-relaxed">Everything you need
to know about our services. Can&apos;t find the answer you&apos;re looking
for? Feel free to contact our friendly team.</p>
          </div>

          <div className="animate-on-scroll fade-left" style={{ animationDelay:
'200ms' }}>
            {FAQ_ITEMS.map((item, index) => (
              <div key={index} className="faq-item border-b border-gray-200">
                <button className="faq-toggle w-full flex items-center justify-
between py-6 text-left" onClick={() => toggle(index)}>
                  <span className="text-base font-semibold text-text pr-
4">{item.question}</span>
                  <span className={`faq-icon text-primary flex-shrink-0
${openIndex === index ? 'open' : ''}`}>
                    <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" className="w-5 h-5">
                      <line x1="12" y1="5" x2="12" y2="19" />
                      <line x1="5" y1="12" x2="19" y2="12" />
                    </svg>
                  </span>
```

```
            </button>
            <div className={`faq-content ${openIndex === index ? 'open' :
"}`}>
              <p className="pb-6 text-sm text-text-light leading-
relaxed">{item.answer}</p>
            </div>
          </div>
        ))}
      </div>
    </div>
  </section>
 );
}

export default FAQ;
```

LocationMap.jsx

```
import { LOCATION_MAP } from '../data/location.js';

function LocationMap() {
 return (
  <section id="contact" className="py-24 lg:py-32 bg-white">
   <div className="max-w-7xl mx-auto px-6 lg:px-8">
    <div className="text-center mb-16 animate-on-scroll fade-up">
     <p className="text-sm font-accent font-semibold uppercase tracking-
[0.3em] text-primary mb-4">GET IN TOUCH</p>
     <h2 className="text-4xl lg:text-5xl font-heading font-bold text-text
mb-6">Visit Our Clinic</h2>
     <p className="text-text-light max-w-2xl mx-
auto">{LOCATION_MAP.description}</p>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
     <div className="animate-on-scroll fade-right">
      <div className="relative w-full rounded-3xl overflow-hidden bg-
gray-100 border border-gray-100" style={{ paddingBottom: '75%' }}>
       <iframe src={LOCATION_MAP.mapEmbed} title="PawCare Clinic
Location" className="absolute inset-0 w-full h-full" style={{ border: 0 }}
```

```
allowFullScreen loading="lazy" referrerPolicy="no-referrer-when-downgrade"></iframe>
          </div>
        </div>

        <div className="animate-on-scroll fade-left">
          <form className="bg-background rounded-3xl p-8 border border-gray-100 mb-6">
            <h3 className="text-xl font-heading font-bold text-text mb-6">Send us a Message</h3>
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-4">
              <input type="text" placeholder="Your Name" className="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm" id="contact-name" />
              <input type="email" placeholder="Your Email" className="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm" id="contact-email" />
            </div>
            <input type="text" placeholder="Subject" className="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm mb-4" id="contact-subject" />
            <textarea placeholder="Your Message" rows="4" className="form-input w-full px-4 py-3 rounded-xl border border-gray-200 bg-white text-sm mb-4 resize-none" id="contact-message"></textarea>
            <button type="submit" className="w-full px-8 py-4 bg-primary text-white text-sm font-semibold rounded-xl hover:bg-primary-dark transition-colors duration-300">Send Message</button>
          </form>

          <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
            <div className="bg-background rounded-2xl p-5 border border-gray-100 text-center">
              <div className="w-10 h-10 rounded-xl bg-secondary/15 flex items-center justify-center text-primary mx-auto mb-3">
                <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5" className="w-5 h-5"><path d="M21 10c0 7-9 13-9 13s-9-6-9-13a9 9 0 0118 0z"/><circle cx="12" cy="10" r="3"/></svg>
              </div>
              <p className="text-xs font-semibold text-text-light uppercase tracking-wider mb-1">Address</p>
```

```
            <p className="text-sm text-text font-
medium">{LOCATION_MAP.address}</p>
          </div>
          <div className="bg-background rounded-2xl p-5 border border-
gray-100 text-center">
            <div className="w-10 h-10 rounded-xl bg-secondary/15 flex items-
center justify-center text-primary mx-auto mb-3">
              <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" className="w-5 h-
5"><path d="M22 16.92v3a2 2 0 01-2.18 2 19.79 19.79 0 01-8.63-3.07 19.5
19.5 0 01-6-6A19.79 19.79 0 013.12 4.18 2 2 0 014.11 2h3a2 2 0 012
1.72c.127.96.361 1.903.7 2.81a2 2 0 01-.45 2.11L8.09 9.91a16 16 0 006
6l1.27-1.27a2 2 0 012.11-.45c.907.339 1.85.573 2.81.7A2 2 0 0122
16.92z"/></svg>
            </div>
            <p className="text-xs font-semibold text-text-light uppercase
tracking-wider mb-1">Phone</p>
            <p className="text-sm text-text font-
medium">{LOCATION_MAP.phone}</p>
          </div>
          <div className="bg-background rounded-2xl p-5 border border-
gray-100 text-center">
            <div className="w-10 h-10 rounded-xl bg-secondary/15 flex items-
center justify-center text-primary mx-auto mb-3">
              <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="1.5" className="w-5 h-
5"><path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-1.1 0-2-.9-2-
2V6c0-1.1.9-2 2-2z"/><polyline points="22,6 12,13 2,6"/></svg>
            </div>
            <p className="text-xs font-semibold text-text-light uppercase
tracking-wider mb-1">Email</p>
            <p className="text-sm text-text font-
medium">{LOCATION_MAP.email}</p>
          </div>
        </div>
      </div>
    </div>
  </section>
 );
}
```

```
export default LocationMap;
```

Footer.jsx

```
import { FOOTER_DATA } from '../data/footer.js';

function Footer() {
  return (
    <footer className="bg-text py-20">
      <div className="max-w-7xl mx-auto px-6 lg:px-8">
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-12
mb-16">
          <div className="lg:col-span-1">
            <h3 className="text-2xl font-heading font-bold text-white mb-4">🐾
{FOOTER_DATA.brand}</h3>
            <p className="text-sm text-white/60 leading-relaxed mb-
6">{FOOTER_DATA.tagline}. Providing world-class veterinary care with
warmth and compassion for every furry family member.</p>
            <div className="flex gap-4">
              {FOOTER_DATA.social.map((s) => (
                <a key={s.name} href={s.href} className="w-10 h-10 rounded-full
bg-white/10 flex items-center justify-center text-white/60 hover:bg-primary
hover:text-white transition-all duration-300" aria-label={s.name}>
                  <span className="text-xs font-bold">{s.name.charAt(0)}</span>
                </a>
              ))}
            </div>
          </div>

          <div>
            <h4 className="text-sm font-semibold uppercase tracking-[0.15em]
text-white/40 mb-5">Quick Links</h4>
            <ul className="space-y-3">
              {FOOTER_DATA.quickLinks.map((link) => (
                <li key={link.label}><a href={link.href} className="text-sm text-
white/60 hover:text-secondary transition-colors duration-
300">{link.label}</a></li>
              ))}
            </ul>
          </div>
```

```
        <div>
          <h4 className="text-sm font-semibold uppercase tracking-[0.15em]
text-white/40 mb-5">Services</h4>
          <ul className="space-y-3">
            {FOOTER_DATA.services.map((link) => (
              <li key={link.label}><a href={link.href} className="text-sm text-
white/60 hover:text-secondary transition-colors duration-
300">{link.label}</a></li>
            ))}
          </ul>
        </div>

        <div>
          <h4 className="text-sm font-semibold uppercase tracking-[0.15em]
text-white/40 mb-5">Newsletter</h4>
          <p className="text-sm text-white/60 mb-4">Subscribe for pet care
tips and clinic updates.</p>
          <form className="flex gap-2">
            <input type="email" placeholder="Your email"
className="newsletter-input flex-1 px-4 py-3 rounded-xl bg-white/10 border
border-white/10 text-white text-sm placeholder-white/40" />
            <button type="submit" className="px-5 py-3 bg-primary text-white
text-sm font-semibold rounded-xl hover:bg-secondary transition-colors
duration-300">Subscribe</button>
          </form>
        </div>
      </div>

      <div className="pt-8 border-t border-white/10 flex flex-col sm:flex-row
justify-between items-center gap-4">
        <p className="text-xs text-white/30">&copy; 2026
{FOOTER_DATA.brand}. All rights reserved.</p>
        <div className="flex gap-6">
          <a href="#" className="text-xs text-white/30 hover:text-white/60
transition-colors duration-300">Privacy Policy</a>
          <a href="#" className="text-xs text-white/30 hover:text-white/60
transition-colors duration-300">Terms of Service</a>
        </div>
      </div>
    </div>
  </footer>
 );
```

```
}

export default Footer;
```